

ARTIFICIAL INTELLIGENCE

Machine Learning & Deep Learning

Lecture Notes

Master's Program in Artificial Intelligence

ACADEMIC YEAR

2025–2026

JACOPO PARRETTI

Contents

I	Introduction to Machine Learning	12
1	What is Machine Learning?	12
1.1	Machine Learning vs Statistical Modelling	12
1.2	Conventional Programming vs Machine Learning	12
1.3	The Machine Learning Recipe	12
2	Machine Learning Tasks	13
2.1	Classification Task	13
2.2	Regression Task	13
2.3	Density Estimation Task	13
2.4	Clustering Task	13
2.5	Dimensionality Reduction Task	14
3	Data in Machine Learning	14
3.1	Data Representation	14
3.2	Features	14
4	Data Distributions and Learning Types	14
4.1	Supervised Learning	14
4.2	Unsupervised Learning	15
4.3	Summary of Learning Types	15
5	Data Sampling and Dataset Splitting	15
5.1	The Data Distribution Problem	15
5.2	Training, Validation, and Test Sets	15
5.3	How to Use the Three Sets	16
5.4	The Importance of Distribution Similarity	16
6	Training and Testing: The Role of Features	17
6.1	Training (Learning, Induction)	17
6.2	Testing	17
6.3	Learning and Generalization	17
7	The Complete Machine Learning Recipe	18
7.1	The Five Ingredients of Machine Learning	18
7.2	The Interplay of Ingredients	18
8	Model and Hypothesis Space	18
8.1	What is a Model?	18
8.2	Types of Learning Paradigms	19
9	Model-Based vs. Instance-Based Learning	20
9.1	Model-Based (Parametric) Learning	20
9.2	Instance-Based (Nonparametric) Learning	20
9.3	Comparison	20
10	Error Function (Objective Function)	20
10.1	What is an Error Function?	21
10.2	Types of Error Functions	21

11 Example: Polynomial Fitting	21
11.1 Problem Setup	21
11.2 Model: Polynomial Functions	22
11.3 Error Function: Mean Squared Error	22
12 Underfitting and Overfitting	23
12.1 Definitions	23
12.2 The Bias-Variance Trade-off	24
12.3 Diagnostic Matrix	24
12.4 How to Handle Overfitting	24
12.5 How to Handle Underfitting	25
12.6 Polynomial Fitting Example Revisited	25
13 Regularization	26
13.1 What is Regularization?	26
13.2 The Regularized Objective Function	26
13.3 Effect of Regularization	27
13.4 Regularization in Polynomial Fitting	27
13.5 Generalization and Data Size	28
II Model Evaluation and Regression Methods	29
14 Model Evaluation for Classification and Regression	29
14.1 Recap: Train, Validation, and Test Sets	29
14.2 The Importance of the Validation Set	29
14.3 Specific Rules for Dataset Splitting	30
15 Cross Validation	32
15.1 When Do We Need Cross Validation?	32
15.2 Key Characteristics of Cross Validation	32
15.3 Fundamental Rules of Cross Validation	32
15.4 K-Fold Cross Validation	32
15.5 Example: 5-Fold Cross Validation	33
15.6 Advantages of Cross Validation	34
15.7 Disadvantages of Cross Validation	34
16 Leave-One-Out Cross Validation	34
16.1 What is Leave-One-Out Cross Validation?	34
16.2 How LOOCV Works	35
16.3 Pros and Cons of LOOCV	35
17 Stratification in Cross Validation	36
17.1 The Problem: Imbalanced Classes	36
17.2 What is Stratification?	36
17.3 How Stratified K-Fold Works	36
17.4 Example: Stratified 5-Fold Cross Validation	36
17.5 Why Stratification Matters	37
18 Evaluation Measures	38
18.1 Why Do We Need Evaluation Measures?	38
18.2 Evaluation Measures by Task Type	38

19 Evaluation Measures for Classification	39
19.1 Types of Classification Problems	39
19.2 The Confusion Matrix	39
19.3 Classification Error and Accuracy	40
19.4 The Problem with Accuracy: Imbalanced Classes	40
19.5 Misses and False Alarms	41
19.6 Cost of the Task	42
19.7 F-Measure (F1 Score)	43
19.8 Multiclass Classification – Evaluation	43
19.9 ROC Curve – Receiver Operating Characteristic	44
20 Regression Evaluation Metrics	45
20.1 The Fundamental Difference	45
20.2 Mean Squared Error (MSE)	46
20.3 Mean Absolute Error (MAE)	46
20.4 Choosing Between MSE and MAE	47
20.5 Other Regression Metrics	47
III Bayes Classifier and Nonparametric Classifiers	48
21 Bayes Classifier	48
21.1 Introduction	48
21.2 An Easy Example	48
21.3 Another Example: Adding Measurements	49
21.4 Bayes Formula	49
21.5 Bayes Decision Rule: Detailed Analysis	51
22 Naïve Bayes Classifier	51
22.1 Core Principles	51
22.2 The Naïve Independence Assumption	52
22.3 Example: Email Spam Classification	52
22.4 The Zero Probability Problem	53
22.5 Advantages and Disadvantages	54
22.6 Classification Rule Summary	54
22.7 Problems with Bayesian Classifiers	55
22.8 Challenges in Estimating Conditional Densities	55
22.9 Solution: Parametric Models	56
22.10 Summary: From Theory to Practice	57
23 MLE in Naïve Bayes: Practical Implementation	57
23.1 MLE for Discrete (Categorical) Features	57
23.2 MLE for Continuous Features: Gaussian Model	58
23.3 Complete Example: Fruit Classification	58
23.4 Key Takeaways from the Example	60
24 Nonparametric Classifiers	61
24.1 Introduction to Nonparametric Methods	61
24.2 Histogram-Based Density Estimation	61
24.3 Drawbacks of Histograms	62
24.4 The Curse of Dimensionality	62
24.5 Probability Density: Mathematical Foundation	63
24.6 Binomial Distribution Approach	64

24.7 Density Estimation Formula	65
24.8 The Trade-off Problem	65
24.9 General Nonparametric Density Estimation Formula	66
24.10 Two Approaches to Nonparametric Density Estimation	66
25 k-Nearest Neighbors (k-NN)	67
25.1 Visual Comparison: k-NN vs. Parzen Windows	67
25.2 Parzen Window: Practical Considerations	67
25.3 k-Nearest Neighbors Algorithm	68
25.4 k-NN for Classification	68
25.5 Choosing k	69
25.6 Advantages and Disadvantages of k-NN	70
25.7 Practical Considerations	70
25.8 Distance Metrics in k-NN	70
25.9 Weighted k-NN	71
25.10 Importance of Feature Scaling (Revisited)	71
25.11 Mahalanobis Distance	72
25.12 How to Select k in k-NN	74
25.13 k-NN: Pros and Cons	76
25.14 How k-NN Was Born: PDF Estimation	78
IV Decision Trees and Ensemble Methods	80
26 Introduction to Decision Trees	80
26.1 Decision Tree Structure	80
26.2 How Decision Trees Work	80
26.3 Decision Tree Inference: Non-terminal Nodes	80
26.4 Decision Tree Inference: Leaf Nodes	81
26.5 Formal Decision Tree Function	81
26.6 Decision Tree Inference Example	81
27 Learning Decision Trees	82
27.1 The Learning Problem	82
27.2 Tree Learning Framework	82
27.3 Tree Growing Strategy	82
28 ID3 and CART Algorithms	82
28.1 Example: Playing Tennis Dataset	82
28.2 Choosing the Best Attribute	83
28.3 Measuring Purity	83
29 Entropy and Information Gain	84
29.1 Entropy	84
29.2 Information Gain	85
29.3 Using Information Gain for Attribute Selection	85
29.4 Problems with Information Gain	85
29.5 Gain Ratio: A Solution	86
29.6 Advantages of Information Gain	86
30 Alternative Splitting Criteria	86
30.1 Gini Impurity	86
30.2 Comparison of Splitting Criteria	87

31 Decision Trees and Overfitting	87
31.1 Why Decision Trees Overfit	87
31.2 Overfitting Example	87
31.3 Techniques to Prevent Overfitting	88
31.4 Pruning (Post-processing)	88
32 Decision Trees for Different Data Types	88
32.1 Continuous Attributes	88
32.2 Multi-class Classification	89
33 Decision Trees: Summary	89
33.1 Advantages (Pros)	89
33.2 Disadvantages (Cons)	89
33.3 Key Takeaways	90
34 Random Forests	90
34.1 What is a Random Forest?	90
34.2 Random Feature Selection	90
34.3 Random Forest Prediction	91
34.4 Why Random Forests Work	91
35 Ensemble Methods	91
35.1 What are Ensemble Methods?	91
35.2 Types of Ensemble Methods	92
36 Bagging (Bootstrap Aggregating)	92
36.1 The Bagging Algorithm	92
36.2 Benefits of Bagging	92
36.3 When to Use Bagging	92
37 Boosting	93
37.1 Key Differences from Bagging	93
37.2 Boosting Intuition	93
37.3 Weak Learners	93
37.4 AdaBoost (Adaptive Boosting)	94
37.5 AdaBoost: Key Steps Explained	94
37.6 AdaBoost Properties	95
38 Stacking	95
38.1 What is Stacking?	95
38.2 The Stacking Process	95
38.3 Why Stacking Works	96
38.4 Stacking Best Practices	96
38.5 Comparison of Ensemble Methods	96
38.6 Key Takeaways	96
V Linear Discriminant Functions and Classification Methods	97
39 Linear Discriminant Functions	97
39.1 Learning the Linear Discriminant Function	97
39.2 Linear Separability	98
39.3 Properties of Linear Discriminant Functions	98

40 Linear Classifiers	99
40.1 Decision Boundary	99
40.2 The Role of Weights	99
40.3 Which Hyperplanes?	100
40.4 Optimization Approaches	100
40.5 The Margin	101
41 Multi-Class Problems	101
42 LDF: Augmented Feature Vector	101
43 LDF: Training Error	102
43.1 Criterion for Determining Weights	102
44 LDF: Normalization	103
44.1 Before and After Normalization	103
45 LDF: Solution Region	103
46 Gradient Descent	104
46.1 Gradient Descent Update Rule	105
46.2 Local vs Global Minimum	105
46.3 Gradient Operator	106
46.4 Gradient Descent Summary	106
46.5 Learning Rate Selection	106
47 Perceptron	107
47.1 Perceptron Criterion Function	107
47.2 Convergence Limitation	107
47.3 Perceptron Gradient Descent	108
47.4 Perceptron Example (Non-Separable Scenario)	108
47.5 Applying the Perceptron Method	109
47.6 Need for Relaxation	110
47.7 Perceptron Summary	110
48 The Relaxation Method	110
48.1 Relaxation Criterion Function (1)	110
48.2 Relaxation Criterion Function (2)	110
49 Minimum Squared Error (MSE) Solution	111
49.1 Problem Transformation	111
49.2 Comparison: Perceptron Rule vs MSE/Relaxation Method	111
49.3 MSE Procedure	111
49.4 Normalization and Dataset Setup	111
49.5 Matrix Formulation	112
49.6 Error Vector and MSE Criterion	112
49.7 Analytical Solution	112
49.8 Gradient Derivation	113
49.9 Finding the Optimal Solution	113
49.10 Proof of the Gradient Result	113
50 MSE Properties	114
50.1 Sensitivity to Outliers	114

50.2 Computational Challenges	114
50.3 Lack of Flexibility	114
51 Example: Perceptron vs MSE	114
51.1 Perceptron Learning	115
51.2 Perceptron Iteration	115
51.3 MSE Solution	115
52 Perceptron vs. MSE (Summary)	116
52.1 Gradient Descent Alternative for MSE	116
53 The Least MSE (LMSE = Widrow-Hoff Method)	116
53.1 Correction Mechanism	116
53.2 Case 2: Prediction is Wrong	117
53.3 Widrow-Hoff Method: Summary	117
53.4 Widrow-Hoff Method: Disadvantages	117
54 The Ho-Kashyap Method	118
54.1 Ho-Kashyap Approach	118
54.2 Gradient Descent Optimization	118
54.3 Ho-Kashyap Rule	119
54.4 Ho-Kashyap Algorithm	119
54.5 Convergence Properties	119
55 Summary: Linear Discriminant Functions	120
VI Linear Transformations and Dimensionality Reduction	121
56 Problems of High-Dimensional Data	121
56.1 Computational Complexity	121
56.2 Curse of Dimensionality	121
56.3 Overfitting in High Dimensions	122
57 Dimensionality Reduction	122
58 Principal Component Analysis (PCA)	123
58.1 When to Apply PCA?	123
58.2 PCA: Direction of Largest Variance	123
58.3 PCA Algorithm	123
58.4 PCA: Mathematical Formulation	123
58.5 PCA: The Transformation	124
58.6 PCA: Dimensionality Reduction	124
58.7 How Many Principal Components?	125
58.8 PCA Drawbacks	125
59 Fisher Linear Discriminant (LDA)	125
59.1 Setup	125
59.2 Separation Measure	126
59.3 Fisher's Criterion Function	126
59.4 Scatter Matrices	126
59.5 Expressing $J(\mathbf{w})$ in Terms of Scatter Matrices	127
59.6 The Fisher Transform (Solution)	127

59.7 FDA: Worked Example	127
59.8 FDA Drawbacks	128
60 Summary: PCA vs. FDA	128
 VII Kernel-Based Learning and Support Vector Machines	 129
61 Binary Classification Recap	129
62 Linear SVM	129
62.1 Large Margin Classifiers	129
62.2 Support Vectors	129
62.3 Finding the Hyperplane	129
62.4 Maximizing the Margin	130
63 Soft Margin SVM	130
63.1 Motivation	130
63.2 Slack Variables	130
63.3 The Role of C	130
63.4 Understanding Slack Variables	131
63.5 Hinge Loss	131
63.6 Support Vectors in the Non-Linear (Soft Margin) Case	131
64 Non-Linear SVM	131
64.1 Motivation	131
64.2 Example: Mapping to Higher Dimensions	132
64.3 The Kernel Trick	132
64.4 Kernel Trick: Example	132
64.5 Standard Kernels	133
64.6 Kernel Selection	133
65 Typical SVM Implementation	133
66 SVM Summary	134
 VIII Clustering	 135
67 Supervised vs. Unsupervised Learning	135
67.1 Unsupervised Learning Approaches	135
68 Clustering	135
68.1 What Is Good Clustering?	135
68.2 How Many Clusters?	136
69 Proximity / Distance Measures	136
69.1 Dissimilarity Measures	136
69.2 Cosine Similarity	136
70 Clustering Approaches	137
71 K-Means Clustering	137
71.1 Problem Formulation	137

71.2	<i>K</i> -Means Algorithm	138
71.3	Initialization Strategies	138
71.4	Choosing the Number of Clusters	138
71.5	<i>K</i> -Means Drawbacks	139
72	Hierarchical Clustering	139
72.1	Dendrograms	139
72.2	Agglomerative vs. Divisive	139
72.3	Agglomerative Hierarchical Clustering	139
72.4	Linkage Criteria	139
72.5	Divisive Hierarchical Clustering	140
72.6	Hierarchical Clustering: Limitations	140
73	DBSCAN	140
73.1	Parameters and Point Types	140
73.2	DBSCAN Algorithm	141
73.3	DBSCAN: Advantages and Limitations	141
74	Clustering Evaluation	141
74.1	Extrinsic Measures	141
74.2	Intrinsic Measures	141
IX	Parametric Unsupervised Learning	144
75	Parametric vs Non-Parametric Unsupervised Learning	144
75.1	Advantages of Having a Parametric Model	144
75.2	Comparison with Supervised Learning	144
76	Gaussian Mixture Models (GMM)	145
76.1	Latent Variables	145
76.2	GMM Parameters	145
76.3	Joint and Marginal Distributions	145
76.4	Responsibilities	146
76.5	Generating Synthetic Data from a GMM	147
77	Maximum Likelihood for GMM	147
77.1	Data Representation	147
77.2	Likelihood and Log-Likelihood	147
78	Technical Issues	147
78.1	Singularity Problem	148
78.2	Identifiability Problem	148
79	Expectation-Maximisation (EM) Algorithm for GMM	148
79.1	Deriving the Parameter Updates	148
79.2	The EM Algorithm — Step by Step	150
79.3	Visualising EM Iterations	150
80	Properties of EM for GMM	151
81	GMM Summary	151

X	Hidden Markov Models	152
82	Sequential Data	152
83	Markov Models (Markov Chains)	152
83.1	Independence Model (Baseline)	152
83.2	First-Order Markov Model	152
83.3	Markov Model Ingredients	152
83.4	Higher-Order Markov Models	153
84	Hidden Markov Models (HMM)	153
84.1	Formal Definition	153
84.2	Conditional Independence in HMMs	154
84.3	Types of HMM Structure	154
85	The Three Key Tasks of HMMs	154
86	Task 1: Computing the Observation Likelihood	155
86.1	Problem Statement	155
86.2	Naïve Approach	155
86.3	The Forward Algorithm	155
87	Task 2: Decoding (Viterbi Algorithm)	156
87.1	Problem Statement	156
87.2	The Viterbi Algorithm	156
88	Task 3: Training (Baum-Welch Algorithm)	157
88.1	Problem Statement	157
88.2	Fully Visible Case (Baseline)	157
88.3	Backward Probability	157
88.4	Key Quantities for Re-Estimation	158
88.5	Parameter Re-Estimation Formulas	158
88.6	The Baum-Welch Algorithm (EM for HMMs)	159
89	HMM Summary	159
XI	Machine Learning vs. Deep Learning — Recap	161
90	ML vs DL: Overview	161
91	ML vs DL: Detailed Comparison	161
91.1	Data Dependency	161
91.2	Feature Engineering	161
91.3	Computational Resources	161
91.4	Model Interpretability	161
91.5	Training Time	161
91.6	Problem Complexity	162
91.7	Flexibility	162
91.8	Scalability	162
92	Course Recap	162
92.1	Foundations	162
92.2	Data Cleaning and Preprocessing	163

92.3 Model Evaluation	163
92.4 Supervised Learning — Linear Models	163
92.5 Supervised Learning — Instance-Based	163
92.6 Supervised Learning — Decision Trees and Ensembles	163
92.7 Support Vector Machines	163
92.8 Unsupervised Learning — Clustering	163
92.9 Unsupervised Learning — Dimensionality Reduction	163
92.10 Probabilistic Models	164

Part I

Introduction to Machine Learning

1 What is Machine Learning?

Machine Learning is a branch of Artificial Intelligence that enables software to use data to find solutions to specific tasks without being explicitly programmed to do so.

1.1 Machine Learning vs Statistical Modelling

In traditional statistical modelling, we follow a structured process:

1. Collect data
2. Verify and clean the data (correct or discard if not clean)
3. Use the clean data to test hypotheses, make predictions and forecasts

In contrast, Machine Learning takes a different approach:

- It is the **data** that determines which analytic techniques to use
- The computer uses data to **train** algorithms to find patterns and make predictions
- Algorithms are no longer **static** but become **dynamic**

1.2 Conventional Programming vs Machine Learning

The fundamental difference between conventional programming and machine learning can be understood through their inputs and outputs:

1.2.1 Conventional Programming

- **Input:** Program + Data
- **Output:** Result
- You write the program, give it data, and get results

1.2.2 Machine Learning

- **Input:** Data + Result
- **Output:** Program
- You give the computer data and desired results, and it learns to generate a program/algorithm

1.3 The Machine Learning Recipe

The 4 Fundamental Steps of ML

Collect data → Define models → Define objective function → Minimize error

The ML recipe consists of 4 fundamental steps:

1. Collect data
2. Define a family of possible models

3. Define an objective (error) function to quantify how well a model fits the data
4. Find the model that minimizes the error function (training/learning a model)

1.3.1 The Key Ingredients

5 Essential Ingredients

Task • Data • Model • Objective Function • Learning Algorithm

Every machine learning problem requires five essential ingredients:

- **Task:** The problem we want to solve
- **Data:** The information we use to learn
- **Model hypothesis:** The family of functions we consider
- **Objective function:** How we measure success
- **Learning algorithm:** How we find the best model

2 Machine Learning Tasks

A task represents the type of prediction being made to solve a problem on some data. We can identify a task with the set of functions that can potentially solve the problem. In general, it consists of functions assigning each **input** $x \in \mathcal{X}$ an **output** $y \in \mathcal{Y}$.

2.1 Classification Task

Definition 2.1 (Classification). Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **discrete** label, $f(x) \in \mathcal{Y} = \{c_1, \dots, c_k\}$.

In classification, the output space is a finite set of discrete categories or classes. The goal is to learn a decision boundary that separates different classes.

2.2 Regression Task

Definition 2.2 (Regression). Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **continuous** label, $f(x) \in \mathcal{Y} = \mathbb{R}$.

In regression, the output is a continuous value, allowing for predictions of quantities such as prices, temperatures, or probabilities.

2.3 Density Estimation Task

Definition 2.3 (Density Estimation). Find a probability distribution $f \in \Delta(\mathcal{X})$ that best fits the data $x \in \mathcal{X}$.

There is no reference to an output space (as in classification and regression tasks). Instead, we make a reasoning on the **input** x itself, trying to model the underlying probability distribution that generated the data.

2.4 Clustering Task

Definition 2.4 (Clustering). Find a function $f \in \mathbb{N}^{\mathcal{X}}$ that assigns each input $x \in \mathcal{X}$ a **cluster index** $f(x) \in \mathbb{N}$.

All points mapped to the same index form a cluster. The goal is to group similar data points together without prior knowledge of the groups.

2.5 Dimensionality Reduction Task

Definition 2.5 (Dimensionality Reduction). Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ that **maps** each (**high-dimensional**) input $x \in \mathcal{X}$ to a **lower-dimensional** embedding $f(x) \in \mathcal{Y}$, where $\dim(\mathcal{Y}) < \dim(\mathcal{X})$.

This task aims to reduce the number of features while preserving the essential information in the data.

3 Data in Machine Learning

3.1 Data Representation

We represent inputs of our algorithm as $x \in \mathcal{X}$ and outputs as $y \in \mathcal{Y}$. The dataset is the set of both, where each input is associated with an output.

Key considerations:

- The quality of the data is crucial for the performance of the algorithm
- It is equally important to have a large amount of data to learn effective models
- Poor quality or insufficient data can lead to poor model performance

3.2 Features

Definition 3.1 (Features). Features are the measurable properties or characteristics of the phenomenon being observed.

Examples:

- To identify a flower: color, petal length, petal width, sepal length, sepal width
- To identify a fruit: color, shape, size, texture, weight
- To classify images: pixel values, edges, textures, shapes

We can say that features are how our algorithm "sees" the data and are in general represented with (fixed-size) vectors.

4 Data Distributions and Learning Types

In machine learning, the information about the problem we want to solve is represented in the form of a data distribution, usually denoted as p_{data} .

4.1 Supervised Learning

4.1.1 Classification and Regression

The data distribution is over pairs of inputs and outputs:

$$p_{\text{data}} \in \Delta(\mathcal{X} \times \mathcal{Y})$$

Here:

- $\mathcal{X}$ is the input space
- $\mathcal{Y}$ is the output space (labels or targets)
- The goal is to learn a mapping from inputs to outputs using labeled data

4.2 Unsupervised Learning

4.2.1 Density Estimation, Clustering, and Dimensionality Reduction

The data distribution is only over the input space:

$$p_{\text{data}} \in \Delta(\mathcal{X})$$

Here:

- We do not have explicit labels or targets
- The goal is to discover structure, patterns, or representations within the data itself

4.3 Summary of Learning Types

Task Type	Data Distribution	Learning Type
Classification, Regression	$p_{\text{data}} \in \Delta(\mathcal{X} \times \mathcal{Y})$	Supervised Learning
Density Estimation, Clustering, Dimensionality Reduction	$p_{\text{data}} \in \Delta(\mathcal{X})$	Unsupervised Learning

Table 1: Comparison of learning types based on data distribution

Note. In **supervised learning**, each data point consists of an input and a corresponding output (label). In **unsupervised learning**, each data point consists only of the input, and the algorithm tries to find patterns or structure without explicit labels.

This distinction is fundamental in machine learning, as it determines the type of algorithms and approaches used to solve different problems.

5 Data Sampling and Dataset Splitting

5.1 The Data Distribution Problem

In both supervised and unsupervised learning, the true data distribution p_{data} is typically **unknown**. This presents a fundamental challenge:

- We do not have direct access to the underlying probability distribution that generates the data
- We only have access to a finite sample of data points drawn from this distribution

Solution: We perform **sampling** from the distribution. By collecting a representative sample of data, we can approximate the true distribution and use it to train our models.

5.2 Training, Validation, and Test Sets

To properly evaluate machine learning models, we split our data into three distinct sets. Each set serves a specific purpose in the machine learning pipeline.

Definition 5.1 (Dataset Splitting). A dataset is typically divided into three subsets:

- **Training set** (D_{train}): Used to train the model
- **Validation set** (D_{val}): Used to tune hyperparameters and select the best model
- **Test set** (D_{test}): Used to evaluate the final model performance

Critical Requirement

All three sets (training, validation, test) must be sampled from the **same probability distribution**

5.3 How to Use the Three Sets

The three datasets serve different purposes in the machine learning workflow:

5.3.1 Learning/Training Phase

During the learning phase, the **training set** is used to train the machine learning algorithm, which learns patterns and relationships in the data to produce a **program** (model).

The **validation set** plays a crucial role in this phase:

- Evaluates different model configurations
- Tunes hyperparameters
- Selects the best performing model

This process involves iterative feedback between training and validation until we find the optimal configuration.

5.3.2 Testing/Model Evaluation Phase

Once the best model is selected, we use the **test set** to evaluate its performance. The trained model is applied to the test data, which it has never seen before. This gives us an unbiased estimate of how well the model will perform in real-world scenarios, telling us whether our model can truly generalize to new data.

5.4 The Importance of Distribution Similarity

5.4.1 Correct Scenario: Same Distribution

When training, validation, and test sets are sampled from the **same distribution**:

- The sets are **similar** in terms of statistical properties
- They represent the same underlying phenomenon
- However, they **do not overlap** (no data point appears in multiple sets)

This ensures that the model can generalize well from training to test data.

Example: If we're classifying fruits, all three sets might contain apples and bananas in similar proportions, but with different specific instances.

5.4.2 Incorrect Scenario: Different Distributions

When training/validation and test sets come from **different distributions**, the model learns patterns from one distribution but is tested on another. This problematic scenario typically leads to:

- Poor performance on the test set
- Unreliable predictions on new data
- Inability to generalize to real-world applications

Example: Training on apples and bananas, but testing on oranges and limes. The model cannot classify fruits it has never seen during training.

Key Principle

Training and test data must come from the **same distribution**
Violating this leads to poor generalization

6 Training and Testing: The Role of Features

6.1 Training (Learning, Induction)

During the training phase, the model learns to distinguish between different classes based on the features provided in the training data.

Example: Consider a fruit classification task with the following training data:

- *red, round, leaf, 3oz, ...* → apple
- *green, round, no leaf, 4oz, ...* → apple
- *yellow, curved, no leaf, 8oz, ...* → banana
- *green, curved, no leaf, 7oz, ...* → banana

The model learns to associate patterns in the features (color, shape, presence of leaf, weight) with the corresponding labels (apple or banana). **What distinguishes apples and bananas is based on the features!** The model identifies which feature combinations are characteristic of each class.

6.2 Testing

Once trained, the model can classify new examples based on the same features it learned during training. When presented with a new example described by the same semantic features, the model makes a prediction.

Example: Given a new fruit with features *red, round, no leaf, 4oz, ...*, the model uses the learned patterns to predict whether it is an apple or a banana.

Critical Assumption

The features of training and testing data **must be the same!**
The model can only make predictions based on features it was trained on

6.3 Learning and Generalization

Learning is fundamentally about **generalizing** from the training data. The goal is not simply to memorize the training examples, but to learn patterns that apply to new, unseen data.

However, the failure of a machine learning algorithm is often caused by a **bad selection of training samples**. The key issue is that we might introduce **unwanted correlations** from which the algorithm derives **wrong conclusions**.

Example: Consider a model trained to recognize objects in images. If all training images were captured on sunny days, the model might learn to associate sunny weather conditions with the objects. When tested on images taken on cloudy days, the model might fail to recognize the same objects because it never encountered cloudy conditions during training. The model incorrectly learned that sunny weather is a relevant feature for object recognition.

Quality and diversity of training data are crucial
Training data should be representative of all real-world conditions

7 The Complete Machine Learning Recipe

We can now revisit the complete machine learning workflow with all its essential ingredients:

7.1 The Five Ingredients of Machine Learning

Every machine learning problem requires five fundamental components that work together:

1. **Task:** The specific problem we want to solve (e.g., classify birds vs. non-birds in images)
2. **Data:** The collection of examples used to train and evaluate the model. Data quality and quantity are critical for success.
3. **Model Hypothesis:** The family of functions or models we consider as potential solutions. This defines the space of possible models the learning algorithm can explore.
4. **Objective Function:** A mathematical function that quantifies how well a model performs on the data. The learning algorithm aims to optimize this function.
5. **Learning Algorithm:** The method used to search through the model hypothesis space and find the model that optimizes the objective function.

7.2 The Interplay of Ingredients

These five ingredients are interconnected:

- The **task** determines what kind of **data** we need and what **model hypothesis** is appropriate
- The **data** influences which features are available and how we define the **objective function**
- The **model hypothesis** constrains what the **learning algorithm** can discover
- The **objective function** guides the **learning algorithm** toward better models

Understanding and carefully selecting each ingredient is essential for building effective machine learning systems.

8 Model and Hypothesis Space

8.1 What is a Model?

Definition 8.1 (Model). A **model** is the implementation of a function $f \in \mathcal{F}_{\text{task}}$ that can be tractably computed.

In practice, when searching for our function f , we don't look at everything in $\mathcal{F}_{\text{task}}$. Instead, we look at a subset called the **hypothesis space**: $\mathcal{H} \subset \mathcal{F}_{\text{task}}$, which is composed of a set of models (e.g., neural networks, decision trees, linear models).

The learning algorithm seeks a **solution within the hypothesis space**. In other words, a model is a simplified representation of the world that we can actually compute and optimize.

Observation. The choice of hypothesis space is crucial:

- If $\mathcal{H}$ is too small, it may not contain a good solution
- If $\mathcal{H}$ is too large, finding the best model becomes computationally intractable
- The hypothesis space defines the types of patterns the model can learn

8.2 Types of Learning Paradigms

Three Learning Paradigms

Machine learning can be categorized into different paradigms based on the type and availability of labeled data:

Supervised Learning

In supervised learning, we have labeled data where each input is paired with its corresponding output. This includes:

- **Classification:** Predicting discrete labels
- **Regression:** Predicting continuous values

Unsupervised Learning

In unsupervised learning, we only have input data without labels. The goal is to discover structure in the data. This includes:

- **Clustering:** Grouping similar data points
- **Density Estimation:** Modeling the probability distribution of data
- **Dimensionality Reduction:** Finding lower-dimensional representations

8.2.1 Supervised Learning

In supervised learning, we have labeled data where each input is paired with its corresponding output.

8.2.2 Unsupervised Learning

In unsupervised learning, we only have input data without labels.

8.2.3 Semi-Supervised Learning

Semi-supervised learning addresses scenarios where we have a large amount of unlabeled data and only some labeled examples. This paradigm has gained significant popularity in recent years to leverage large datasets through self-supervised learning techniques.

The key idea is to provide the model with tasks different from the one to be solved, but that allow the model to learn the data distribution from the unlabeled data. Once the model understands the general structure of the data, it can then specialize on the few labeled examples to solve the target task.

Example: Training a language model on large amounts of unlabeled text to learn general language patterns, then fine-tuning it on a small labeled dataset for sentiment analysis.

9 Model-Based vs. Instance-Based Learning

Machine learning algorithms can be further categorized based on how they make predictions:

9.1 Model-Based (Parametric) Learning

Definition 9.1 (Parametric Model). A learning model that summarizes data with a **set of parameters of fixed size** (independent of the number of training examples) is called a **parametric model**.

Parametric Model Characteristics

Key characteristics:

- The model learns a fixed set of parameters from the training data
- Once trained, the original training data can be discarded
- No matter how much data you give to a parametric model, it won't change its mind about how many parameters it needs
- The model makes predictions using only the learned parameters

Examples: Naive Bayes, Linear Regression, Logistic Regression, Neural Networks

In a model-based approach, the algorithm learns a function (e.g., a decision boundary) described by parameters. For instance, a linear classifier might learn the curve $\alpha x_1 + \beta x_2 + \gamma$ that separates two classes, where α, β, γ are the parameters.

9.2 Instance-Based (Nonparametric) Learning

Definition 9.2 (Nonparametric Model). These algorithms **do not learn parameters** but instead use the data itself to compare a new example through similarity metrics.

Nonparametric Model Characteristics

Key characteristics:

- The model stores (some or all of) the training data
- Predictions are made by comparing new examples to stored training examples
- The model's complexity can grow with the amount of training data
- No explicit training phase that produces a fixed set of parameters

Examples: K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest

In an instance-based approach, when a new example arrives, it is compared with nearby training examples and classified according to a similarity policy. For example, in KNN, a new point is classified based on the majority class of its k nearest neighbors in the training data.

9.3 Comparison

10 Error Function (Objective Function)

To allow an algorithm to learn, we need to provide it with a metric, a measure that helps it understand the direction of the optimal solution we are seeking. For this reason, **performance measures** are defined to enable the algorithm to learn.

Aspect	Model-Based	Instance-Based
Parameters	Fixed number of parameters	No fixed parameters
Training data	Can be discarded after training	Must be retained for predictions
Prediction speed	Fast (only uses parameters)	Slower (compares with stored data)
Memory	Low (stores only parameters)	High (stores training examples)
Flexibility	Fixed model complexity	Complexity grows with data

Table 2: Comparison between model-based and instance-based learning

10.1 What is an Error Function?

In machine learning, **training a model** means finding a **function** which maps a set of values x to a value y . We can calculate how well a predictive model is doing by comparing the **predicted values** with the **true values** for y .

Definition 10.1 (Training Error vs. Test Error). • If we apply the model to the data it was trained on, we are calculating the **training error**

- If we calculate the error on data which was **unknown** in the training phase, we are calculating the **test error**

The test error is the most important metric because it tells us how well the model generalizes to new, unseen data.

10.2 Types of Error Functions

There are different types of error functions (also called loss functions or objective functions), each suited for different tasks:

- **Mean Absolute Error (MAE)**: Measures the average absolute difference between predictions and true values
- **Mean Squared Error (MSE)**: Measures the average squared difference between predictions and true values
- **Binary Cross-Entropy**: Used for binary classification problems
- **Mean Average Precision**: Used for ranking and information retrieval tasks

The choice of error function depends on the task and the desired properties of the model.

11 Example: Polynomial Fitting

Let's consider a concrete example to understand how models, hypothesis spaces, and error functions work together.

11.1 Problem Setup

Consider a regression problem with the following setup:

- **Data**: $\mathcal{D}_n = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- Data generated from $\sin(2\pi x) + \text{noise}$
- Training set with $n = 10$ points
- **Goal**: Learn a function (shown as the purple line) that fits the data

The correct solution should capture the underlying sinusoidal pattern while being robust to the noise in the training data.

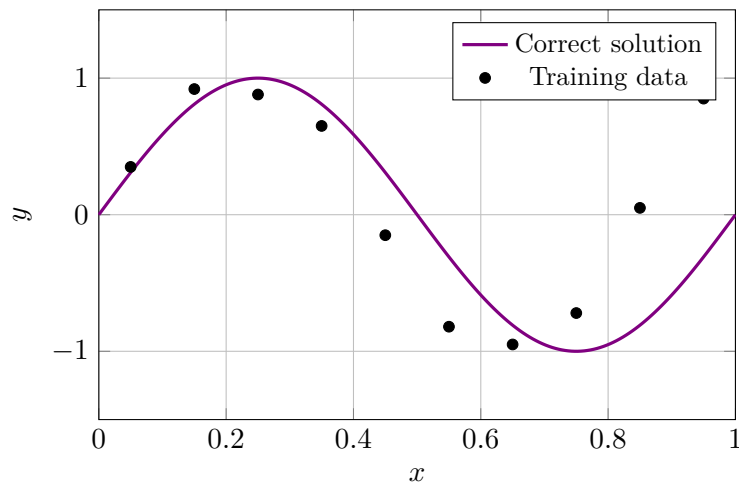


Figure 1: Polynomial Fitting Problem: Data generated from $\sin(2\pi x)$ with noise

11.2 Model: Polynomial Functions

Polynomial Model

We choose to model the data using polynomial functions:

$$f_w(x) = \sum_{j=0}^M w_j x^j$$

where:

- M is the **degree of the polynomial** (a hyperparameter)
- $w = \{w_0, \dots, w_M\}$ are the **parameters** to be learned
- The **hypothesis space** for a fixed degree M is: $\mathcal{H}_M = \{f_w : w \in \mathbb{R}^{M+1}\}$

Different values of M give us different hypothesis spaces:

- $M = 0$: Constant functions (horizontal lines)
- $M = 1$: Linear functions (straight lines)
- $M = 2$: Quadratic functions (parabolas)
- $M = 3$: Cubic functions
- Higher M : More complex curves

11.3 Error Function: Mean Squared Error

To measure how well our polynomial fits the data, we use the Mean Squared Error (MSE):

$$E(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n [f(x_i) - y_i]^2$$

This error function:

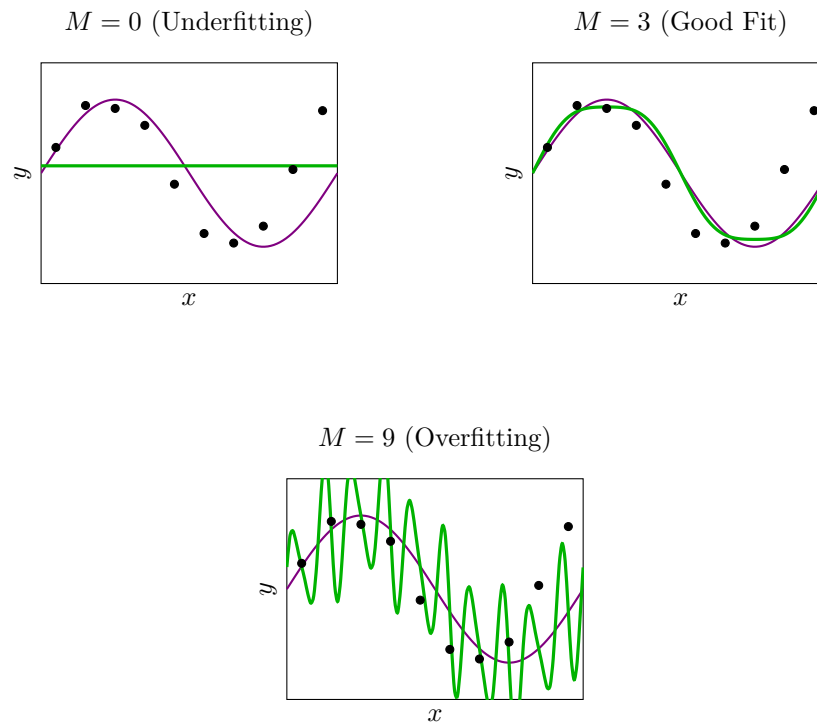


Figure 2: Polynomial fits with different degrees: $M = 0$ (underfitting), $M = 3$ (good fit), $M = 9$ (overfitting). Purple: true function, Green: fitted polynomial, Black dots: training data.

- Computes the squared difference between the prediction $f(x_i)$ and the ground-truth label y_i for each point
- Averages these squared differences over all n training points
- Is also called a **pointwise loss** because it measures the error at each individual data point

The learning algorithm's goal is to find the parameters w that minimize this error function:

$$w^* = \arg \min_{w \in \mathbb{R}^{M+1}} E(f_w; \mathcal{D}_n)$$

Choice of Model Complexity is Crucial:

- Too small M : Underfitting (cannot capture pattern)
- Appropriate M : Good fit (captures underlying pattern)
- Too large M : Overfitting (fits noise, poor generalization)

12 Underfitting and Overfitting

Two Fundamental Problems in ML

Underfitting (too simple) $\leftrightarrow$ Overfitting (too complex)

Two fundamental problems can occur when training machine learning models: underfitting and overfitting. Understanding these concepts is crucial for building models that generalize well.

12.1 Definitions

Definition 12.1 (Underfitting). **Underfitting** is a scenario where a model is unable to capture the relationship between the input (x) and output (y) variables accurately, generating a high

error rate on both the training set and unseen data (e.g., testing set).

The model is too simple and has not yet learned from the data. It performs poorly on both training and test data.

Definition 12.2 (Overfitting). **Overfitting** occurs when the model gives accurate predictions for training data (lower training error) but not for testing data (higher testing error).

The model is too sensitive to the training data and has essentially memorized it, including its noise and peculiarities, rather than learning the underlying pattern.

12.2 The Bias-Variance Trade-off

The relationship between model complexity, training error, and test error can be visualized as follows:

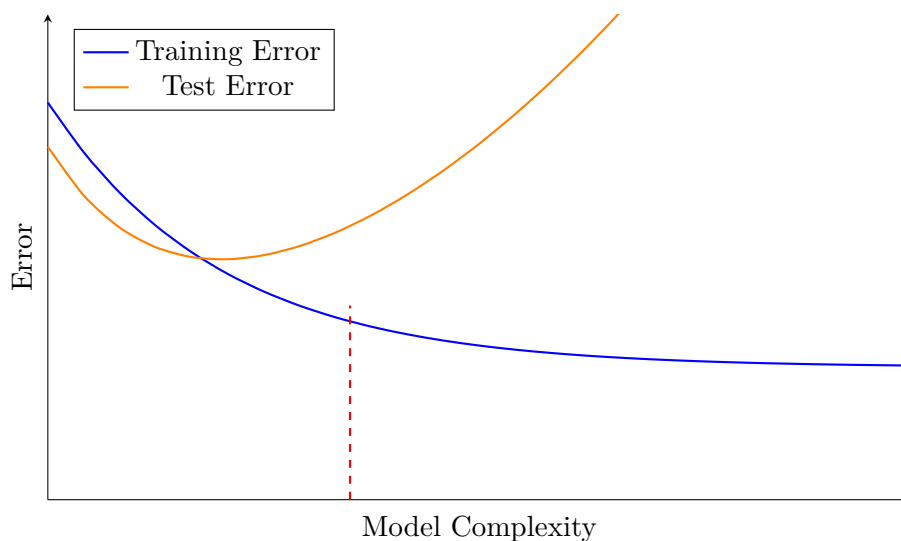


Figure 3: Bias-Variance Trade-off: Training and Test Error vs. Model Complexity

- **Left side of the graph (low complexity):** High error rate in both training and testing → **Underfitting**
- **Middle region (optimal complexity):** Low error rate in both training and testing → **Best Fit**
- **Right side of the graph (high complexity):** Low error rate in training but high error rate in testing → **Overfitting**

12.3 Diagnostic Matrix

We can diagnose the state of our model by examining both training and test errors:

	Low Training Error	High Training Error
Low Test Error	OK (Good fit)	Underfitting
High Test Error	Overfitting	Underfitting

Table 3: Diagnostic matrix for model performance

12.4 How to Handle Overfitting

When the model is too sensitive to training data and overfits, we can apply several strategies:

1. **Try a simpler model:** Use a model with fewer parameters or lower complexity
2. **Try a less powerful model:** Choose a model architecture with reduced capacity
3. **Increase regularization impact:** Apply techniques that penalize model complexity:
 - Early stopping: Stop training before the model overfits
 - L1/L2 regularization: Add penalty terms to the loss function
4. **Use a smaller number of features:**
 - Remove some features that may be causing overfitting
 - Apply feature selection techniques to identify the most relevant features
5. **Get more data:** Increasing the training set size helps the model learn more generalizable patterns

12.5 How to Handle Underfitting

When the model has not yet learned from the data and underfits, we can try:

1. **Try more complex models:** Use models with a larger number of parameters that can capture more intricate patterns
 - Ensemble learning: Combine multiple models
2. **Less regularization:** Reduce or remove regularization constraints that may be limiting the model's capacity
3. **A larger quantity of features:** Add more features or engineer new features that better capture the underlying relationships (get more features)

12.6 Polynomial Fitting Example Revisited

Let's examine how different polynomial degrees affect the fit:

- $M = 0$ (constant function): Severe underfitting. The horizontal line cannot capture any of the sinusoidal pattern. Both training and validation errors are high.
- $M = 1$ (linear function): Still underfitting. A straight line cannot represent the curved pattern. Both errors remain high.
- $M = 3$ (cubic polynomial): Good fit. The model captures the underlying sinusoidal pattern well. Both training and validation errors are low, and the curves are similar.
- $M = 9$ (9th-degree polynomial): Overfitting. The model passes through all training points (very low training error) but oscillates wildly between them. The validation error is high because the model has learned the noise rather than the signal.

The error plot shows:

- **Training error** (blue line): Decreases monotonically as M increases
- **Validation error** (orange line): Decreases initially, reaches a minimum around $M = 3$, then increases again
- The optimal model complexity is where the validation error is minimized

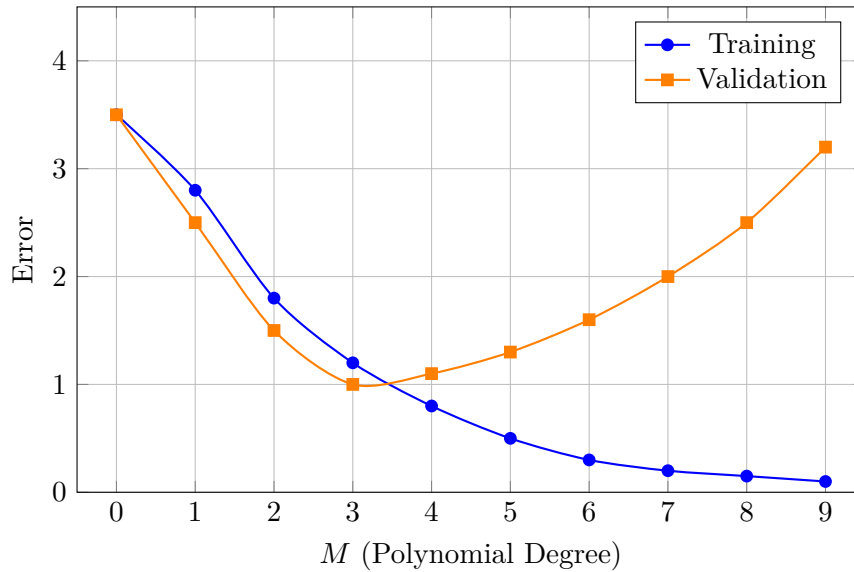


Figure 4: Training and Validation Error vs. Polynomial Degree M

Key Insight

Minimizing training error alone is **not sufficient**
 Must monitor validation/test error for generalization

13 Regularization

One of the Most Important Techniques in ML

Prevents overfitting and improves generalization

Regularization is one of the most important techniques in machine learning to prevent overfitting and improve model generalization.

13.1 What is Regularization?

Definition 13.1 (Regularization). Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting.

In practice, regularization is a modification of the training error function by appending a term $\Omega(f)$ that typically penalizes complex solutions.

13.2 The Regularized Objective Function

The regularized objective function has the following form:

$$E_{\text{reg}}(f; \mathcal{D}_n) = E(f; \mathcal{D}_n) + \lambda_n \Omega(f)$$

where:

- $E(f; \mathcal{D}_n)$ is the **training error function** (e.g., MSE)
- $\Omega(f)$ is the **regularization term** that penalizes model complexity
- λ_n is the **tradeoff hyperparameter** that controls the strength of regularization

The regularization term acts as a penalty that discourages the model from becoming too complex. By minimizing this combined objective, we find models that fit the data well while remaining relatively simple.

13.3 Effect of Regularization

The regularization term $\Omega(f)$ typically measures some notion of model complexity. When we minimize the regularized objective:

- We balance fitting the training data (low $E(f; \mathcal{D}_n)$) with keeping the model simple (low $\Omega(f)$)
- The hyperparameter λ_n controls this tradeoff
- Higher λ_n means stronger regularization (simpler models)
- Lower λ_n means weaker regularization (more complex models allowed)

13.4 Regularization in Polynomial Fitting

For the polynomial fitting example, we can regularize by penalizing polynomials with large coefficients. The regularized error function becomes:

$$E_{\text{reg}}(f_w; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n [f_w(x_i) - y_i]^2 + \frac{\lambda}{n} \|w\|^2$$

where:

- The first term is the training error (MSE)
- $\frac{\lambda}{n} \|w\|^2 = \frac{\lambda}{n} \sum_{j=0}^M w_j^2$ is the regularization term (L2 regularization)
- λ is the tradeoff hyperparameter

13.4.1 Effect of the Regularization Parameter λ

The choice of λ significantly affects the model:

- $\lambda \approx 10^{-18}$ (**very small**): Almost no regularization. The model can have large coefficients and may overfit. The polynomial fits the training data very closely, including noise.
- $\lambda = 1$ (**moderate**): Balanced regularization. The model is penalized for having large coefficients, leading to a smoother curve that generalizes better. This often provides the best tradeoff between training and validation error.
- λ **very large**: Strong regularization. The penalty for large coefficients is so high that the model becomes too simple and may underfit. The curve becomes nearly flat.

13.4.2 Regularization Path

The plot of error vs. $\ln(\lambda)$ shows:

- **Left side (small λ)**: Low training error, high validation error $\rightarrow$ Overfitting
- **Middle region**: Both errors are low $\rightarrow$ Good fit
- **Right side (large λ)**: Both errors increase $\rightarrow$ Underfitting

The optimal λ is found where the validation error is minimized.

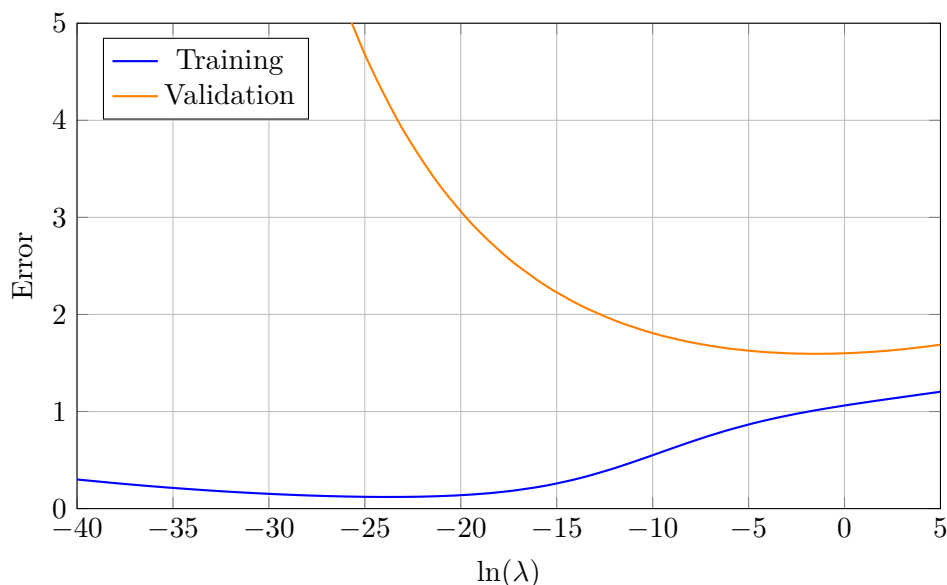


Figure 5: Training and Validation Error vs. $\ln(\lambda)$ (Regularization Strength)

13.5 Generalization and Data Size

An important theoretical result in machine learning relates training error to generalization error:

Theorem 13.1 (Generalization with Infinite Data). *As the number of training samples approaches infinity, the training error approximates the generalization error:*

$$E(f; \mathcal{D}_n) \rightarrow E(f; p_{data}) \quad \text{as } n \rightarrow \infty$$

This means:

- With a small dataset ($n = 15$), even a complex model ($M = 9$) may overfit because there isn't enough data to constrain the parameters
- With a large dataset ($n = 100$), the same complex model can generalize well because the abundant data prevents overfitting
- More data allows us to use more complex models without overfitting

Visual example:

- $n = 15$, $M = 9$: The 9th-degree polynomial overfits, oscillating wildly between the few training points
- $n = 100$, $M = 9$: With 100 training points, the same 9th-degree polynomial fits smoothly and generalizes well

"Get more data" is often the most effective solution to overfitting
 With sufficient data, even complex models can generalize well

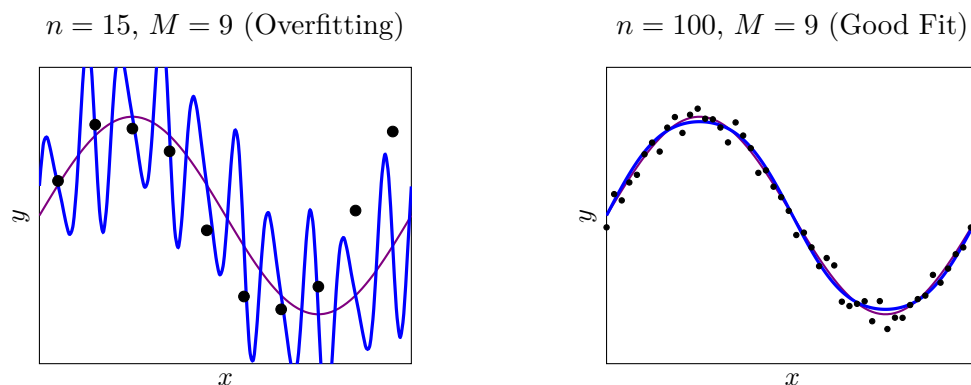


Figure 6: Effect of data size on generalization: With few data points ($n = 15$), a complex model ($M = 9$) overfits. With many data points ($n = 100$), the same model generalizes well. Purple: true function, Blue: fitted polynomial, Black dots: training data.

Part II

Model Evaluation and Regression Methods

14 Model Evaluation for Classification and Regression

14.1 Recap: Train, Validation, and Test Sets

The machine learning workflow involves three distinct datasets, each serving a specific purpose:

14.1.1 The Three Datasets

1. **Training Set:** Used to train the machine learning algorithm
 - The model learns patterns and relationships from this data
 - Parameters are optimized to minimize training error
2. **Validation Set:** Used during the learning phase
 - Also called the *mini test set*
 - Helps select the better performing algorithm among alternatives
 - Used to tune hyperparameters of the model
 - Provides feedback to adjust the model before final testing
3. **Test Set:** Used for final model evaluation
 - Provides an unbiased assessment of model performance
 - The model has never seen this data during training or validation
 - Tells us how well the model will perform on new, real-world data

14.2 The Importance of the Validation Set

The validation set plays a crucial role in the machine learning pipeline and must be kept **separate from the training set**.

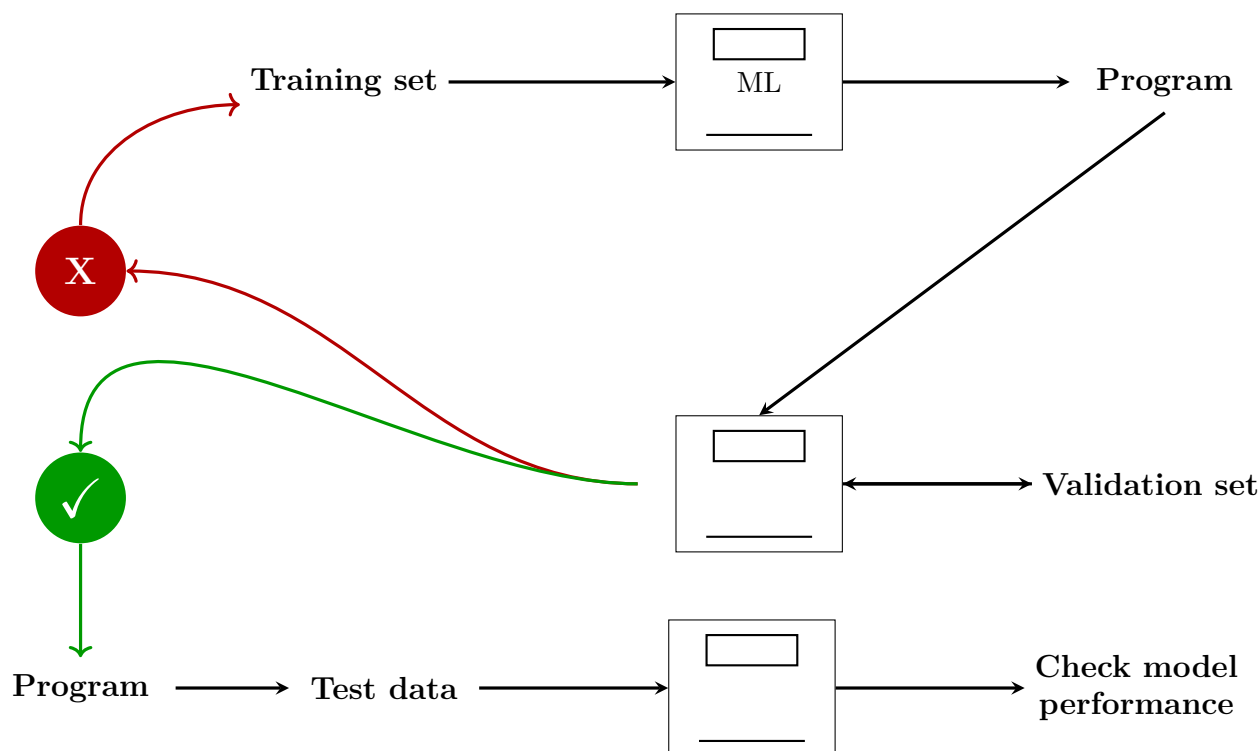


Figure 7: Machine Learning Workflow: Training set trains the model, validation set evaluates and provides feedback (thumbs down = retrain, thumbs up = proceed to testing), test set checks final performance

14.2.1 Why Separate the Validation Set?

The validation set is used for two main purposes:

1. **Algorithm Selection:** To pick the better performing algorithm
 - Compare different model architectures (e.g., decision trees vs. neural networks)
 - Select the model that performs best on unseen data
2. **Hyperparameter Tuning:** To decide the hyperparameters of an algorithm
 - Hyperparameters are configuration settings that are not learned from data
 - Examples: learning rate, regularization strength (λ), number of layers, tree depth
 - The validation set helps find the optimal hyperparameter values

Note. **ATTENTION:** Splitting the datasets into training and validation sets can be done randomly to avoid bias. However, there are some **specific rules** to apply when performing this split to ensure reliable model evaluation.

14.3 Specific Rules for Dataset Splitting

When splitting data into training, validation, and testing sets, you can have conflicting priorities. The key is to balance these priorities appropriately.

14.3.1 Priority 1: Accurate Error Estimation

Goal: Estimate future error (i.e., validation/testing error) as accurately as possible.

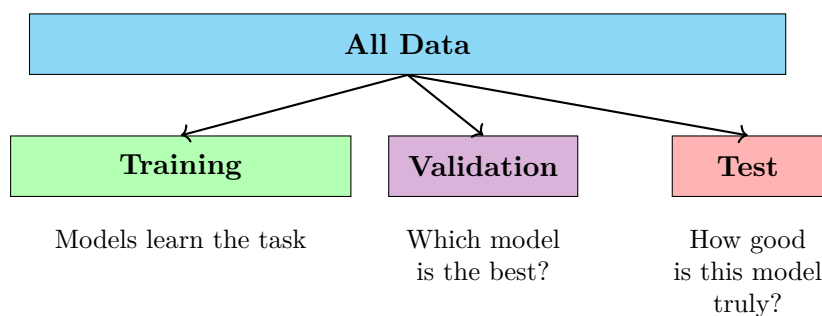


Figure 8: Dataset splitting: Training for learning, Validation for model selection, Test for final evaluation

How to achieve this: By making the **validation set as big as possible**.

(High confidence interval)

- A larger validation set provides more reliable error estimates
- Reduces variance in performance metrics
- Gives higher confidence that the measured performance reflects true generalization

14.3.2 Priority 2: Accurate Classifier Learning

Goal: Learn classifier as accurately as possible.

How to achieve this: By making the **training set as big as possible**.

(Better estimates, maybe better generalization)

- More training data allows the model to learn more robust patterns
- Reduces overfitting by providing diverse examples
- Generally leads to better generalization on unseen data

14.3.3 The Critical Rule

CRITICAL REQUIREMENT

Training and validation/testing instances **CANNOT OVERLAP !!!!**

- If the same data appears in both training and validation/test sets, the evaluation is invalid
- The model would be tested on data it has already seen during training
- This leads to overly optimistic performance estimates that don't reflect real-world performance
- Data leakage between sets is one of the most common mistakes in machine learning

Observation. The split between training and validation/test sets represents a fundamental trade-off:

- Larger training set → Better model learning, but less reliable error estimation
- Larger validation/test set → More reliable error estimation, but potentially worse model learning

Common splits include 70-15-15, 80-10-10, or 60-20-20 (train-validation-test), depending on the total dataset size and specific requirements.

15 Cross Validation

15.1 When Do We Need Cross Validation?

In cases where we don't have enough data to randomly split between training (e.g., 60% of the total data), validation (e.g., 20% of the total data), and testing (e.g., 20% of the total data), we need to use **cross-validation**.

Definition 15.1 (Cross Validation). Cross-validation is a resampling technique that involves reusing a portion of the training set itself to validate performance by retraining not just one, but N models.

15.2 Key Characteristics of Cross Validation

- **Multiple models:** Cross-validation involves training N models instead of just one
- **Computational cost:** Performing cross-validation requires N training sessions and can be more computationally demanding
- **Better use of limited data:** When data is scarce, cross-validation allows us to use the available data more efficiently for both training and validation

15.3 Fundamental Rules of Cross Validation

Cross validation must respect certain constraints to ensure valid model evaluation:

1. **No overlap:** Training and validation cannot overlap, but $n_{\text{train}} + n_{\text{validation}} = \text{constant}$
2. **Each instance used once per fold:** At each fold (step), you use each instance only in one set, so there is no overlapping within a single fold
3. **Every sample participates:** Every sample is in both training and testing but not at the same time
 - This reduces the chances of getting a biased training set
 - Ensures all data contributes to both training and validation across different folds

15.4 K-Fold Cross Validation

K-fold cross validation is the most common form of cross validation, where the dataset is divided into k equal parts (folds).

15.4.1 How K-Fold Cross Validation Works

In the case of k-fold cross-validation:

1. We split our dataset into k groups (folds)
2. We perform training k times on $(N/k)(k-1)$ data points
3. We measure performance on the last N/k data points (the validation fold)
4. Our cross-validation performance will be the **average of the performance of the k tests**

Note. Important constraint: Training set and validation set cannot overlap, but the sum of training and validation data is a constant (the total dataset size).

15.4.2 Visual Representation of K-Fold Cross Validation

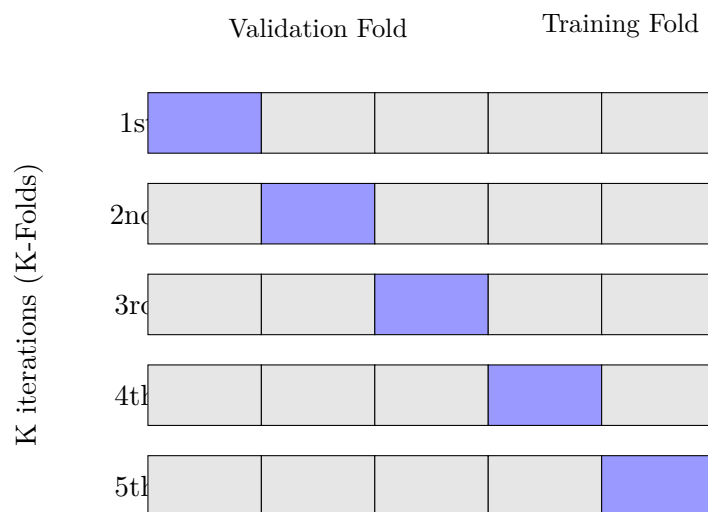


Figure 9: K-Fold Cross Validation: Each row represents one iteration where a different fold serves as validation (blue) while the rest serve as training (gray)

15.5 Example: 5-Fold Cross Validation

Let's examine a concrete example with 5 folds:

- **Step 1:** Randomly split the data into 5 folds
- **Step 2:** Test on each fold while training on 4 other folds (80% train, 20% test)
- **Step 3:** Average the results over 5 folds

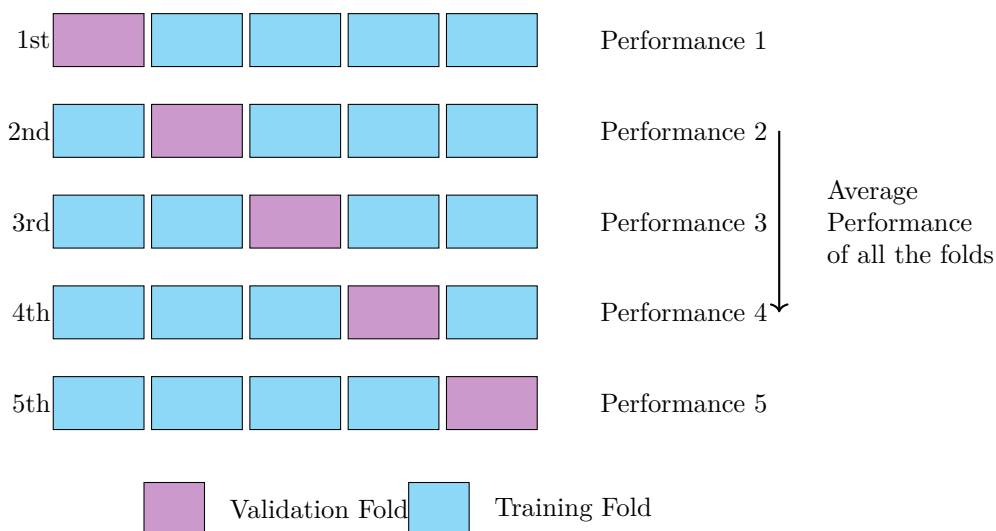


Figure 10: 5-Fold Cross Validation Example: Each fold serves as validation once, and the final performance is the average of all 5 iterations

15.5.1 The Process in Detail

For each of the 5 iterations:

1. **Iteration 1:** Use fold 1 as validation, folds 2-5 as training → Get Performance 1
2. **Iteration 2:** Use fold 2 as validation, folds 1,3-5 as training → Get Performance 2
3. **Iteration 3:** Use fold 3 as validation, folds 1-2,4-5 as training → Get Performance 3
4. **Iteration 4:** Use fold 4 as validation, folds 1-3,5 as training → Get Performance 4
5. **Iteration 5:** Use fold 5 as validation, folds 1-4 as training → Get Performance 5

Final Cross-Validation Performance:

$$\text{CV Performance} = \frac{1}{5} \sum_{i=1}^5 \text{Performance}_i$$

15.6 Advantages of Cross Validation

- **Better use of limited data:** Every data point is used for both training and validation
- **More reliable performance estimates:** Averaging over multiple folds reduces variance in the performance estimate
- **Reduces bias:** Every sample participates in validation, reducing the risk of a biased validation set
- **Detects overfitting:** If performance varies significantly across folds, it may indicate overfitting or data quality issues

15.7 Disadvantages of Cross Validation

- **Computationally expensive:** Requires training k models instead of one
- **Time-consuming:** Training time is multiplied by the number of folds
- **Not suitable for very large datasets:** When data is abundant, a simple train-validation-test split is more efficient

Observation. The choice of k (number of folds) involves a tradeoff:

- **Larger k :** More accurate performance estimate, but more computationally expensive
- **Smaller k :** Faster computation, but less reliable performance estimate
- Common choices: $k = 5$ or $k = 10$
- Extreme case: $k = N$ (Leave-One-Out Cross Validation) uses each single sample as validation once

16 Leave-One-Out Cross Validation

16.1 What is Leave-One-Out Cross Validation?

Leave-One-Out Cross Validation (LOOCV) is an extreme case of k -fold cross validation that is particularly useful when we have very few data points.

Definition 16.1 (Leave-One-Out Cross Validation). LOOCV is n -fold cross validation where n equals the total number of samples. In each iteration, we train on all $(n - 1)$ samples and test on exactly 1 instance.

16.2 How LOOCV Works

The process involves:

1. Take the entire dataset of n samples
2. For each sample i (where $i = 1, 2, \dots, n$):
 - Use sample i as the validation set (1 sample)
 - Use all other samples ($n - 1$) as the training set
 - Train the model and evaluate on the single validation sample
3. Repeat this process n times (once for each sample)
4. Average the performance across all n experiments

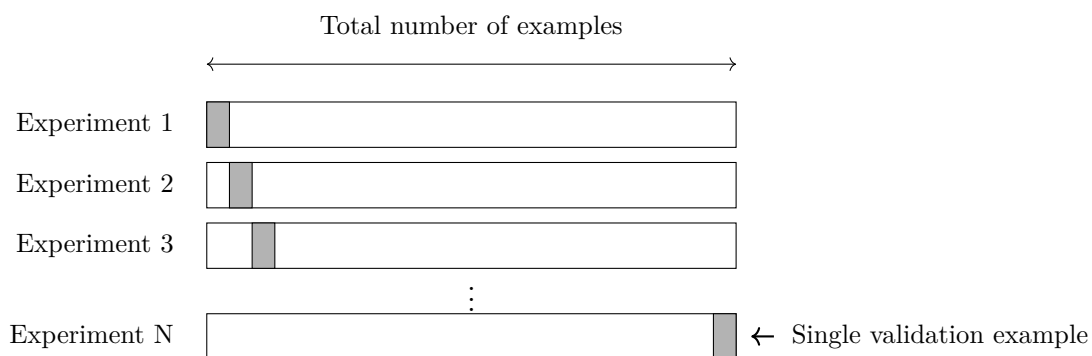


Figure 11: Leave-One-Out Cross Validation: Each experiment uses exactly one sample for validation (gray) and all others for training (white)

16.3 Pros and Cons of LOOCV

16.3.1 Advantages

- **Best possible classifier:** The model is learned from $n - 1$ training examples, which is the maximum possible training data
- **Maximizes training data:** Each model uses almost all available data for training
- **Unbiased estimate:** Provides a nearly unbiased estimate of model performance
- **Deterministic:** No randomness in the splitting process (every sample is used exactly once as validation)

16.3.2 Disadvantages

- **High computational cost:** Must re-learn everything for n times, where n is the total number of samples
- **Time-consuming:** For large datasets, LOOCV becomes impractical
- **High variance:** Performance estimates can have high variance, especially with small datasets

Note. LOOCV is most appropriate when:

- The dataset is very small (e.g., $n < 50$)
- Computational resources are available

- You need the most accurate performance estimate possible

For larger datasets, standard k -fold cross validation (with $k = 5$ or $k = 10$) is typically preferred due to its better balance between computational cost and performance estimation accuracy.

17 Stratification in Cross Validation

17.1 The Problem: Imbalanced Classes

In many real-world datasets, class labels are not evenly distributed. For example, in a medical diagnosis dataset, healthy patients might vastly outnumber sick patients. When performing cross validation on such imbalanced datasets, random splitting can lead to folds with very different class distributions.

17.2 What is Stratification?

Definition 17.1 (Stratification). Stratification is a technique to keep class labels **balanced across training and validation sets** during cross validation. Instead of randomly dividing the dataset, we ensure that each fold maintains approximately the same proportion of samples for each class as the complete dataset.

17.3 How Stratified K-Fold Works

The stratification process works as follows:

1. **Divide by class:** Instead of taking the dataset and dividing it randomly into K parts, take the dataset and divide it into individual classes
2. **Split each class:** Then for each class, divide the instances into K parts
3. **Assemble folds:** Assemble the i -th part from all classes to make the i -th fold
4. **Important:** It is still random! The splitting within each class is random, but the proportions are maintained

17.4 Example: Stratified 5-Fold Cross Validation

Let's consider a concrete scenario with an imbalanced dataset containing 300 samples, divided into three classes (A, B, and C), where Class A has more samples than the other two.

17.4.1 Imbalanced Dataset

- **Class A:** 200 samples (66.7%)
- **Class B:** 50 samples (16.7%)
- **Class C:** 50 samples (16.7%)
- **Total:** 300 samples

17.4.2 Split the Dataset into Five Folds with Stratification

Each fold will have 60 samples total, maintaining the class proportions:

- **Fold 1:** 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 2:** 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 3:** 60 samples (Class A: 40, Class B: 10, Class C: 10)

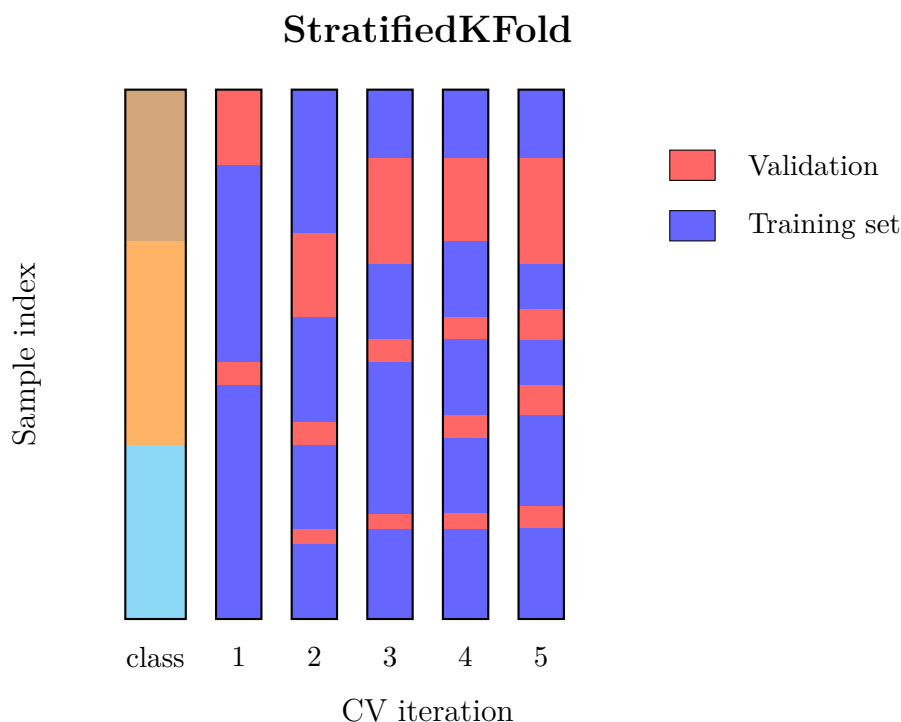


Figure 12: Stratified K-Fold: Each fold maintains the same class proportions across all folds

- **Fold 4:** 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 5:** 60 samples (Class A: 40, Class B: 10, Class C: 10)

Notice that each fold maintains the same 66.7%-16.7%-16.7% class distribution as the original dataset.

17.4.3 Train and Validate

The cross-validation process proceeds as follows:

1. **Iteration 1:** Train on Folds 1, 2, 3, and 4, Validate on Fold 5
2. **Iteration 2:** Train on Folds 2, 3, 4, and 5, Validate on Fold 1
3. **Iteration 3:** Train on Folds 1, 3, 4, and 5, Validate on Fold 2
4. **Iteration 4:** Train on Folds 1, 2, 4, and 5, Validate on Fold 3
5. **Iteration 5:** Train on Folds 1, 2, 3, and 5, Validate on Fold 4

17.4.4 Average the Performance Metrics

- Calculate performance metrics for each iteration
- Average these metrics to evaluate the model's performance robustly

17.5 Why Stratification Matters

- **Prevents biased folds:** Without stratification, some folds might have very few (or no) examples of minority classes
- **Consistent evaluation:** Each fold represents the overall dataset distribution, leading to more reliable performance estimates

- **Better for imbalanced datasets:** Particularly important when dealing with class imbalance
- **Reduces variance:** Performance estimates have lower variance across folds

Note. **When to use stratification:**

- Always use stratification for classification problems with imbalanced classes
- It's generally a good practice even for balanced datasets
- Most machine learning libraries (e.g., scikit-learn) provide stratified cross-validation functions
- For regression problems, stratification is not applicable (since there are no discrete classes)

Observation. Stratification ensures that:

- Each fold is a good representative of the whole dataset
- Minority classes are adequately represented in both training and validation sets
- Performance metrics are more stable and reliable across folds
- The model sees a balanced distribution of classes during each training iteration

18 Evaluation Measures

Once we have trained our models and performed cross-validation, we need to evaluate their performance. Different tasks require different evaluation metrics.

18.1 Why Do We Need Evaluation Measures?

Evaluation measures serve two primary purposes:

1. **To decide if our model is performing well:** Assess whether the model meets the requirements for the task
2. **To decide out of many models, which one performs better than the other:** Compare different models and select the best one for deployment

18.2 Evaluation Measures by Task Type

Different machine learning tasks require different evaluation approaches:

18.2.1 Classification

Question: How often does our model classify something right/wrong?

Classification metrics measure the correctness of predictions, focusing on whether the predicted class matches the actual class.

18.2.2 Regression

Question: How close is our model to what we are trying to predict?

Regression metrics measure the distance or error between predicted continuous values and actual values.

18.2.3 Clustering

Question: How well does our model describe our data?

Clustering evaluation is generally very hard because there's often no ground truth to compare against. Metrics must assess the quality of discovered patterns.

19 Evaluation Measures for Classification

19.1 Types of Classification Problems

A classification model takes a set of features as input and produces one or more classes as output.

Definition 19.1 (Classification Types). Classification can be categorized into different types based on the number of classes:

- **Binary Classification:** In the case of two classes (0/1) or one vs rest (e.g., apples vs other fruits)
- **Multi-class Classification:** In the case of having N classes to choose from (e.g., cat, dog, wolf, etc.)
- **Multi-label Classification:** A problem where an example can belong to multiple classes simultaneously

19.2 The Confusion Matrix

The confusion matrix is a fundamental tool for evaluating binary classification models. It provides a complete picture of how the model performs across different types of predictions.

19.2.1 Confusion Matrix for Binary Classification

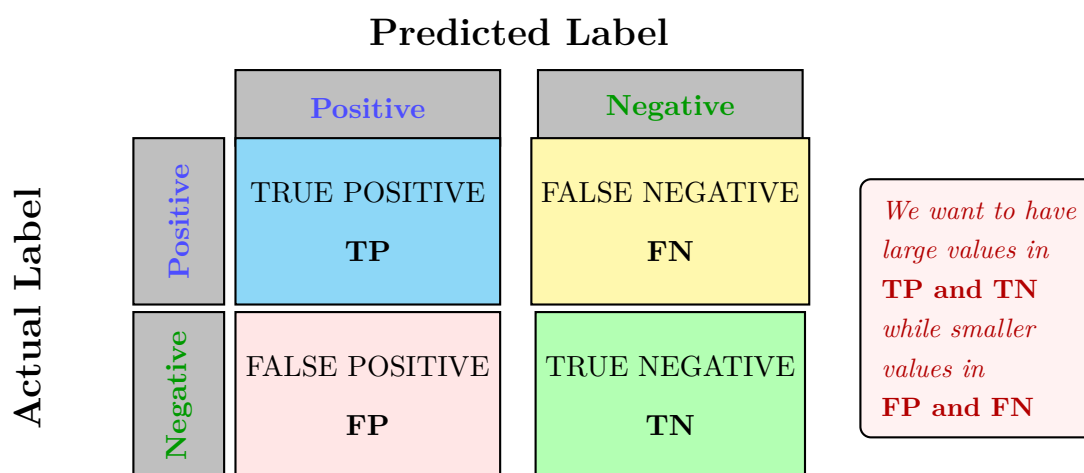


Figure 13: Confusion Matrix for Binary Classification

19.2.2 Understanding the Confusion Matrix

The confusion matrix has four components:

- **True Positive (TP):** The model correctly predicted the positive class
 - Actual label: Positive
 - Predicted label: Positive

- Good prediction!
- **True Negative (TN)**: The model correctly predicted the negative class
 - Actual label: Negative
 - Predicted label: Negative
 - Good prediction!
- **False Positive (FP)**: The model incorrectly predicted positive (Type I error)
 - Actual label: Negative
 - Predicted label: Positive
 - Bad prediction! (False alarm)
- **False Negative (FN)**: The model incorrectly predicted negative (Type II error)
 - Actual label: Positive
 - Predicted label: Negative
 - Bad prediction! (Missed detection)

Note. **Goal:** We want to maximize TP and TN (correct predictions) while minimizing FP and FN (incorrect predictions).

19.3 Classification Error and Accuracy

From the confusion matrix, we can derive several important metrics.

19.3.1 Classification Error

Definition 19.2 (Classification Error). The classification error (also called misclassification rate) measures the proportion of incorrect predictions:

$$\text{Classification Error} = \frac{FP + FN}{TP + TN + FP + FN}$$

This represents the fraction of all predictions that were wrong.

19.3.2 Accuracy

Definition 19.3 (Accuracy). Accuracy measures the proportion of correct predictions:

$$\text{Accuracy} = 1 - \text{Error} = \frac{TP + TN}{TP + TN + FP + FN}$$

This represents the fraction of all predictions that were correct.

19.4 The Problem with Accuracy: Imbalanced Classes

While accuracy is a basic measure of "goodness" of a classifier, it can be very misleading when dealing with imbalanced classes.

ATTENTION!

Accuracy is **very misleading** if we have **imbalanced classes**

19.4.1 Example: The Imbalanced Dataset Problem

Consider a dataset with severe class imbalance:

- 90 samples for class "NO"
- 10 samples for class "YES"

Naive Strategy: A trivial classifier that predicts "NO" for every sample would achieve:

$$\text{Accuracy} = \frac{90}{100} = 90\%$$

The Problem: Despite having 90% accuracy, this model is completely useless! It cannot classify any data belonging to the "YES" class. The model has learned nothing meaningful about the data.

Observation. This example illustrates why accuracy alone is insufficient for evaluating classifiers, especially with imbalanced datasets. We need additional metrics that provide a more nuanced view of model performance across all classes.

Note. When dealing with imbalanced datasets, always consider:

- Using stratified cross-validation
- Examining the confusion matrix in detail
- Using additional metrics beyond accuracy (precision, recall, F1-score)
- Considering class-specific performance
- Using techniques like oversampling, undersampling, or class weights

19.5 Misses and False Alarms

Beyond accuracy, we need more specific metrics to understand model performance. These metrics focus on different aspects of classification errors.

19.5.1 False Alarm Rate (False Positive Rate)

Definition 19.4. False Alarm Rate = False Positive Rate = $\frac{FP}{FP+TN}$

This represents the *percentage of negative samples we misclassified as positive*.

19.5.2 Miss Rate (False Negative Rate)

Definition 19.5. Miss Rate = False Negative Rate = $\frac{FN}{TP+FN}$

This represents the *percentage of positive samples we misclassified as negative*.

19.5.3 Recall (True Positive Rate)

Definition 19.6. Recall = True Positive Rate = $\frac{TP}{TP+FN}$

This represents the *percentage of positive samples we classified correctly*.

It is also known as **Sensitivity** and can be expressed as: Recall = 1 - Miss Rate

Observation. Recall answers the question: "Of all the actual positive cases, how many did we identify?"

19.5.4 Precision

Definition 19.7. Precision $= \frac{TP}{TP+FP}$

This represents the *percentage positive out of what we predicted was positive*.

Observation. Precision answers the question: "Of all the cases we predicted as positive, how many were actually positive?"

19.6 Cost of the Task

In practice, we typically do not use evaluation metrics like accuracy, recall, or precision alone. Instead, we declare a combination of them together based on the specific requirements of the task.

19.6.1 The Need for a Single Evaluation Measure

- In order to optimize a learner automatically (i.e., during training), we need a **single evaluation measure**
- How do we decide that single metric?
 - **Domain specific!** – It depends on the task

19.6.2 Weighting Errors by Cost

Depending on the **cost of the task**, we can decide whether we take care more about having:

- *Less false positives*, or
- *Less false negatives*

This is achieved through weighting them:

$$\text{Cost} = C_{FP} \cdot FP + C_{FN} \cdot FN \quad (1)$$

where C_{FP} and C_{FN} are the costs associated with false positives and false negatives, respectively.

Example 19.1. Medical Diagnosis (Cancer Detection):

- False Negative (missing a cancer diagnosis): Very high cost – patient doesn't receive treatment
- False Positive (false alarm): Lower cost – patient undergoes additional tests
- Therefore: $C_{FN} \gg C_{FP}$, we prioritize high recall

Example 19.2. Spam Email Filter:

- False Positive (marking important email as spam): High cost – user misses important messages
- False Negative (spam gets through): Lower cost – minor annoyance
- Therefore: $C_{FP} > C_{FN}$, we prioritize high precision

19.7 F-Measure (F1 Score)

The F-measure combines precision and recall into a single metric, providing a balance between the two.

Definition 19.8. The **F1 Score** is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

Harmonic mean of precision and recall

One of the most frequently used metrics in machine learning

19.7.1 Properties of F1 Score

- (+) If you do some mathematics, you will see that the F1 measure is sort of an accuracy without TN
- (+) Used frequently in information retrieval systems
- (+) Provides a single number that balances precision and recall
- **Range:** $F_1 \in [0, 1]$, where 1 is perfect and 0 is the worst

Note. The harmonic mean (used in F1) is always less than or equal to the arithmetic mean. This means F1 score gives more weight to the lower value between precision and recall, ensuring both metrics need to be reasonably high for a good F1 score.

19.7.2 Generalized F-Measure

The F1 score can be generalized to F_β score, which allows adjusting the relative importance of precision and recall:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (3)$$

where:

- $\beta < 1$: Emphasizes precision
- $\beta > 1$: Emphasizes recall
- $\beta = 1$: Equal weight (standard F1 score)

19.8 Multiclass Classification – Evaluation

For multiclass classification problems (more than two classes), we need to extend our evaluation metrics.

19.8.1 Multiclass Confusion Matrix

- n_{ij} is the number of examples with actual label (true label) y_i and predicted label y_j
- The **main diagonal** contains true positives for each class
- The **sum of off-diagonal elements along a column** is the number of false positives for the column label

- The **sum of off-diagonal elements along a row** is the number of false negatives for the row label

19.8.2 Multiclass Metrics

For each class i , we can compute:

$$FP_i = \sum_{j \neq i} n_{ji} \quad FN_i = \sum_{j \neq i} n_{ij} \quad (4)$$

$$\text{Precision}_i = \frac{n_{ii}}{n_{ii} + FP_i} \quad \text{Recall}_i = \frac{n_{ii}}{n_{ii} + FN_i} \quad (5)$$

$$\text{Multiclass Accuracy (MAcc)} = \frac{\sum_i n_{ii}}{\sum_i \sum_j n_{ij}} \quad (6)$$

19.8.3 Macro vs. Micro Averaging

When reporting overall metrics for multiclass problems, we have two main approaches:

- **Macro-averaging:** Compute metric for each class independently, then take the average

$$\text{Macro-Precision} = \frac{1}{C} \sum_{i=1}^C \text{Precision}_i \quad (7)$$

This gives equal weight to each class regardless of size

- **Micro-averaging:** Aggregate the contributions of all classes to compute the average metric

$$\text{Micro-Precision} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)} \quad (8)$$

This gives equal weight to each sample

19.9 ROC Curve – Receiver Operating Characteristic

The ROC curve is a powerful tool for evaluating and comparing binary classifiers, especially when the decision threshold can be varied.

19.9.1 What is the ROC Curve?

The **ROC curve** is a graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied.

- **X-axis:** False Positive Rate (FPR) = $\frac{FP}{FP+TN}$
- **Y-axis:** True Positive Rate (TPR) = Recall = $\frac{TP}{TP+FN}$

19.9.2 Uses of ROC Curves

The ROC curve allows us to:

- **Compare different classifiers:** Different models can be plotted on the same ROC space
- **Estimate the false positive rate / negative rate depending on the thresholds** that we set to classify
- Visualize the trade-off between sensitivity (recall) and specificity

19.9.3 Understanding the ROC Space

- **Perfect Classifier:** A point at $(0, 1)$ – 100% TPR, 0% FPR
- **Random Classifier:** Lies on the diagonal line from $(0,0)$ to $(1,1)$
- **Better Classifiers:** Curves that bow toward the upper left corner
- **Worse Classifiers:** Curves below the diagonal (worse than random guessing)

19.9.4 Area Under the ROC Curve (AUC)

Definition 19.9. The **Area Under the ROC Curve (AUC)** is a single scalar value that summarizes the performance of a classifier across all possible thresholds.

$$\text{AUC} \in [0, 1] \quad (9)$$

Goal: Maximize the Area Under the ROC Curve (AUC)

19.9.5 Interpreting AUC Values

- **AUC = 1.0:** Perfect classifier
- **AUC = 0.9 - 1.0:** Excellent
- **AUC = 0.8 - 0.9:** Good
- **AUC = 0.7 - 0.8:** Fair
- **AUC = 0.6 - 0.7:** Poor
- **AUC = 0.5:** No better than random guessing
- **AUC < 0.5:** Worse than random (predictions are inverted)

Observation. The AUC can be interpreted as the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance.

19.9.6 Advantages of ROC and AUC

- **Threshold-independent:** Evaluates the model across all possible thresholds
- **Class imbalance robust:** Unlike accuracy, AUC is less sensitive to class imbalance
- **Single metric:** Provides a single value for model comparison
- **Visual interpretation:** The ROC curve provides intuitive visualization of classifier performance

20 Regression Evaluation Metrics

While classification tasks allow us to count correct and incorrect predictions, regression tasks require different evaluation approaches since we're predicting continuous values.

20.1 The Fundamental Difference

- **Classification:** We can count how often we are correct or wrong in our predictions
- **Regression:** We cannot do that counting!

- Predicting y_i (not discrete, continuous value) from inputs x_i
- The question is not "*how many times your method is wrong*" but "*how much your method is wrong with respect to the groundtruth-labels*"

Key Difference

Classification: Count errors $\leftrightarrow$ Regression: Measure error magnitude

20.2 Mean Squared Error (MSE)

The Mean Squared Error is one of the most commonly used metrics for regression tasks.

Definition 20.1 (Mean Squared Error).

$$MSE(f) = \frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2 \quad (10)$$

where:

- f is the model that takes a feature vector x as input and generates a prediction $f(x_i)$
- i , ranging from 1 to N , denotes the index of a sample in the dataset
- y_i is the ground truth (actual value)

20.2.1 Properties of MSE

- The sum of predictions minus real values over N indicates the average
- **Squaring** removes the negative sign and gives more weight to larger differences
- **The lower the MSE, the better**
- MSE is **influenced by outliers**, i.e., data values that are abnormally distant from the true regression line
- MSE is **differentiable**, which makes it suitable for gradient-based optimization algorithms

Observation. The squaring operation in MSE has two important effects:

1. It ensures all errors are positive (no cancellation between positive and negative errors)
2. It penalizes large errors more heavily than small errors (quadratic penalty)

20.3 Mean Absolute Error (MAE)

The Mean Absolute Error provides an alternative way to measure regression performance.

Definition 20.2 (Mean Absolute Error).

$$MAE(f) = \frac{1}{N} \sum_{i=1}^N |f(x_i) - y_i| \quad (11)$$

20.3.1 Properties of MAE

- MAE is similar to MSE in that it returns absolute values of the residuals $f(x_i) - y_i$ without squaring
- It does not consider the direction of the error, which means we will not know whether negative or positive errors weigh more on the overall average

- **MAE is more robust to outliers** precisely due to the absence of squaring of distant forecast error values
- However, MAE is **not differentiable** at zero. It means that we cannot take the derivative of it, and taking derivatives is important if you want to build an algorithm that minimizes the function. Because of this, it is used a lot less than MSE.

MSE vs MAE Comparison:

- **MSE:** Differentiable, penalizes large errors more, sensitive to outliers
- **MAE:** Not differentiable at zero, treats all errors equally, robust to outliers

20.4 Choosing Between MSE and MAE

The choice between MSE and MAE depends on the specific requirements of your regression task:

Metric	Use When	Avoid When
MSE	• Large errors are particularly undesirable • You need gradient-based optimization • Outliers should be heavily penalized	• Dataset contains many outliers • All errors should be treated equally
MAE	• Outliers are present in the data • All errors should be weighted equally • You want a more interpretable metric	• You need differentiability for optimization • Large errors should be penalized more

Table 4: Guidelines for choosing between MSE and MAE

20.5 Other Regression Metrics

Beyond MSE and MAE, there are other useful metrics for evaluating regression models:

20.5.1 Root Mean Squared Error (RMSE)

$$RMSE(f) = \sqrt{\frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2} \quad (12)$$

RMSE is simply the square root of MSE. It has the advantage of being in the same units as the target variable, making it more interpretable.

20.5.2 R-Squared (R^2) Score

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - f(x_i))^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (13)$$

where $\bar{y}$ is the mean of the observed values. R^2 represents the proportion of variance in the dependent variable that is predictable from the independent variables. It ranges from 0 to 1, with 1 indicating perfect prediction.

Note. In practice, it's often beneficial to report multiple metrics to get a comprehensive view of model performance. For example, reporting both MSE (or RMSE) and MAE can provide insights into both the typical error magnitude and the presence of outliers.

Part III

Bayes Classifier and Nonparametric Classifiers

21 Bayes Classifier

21.1 Introduction

The Bayes Classifier represents a fundamental **statistical approach** for classification problems. It provides an optimal decision-making framework when all relevant probabilities are known.

Core Hypothesis

The decision problem is cast in **probabilistic terms**, where all relevant **probabilities are known**.

21.1.1 Goal of Bayes Classifier

The primary objectives are:

- **Discriminate** between different decision rules using the probabilities and their associated costs
- **Estimate** the **joint probability** $p(\mathbf{x}, y)$ from the training data set

Key Concept

The Bayes Classifier aims to estimate the joint probability distribution $p(\mathbf{x}, y)$ to make optimal classification decisions.

21.2 An Easy Example

Let's start with a simple example to understand the basic principles.

Example 21.1 (Two-Class Classification). Let y be what we want to classify, and it is to be probabilistically described.

There are **two classes** y_1 and y_2 which correspond to two possible states:

a) $P(y = y_1) = 0.7$

b) $P(y = y_2) = 0.3$

These are called **a-priori** or **prior probabilities**.

Scenario: There are no measurements or observations available to help make a decision about which class y belongs to.

21.2.1 Decision Rule

Given only the prior probabilities, the optimal decision rule is straightforward:

Decision Rule:

Decide y_1 if $P(y_1) > P(y_2)$; otherwise decide y_2

Observation. Given the probabilities (70% for y_1 and 30% for y_2), the decision rule would lead to classifying y as y_1 since 0.7 is greater than 0.3.

This makes intuitive sense: without any additional information, we should choose the more probable class.

21.3 Another Example: Adding Measurements

Now let's consider a more realistic scenario where we have measurements to help us decide.

Example 21.2 (Classification with Measurements). Having the previous hypothesis and additionally a **single measurement** $\mathbf{x}$, which is a random variable that depends on the class y_j , we can get:

$$p(\mathbf{x}|y_j)_{j=1,2} = \text{Likelihood or class-conditional probability density function}$$

i.e., the probability of having the measurement $\mathbf{x}$ knowing that y is y_j .

Observation. When we fix the measurement $\mathbf{x}$, the higher $p(\mathbf{x}|y_j)$ is, the more likely y_j is the correct state.

This is the key insight: measurements provide evidence that can shift our belief from the prior probabilities toward the class that better explains the observed data.

21.3.1 Visual Interpretation

The class-conditional probability density functions can be visualized as distributions over the measurement space. For two classes:

- Each class has its own probability distribution $p(\mathbf{x}|y_j)$
- These distributions may overlap in the measurement space
- Where they overlap, we need a decision rule to determine which class is more likely
- The shape and position of these distributions reflect how the measurement $\mathbf{x}$ is distributed for each class

21.4 Bayes Formula

The Bayes formula provides the mathematical framework for combining prior knowledge with observed evidence.

Bayes Formula

Assuming known $P(y_j)$ and $p(\mathbf{x}|y_j)$, the decision of ω becomes, for Bayes:

$$p(y_j, \mathbf{x}) = P(y_j|\mathbf{x})p(\mathbf{x}) = p(\mathbf{x}|y_j)P(y_j)$$

that is:

$$P(y_j|\mathbf{x}) = \frac{p(\mathbf{x}|y_j)P(y_j)}{p(\mathbf{x})} \propto p(\mathbf{x}|y_j)P(y_j)$$

where:

- $P(y_j) = \mathbf{Prior}$
- $p(\mathbf{x}|y_j) = \mathbf{Likelihood}$
- $P(y_j|\mathbf{x}) = \mathbf{Posterior}$
- $p(\mathbf{x}) = \sum_{j=1}^J p(\mathbf{x}|y_j)P(y_j) = \mathbf{Evidence}$

21.4.1 Understanding the Components

Definition 21.1 (Bayes Formula Components). • **Prior** $P(y_j)$: Our initial belief about the probability of class y_j before seeing any data

- **Likelihood** $p(\mathbf{x}|y_j)$: The probability of observing measurement $\mathbf{x}$ given that the true class is y_j
- **Posterior** $P(y_j|\mathbf{x})$: Our updated belief about the probability of class y_j after observing measurement $\mathbf{x}$
- **Evidence** $p(\mathbf{x})$: The total probability of observing measurement $\mathbf{x}$ across all classes (acts as a normalization constant)

Observation. The evidence $p(\mathbf{x})$ is the same for all classes, which is why we can use the proportionality:

$$P(y_j|\mathbf{x}) \propto p(\mathbf{x}|y_j)P(y_j)$$

This means for classification, we only need to compare $p(\mathbf{x}|y_j)P(y_j)$ across different classes without computing the evidence explicitly.

21.4.2 Bayes Decision Rule

Optimal Bayes Decision Rule

Choose class y_j that maximizes the posterior probability:

$$\hat{y} = \arg \max_j P(y_j|\mathbf{x}) = \arg \max_j p(\mathbf{x}|y_j)P(y_j)$$

Note. The Bayes Classifier is optimal in the sense that it minimizes the probability of misclassification when all probabilities are known exactly. However, in practice, we must estimate these probabilities from data, which introduces approximation errors.

21.5 Bayes Decision Rule: Detailed Analysis

Bayes Formula Relationship

$$P(y_j|\mathbf{x}) = \frac{p(\mathbf{x}|y_j)P(y_j)}{p(\mathbf{x})} \iff \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

21.5.1 Understanding the Posterior

- The posterior or **a-posteriori probability** is the probability that y is y_j given the observation $\mathbf{x}$
- The most important factor is the product likelihood $\times$ prior
- The evidence $p(\mathbf{x})$ is simply a scale factor, which ensures that:

$$\sum_j P(y_j|\mathbf{x}) = 1$$

Observation. Since $p(\mathbf{x})$ is the same for all classes, it acts as a normalization constant. For classification purposes, we can ignore it and simply compare the numerators $p(\mathbf{x}|y_j)P(y_j)$ across different classes.

21.5.2 The Decision Rule

From the formula of Bayes derives the **Bayes' decision rule**:

Bayes Decision Rule

Decide y_1 if $P(y_1|\mathbf{x}) > P(y_2|\mathbf{x})$, and y_2 otherwise

Equivalently, since we can ignore the evidence:

Decide y_1 if $p(\mathbf{x}|y_1)P(y_1) > p(\mathbf{x}|y_2)P(y_2)$, and y_2 otherwise

22 Naïve Bayes Classifier

The Naïve Bayes Classifier is a practical implementation of Bayes' theorem that makes a simplifying assumption about feature independence.

22.1 Core Principles

Naïve Bayes Classifier

A probabilistic classification algorithm based on **Bayes' theorem** that:

- Assumes **independence** among features
- Calculates the **posterior probability** using:

$$P(y_j|\mathbf{x}) = \frac{p(\mathbf{x}|y_j)P(y_j)}{p(\mathbf{x})}$$

22.2 The Naïve Independence Assumption

The key assumption is that features $x_1, x_2, \dots, x_n$ are **independent** given the class label ω :

$$P(\mathbf{x}|y) = P(x_1|y) \cdot P(x_2|y) \cdots P(x_n|y) = \prod_{i=1}^n P(x_i|y) \quad (14)$$

Observation. This is called "naïve" because in reality, features are often correlated. However, despite this simplification, Naïve Bayes often performs surprisingly well in practice, especially for text classification tasks.

22.2.1 Training the Classifier

Key Point: Probability of each class is determined from training data.

The training process involves:

1. Estimating the prior probabilities $P(y_j)$ from the class frequencies in the training data
2. Estimating the conditional probabilities $P(x_i|y_j)$ for each feature given each class

22.3 Example: Email Spam Classification

Let's work through a complete example to understand how Naïve Bayes works in practice.

Example 22.1 (Spam Detection). We want to classify an email as either:

- **Class 1:** Spam
- **Class 2:** Not spam (Ham)

We'll use a few features (words) that appear in the email.

22.3.1 Training Data

Consider the following training dataset:

Email	Contains "Free"?	Contains "Win"?	Class
E1	Yes	Yes	Spam
E2	Yes	No	Spam
E3	No	Yes	Spam
E4	No	No	Ham
E5	No	No	Ham

Table 5: Training data for spam classification

22.3.2 Step 1: Compute Class Priors

From the training data:

$$P(\text{Spam}) = \frac{3}{5} \quad (15)$$

$$P(\text{Ham}) = \frac{2}{5} \quad (16)$$

Observation. The prior probabilities reflect the class distribution in our training set: 3 out of 5 emails are spam, and 2 out of 5 are ham.

22.3.3 Step 2: Estimate Conditional Probabilities

For each feature (word) and class, we estimate how often the word appears when the email belongs to that class:

For "Free":

$$P(\text{Free} = \text{Yes}|\text{Spam}) = \frac{2}{3} \quad (17)$$

$$P(\text{Free} = \text{Yes}|\text{Ham}) = \frac{0}{2} = 0 \quad (18)$$

For "Win":

$$P(\text{Win} = \text{Yes}|\text{Spam}) = \frac{2}{3} \quad (19)$$

$$P(\text{Win} = \text{Yes}|\text{Ham}) = \frac{0}{2} = 0 \quad (20)$$

Note. Note that we also need the probabilities for the words *not* appearing:

$$P(\text{Free} = \text{No}|\text{Spam}) = 1 - \frac{2}{3} = \frac{1}{3}$$

$$P(\text{Win} = \text{No}|\text{Spam}) = 1 - \frac{2}{3} = \frac{1}{3}$$

$$P(\text{Free} = \text{No}|\text{Ham}) = 1 - 0 = 1$$

$$P(\text{Win} = \text{No}|\text{Ham}) = 1 - 0 = 1$$

22.3.4 Step 3: Classify a New Email

Validation/Testing: Consider a new email that contains "Free" but does **not** contain "Win".

We compute (ignoring the denominator (evidence) since it's the same for both classes):

$$P(\text{Spam}|\text{Free, no Win}) \propto P(\text{Spam}) \cdot P(\text{Free}|\text{Spam}) \cdot P(\text{no Win}|\text{Spam}) \quad (21)$$

$$= \frac{3}{5} \times \frac{2}{3} \times \left(1 - \frac{2}{3}\right) \quad (22)$$

$$= \frac{3}{5} \times \frac{2}{3} \times \frac{1}{3} = \frac{2}{15} \quad (23)$$

$$P(\text{Ham}|\text{Free, no Win}) \propto P(\text{Ham}) \cdot P(\text{Free}|\text{Ham}) \cdot P(\text{no Win}|\text{Ham}) \quad (24)$$

$$= \frac{2}{5} \times 0 \times 1 = 0 \quad (25)$$

Prediction: The model predicts **Spam**, because its probability is greater.

$$P(\text{Spam}|\text{Free, no Win}) = \frac{2}{15} > P(\text{Ham}|\text{Free, no Win}) = 0$$

22.4 The Zero Probability Problem

Observation. Notice that $P(\text{Free}|\text{Ham}) = 0$ caused the entire probability for Ham to become zero. This is a common problem in Naïve Bayes called the **zero probability problem**.

22.4.1 Solution: Laplace Smoothing

To avoid zero probabilities, we use **Laplace smoothing** (also called additive smoothing):

$$P(x_i|y_j) = \frac{\text{count}(x_i, y_j) + \alpha}{\text{count}(y_j) + \alpha \cdot n} \quad (26)$$

where:

- α is the smoothing parameter (typically $\alpha = 1$)
- n is the number of possible values for feature x_i

Note. Laplace smoothing ensures that no probability is exactly zero, which prevents the entire product from becoming zero when a feature value hasn't been observed in the training data for a particular class.

22.5 Advantages and Disadvantages

Advantages	Disadvantages
• Simple and fast to train and predict • Works well with high-dimensional data • Requires relatively small training data • Performs well for text classification • Provides probabilistic predictions	• Independence assumption is often violated • Can be outperformed by more sophisticated models • Zero probability problem requires smoothing • Cannot learn feature interactions • Sensitive to irrelevant features

Table 6: Advantages and disadvantages of Naïve Bayes

22.6 Classification Rule Summary

Naïve Bayes Classification Rule

Classify an observation $\mathbf{x}$ to class y_j if:

$$P(y_j|\mathbf{x}) = \frac{p(\mathbf{x}|y_j)P(y_j)}{p(\mathbf{x})} \text{ is maximized}$$

22.6.1 Key Points

Advantages:

- Fast to train and predict, suitable for large datasets
- Works well with high-dimensional data

Limitations:

- Independence assumption may not hold in real-world scenarios, leading to suboptimal performance
- Zero probability problem: requires techniques (like Laplace smoothing) for unseen feature-class combinations

22.7 Problems with Bayesian Classifiers

To create an optimal classifier that uses the Bayesian decision rule, you need to know:

1. **Prior probabilities** $P(y_j)$
2. **Class-conditional densities (likelihood)** $p(\mathbf{x}|y_j)$

Observation. The performance of a classifier **strongly** depends on the **goodness** of these components.

The Fundamental Problem

BUT PRACTICALLY ALL THIS INFORMATION IS NEVER AVAILABLE!!

22.7.1 What We Actually Have

More often we only have:

- A vague knowledge of the problem, from which to extract vague **a-priori probabilities**
- Some particularly representative patterns, training data, used to train the classifier (**often too few or sparse!**)

Observation. • Estimating a-priori probabilities is usually not particularly difficult

- Estimating conditional densities (likelihood) is **more complex** → needs approximations

22.8 Challenges in Estimating Conditional Densities

Estimating conditional densities (likelihood) is more complex and needs approximations due to several challenges:

1. **High-dimensional problem:** Data can have many features → exponential combinations
 - With d features, each with k possible values, there are k^d possible feature combinations
 - This leads to the **curse of dimensionality**
2. **Zero probabilities:** Some feature combinations may never appear in training data → zero probabilities
 - This can cause the entire posterior probability to become zero
 - Requires smoothing techniques
3. **Continuous features:** Features may be continuous, requiring parametric assumptions (e.g., Gaussian)
 - We need to assume a probability distribution family
 - Then estimate the parameters of that distribution

Key Challenge:

The complexity of estimating $p(\mathbf{x}|y_j)$ grows exponentially with the number of features, making it impractical to estimate directly from data without making simplifying assumptions.

22.9 Solution: Parametric Models

One solution (out of many) is to approximate the conditional densities by **fitting a parametric model**.

Parametric Approach

We approximate the likelihood by fitting a **parametric model** (e.g., Gaussian) to the training data and estimate its parameters using, for example, **Maximum Likelihood Estimation (MLE)**.

- It is a standard way to estimate parameters associated to the likelihood from training data
- For a Gaussian model, the parameters are:

$$\boldsymbol{\theta}_j = (\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

where $\boldsymbol{\mu}_j$ is the mean vector and $\boldsymbol{\Sigma}_j$ is the covariance matrix for class y_j

- The likelihood is then approximated as:

$$P(\mathbf{x}|y_j) \approx \mathcal{N}(\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

22.9.1 Maximum Likelihood Estimation (MLE)

Definition 22.1 (Maximum Likelihood Estimation). MLE chooses the parameter values that make the observed data **most probable**.

For a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$, the likelihood function is:

$$L(\boldsymbol{\theta}_j) = \prod_{i:y_i=y_j} p(\mathbf{x}_i|y_j, \boldsymbol{\theta}_j)$$

MLE finds:

$$\hat{\boldsymbol{\theta}}_j = \arg \max_{\boldsymbol{\theta}_j} L(\boldsymbol{\theta}_j)$$

Observation. In practice, we often maximize the **log-likelihood** instead:

$$\log L(\boldsymbol{\theta}_j) = \sum_{i:y_i=y_j} \log p(\mathbf{x}_i|y_j, \boldsymbol{\theta}_j)$$

This is mathematically equivalent but numerically more stable and easier to optimize.

22.9.2 Gaussian Naïve Bayes

When features are continuous and we assume they follow a Gaussian distribution:

- For each class y_j and feature x_i , we estimate:

$$\mu_{ij} = \frac{1}{N_j} \sum_{k:y_k=y_j} x_{ki} \quad (27)$$

$$\sigma_{ij}^2 = \frac{1}{N_j} \sum_{k:y_k=y_j} (x_{ki} - \mu_{ij})^2 \quad (28)$$

where N_j is the number of training samples in class y_j

- The likelihood for feature x_i given class y_j is:

$$p(x_i|y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} \exp\left(-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}\right)$$

- Under the independence assumption:

$$p(\mathbf{x}|y_j) = \prod_{i=1}^d p(x_i|y_j)$$

Note. The Gaussian assumption is reasonable for many real-world problems, but it's important to verify whether your data actually follows this distribution. For non-Gaussian data, other parametric models (e.g., multinomial for discrete features) or non-parametric methods may be more appropriate.

22.10 Summary: From Theory to Practice

Bayesian Classification Pipeline:

1. **Theoretical Foundation:** Bayes' theorem provides the optimal decision rule
2. **Practical Challenge:** We don't have access to true probability distributions
3. **Naïve Assumption:** Assume feature independence to simplify computation
4. **Parametric Modeling:** Fit parametric models (e.g., Gaussian) to estimate likelihoods
5. **Parameter Estimation:** Use MLE to learn model parameters from training data
6. **Smoothing:** Apply techniques like Laplace smoothing to handle zero probabilities

23 MLE in Naïve Bayes: Practical Implementation

23.1 MLE for Discrete (Categorical) Features

Suppose you're training a Naïve Bayes classifier with **categorical features**. For each class y , you estimate:

- the **prior** $P(y)$ and the **likelihoods** $P(x_i|y)$

23.1.1 (a) Prior via MLE

Prior Estimation

$$P(y) = \frac{\text{number of samples with class } y}{\text{total number of samples}}$$

Observation. This is simply the relative frequency of class y in the training data. If we have 100 samples and 60 belong to class y_1 , then $P(y_1) = 0.6$.

23.1.2 (b) Likelihood via MLE

For categorical features (e.g., words in text classification):

Likelihood Estimation for Categorical Features

If x_i is a categorical feature (e.g., word):

$$P(x_i = v_k | y) = \frac{\text{count of } (x_i = v_k, y)}{\text{count of } y}$$

where v_k is a specific value (category) that feature x_i can take.

Observation. This is the **maximum likelihood estimate** of the conditional probability $P(x_i = v_k | y)$. You're simply estimating probabilities by **relative frequencies**: the parameter values that **maximise** the likelihood of your training data.

Example 23.1 (Word Frequency in Spam Detection). If we're classifying emails as spam or ham, and the word "free" appears in 40 out of 60 spam emails, then:

$$P(\text{free} = \text{Yes} | \text{Spam}) = \frac{40}{60} = 0.667$$

23.2 MLE for Continuous Features: Gaussian Model

If features are continuous and you assume a Gaussian likelihood:

$$P(x_i | y) = \mathcal{N}(x_i; \mu_{iy}, \sigma_{iy}^2) \Rightarrow P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_{iy}^2}} \exp\left(-\frac{(x_i - \mu_{iy})^2}{2\sigma_{iy}^2}\right) \quad (29)$$

Then MLE gives:

Gaussian MLE Parameters

$$\hat{\mu}_{iy} = \frac{1}{N_y} \sum_{j:y_j=y} x_{ij} \quad (30)$$

$$\hat{\sigma}_{iy}^2 = \frac{1}{N_y} \sum_{j:y_j=y} (x_{ij} - \hat{\mu}_{iy})^2 \quad (31)$$

where N_y is the number of samples in class y .

Key Insight

These are just the **sample mean and variance** of that feature within each class.

23.3 Complete Example: Fruit Classification

Let's work through a complete example of Gaussian Naïve Bayes with continuous features.

Example 23.2 (Classifying Fruits by Weight). We have a dataset of fruits with their weights (in grams):

We want to classify a new fruit with weight $x = 158$ g.

23.3.1 Step 1: Estimate Gaussian Parameters (MLE)

For Apples:

Fruit	Weight (g)
Apple	150
Apple	160
Apple	155
Orange	180
Orange	190
Orange	185

Table 7: Training data for fruit classification

$$\mu_{\text{Apple}} = \frac{150 + 160 + 155}{3} = \frac{465}{3} = 155 \quad (32)$$

$$\sigma_{\text{Apple}}^2 = \frac{(150 - 155)^2 + (160 - 155)^2 + (155 - 155)^2}{3} \quad (33)$$

$$= \frac{25 + 25 + 0}{3} = \frac{50}{3} \approx 16.67 \quad (34)$$

For Oranges:

$$\mu_{\text{Orange}} = \frac{180 + 190 + 185}{3} = \frac{555}{3} = 185 \quad (35)$$

$$\sigma_{\text{Orange}}^2 = \frac{(180 - 185)^2 + (190 - 185)^2 + (185 - 185)^2}{3} \quad (36)$$

$$= \frac{25 + 25 + 0}{3} = \frac{50}{3} \approx 16.67 \quad (37)$$

Observation. Notice that both classes have the same variance in this example. This is coincidental and won't always be the case in real data.

23.3.2 Step 2: Compute the Likelihood for a New Sample

New fruit weight: $x = 158$ g.

Using the Gaussian likelihood formula:

$$P(x|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x - \mu_y)^2}{2\sigma_y^2}\right)$$

For Apple:

$$P(158|\text{Apple}) = \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 155)^2}{2 \cdot 16.67}\right) \quad (38)$$

$$= \frac{1}{\sqrt{104.67}} \exp\left(-\frac{9}{33.34}\right) \quad (39)$$

$$\approx 0.097 \cdot \exp(-0.27) \quad (40)$$

$$\approx 0.097 \cdot 0.763 \quad (41)$$

$$\approx 0.072 \quad (42)$$

For Orange:

$$P(158|\text{Orange}) = \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 185)^2}{2 \cdot 16.67}\right) \quad (43)$$

$$= \frac{1}{\sqrt{104.67}} \exp\left(-\frac{729}{33.34}\right) \quad (44)$$

$$\approx 0.097 \cdot \exp(-21.87) \quad (45)$$

$$\approx 0.097 \cdot 3.0 \times 10^{-10} \quad (46)$$

$$\approx 1.6 \times 10^{-11} \quad (47)$$

Observation. The likelihood for Apple is much higher than for Orange. This makes intuitive sense: 158g is much closer to the Apple mean (155g) than to the Orange mean (185g).

23.3.3 Step 3: Multiply by Class Prior

Assume equal priors (we have 3 apples and 3 oranges):

$$P(\text{Apple}) = P(\text{Orange}) = 0.5$$

Computing the posterior (proportional to likelihood $\times$ prior):

$$P(\text{Apple}|158) \propto P(158|\text{Apple}) \cdot P(\text{Apple}) \quad (48)$$

$$= 0.072 \cdot 0.5 \quad (49)$$

$$\approx 0.036 \quad (50)$$

$$P(\text{Orange}|158) \propto P(158|\text{Orange}) \cdot P(\text{Orange}) \quad (51)$$

$$= 1.6 \times 10^{-11} \cdot 0.5 \quad (52)$$

$$\approx 8 \times 10^{-12} \quad (53)$$

Prediction: Apple, because posterior probability is much higher.

$$P(\text{Apple}|158) \approx 0.036 \gg P(\text{Orange}|158) \approx 8 \times 10^{-12}$$

23.4 Key Takeaways from the Example

1. **Parameter Estimation:** We used MLE to estimate μ and σ^2 from the training data
2. **Likelihood Computation:** We computed $P(x|y)$ using the Gaussian formula with the estimated parameters
3. **Prior Incorporation:** We multiplied by the class priors (which were equal in this case)
4. **Decision:** We chose the class with the higher posterior probability
5. **Distance Matters:** The prediction makes intuitive sense—158g is much closer to the Apple distribution than the Orange distribution

Note. In practice, when computing posteriors for multiple classes, we often work with log-probabilities to avoid numerical underflow:

$$\log P(y|\mathbf{x}) = \log P(y) + \sum_{i=1}^d \log P(x_i|y) - \log P(\mathbf{x})$$

Since we're comparing classes, we can ignore the $\log P(\mathbf{x})$ term (it's the same for all classes).

24 Nonparametric Classifiers

24.1 Introduction to Nonparametric Methods

So far, we've discussed parametric approaches (like Gaussian Naïve Bayes) that assume a specific form for the probability distribution. Now we turn to **nonparametric methods**.

Nonparametric Methods

Given a set of examples, model the density function of the data **without making any assumptions about the form of the distribution** (e.g., Gaussian distribution).

- Estimate the probability density function **directly from the samples**
- No predetermined functional form (e.g., no assumption of Gaussian, exponential, etc.)

Observation. The term "nonparametric" is somewhat misleading—these methods often have parameters (like bin width in histograms or bandwidth in kernel density estimation), but they don't assume a specific parametric form for the underlying distribution.

Key Idea

The simplest form of nonparametric density estimation is the **histogram**.

24.2 Histogram-Based Density Estimation

The histogram is the most intuitive nonparametric density estimator.

24.2.1 How Histograms Work

Definition 24.1 (Histogram Density Estimator). Dividing the sample space into a number of bins and approximate the density at the **center of each bin** by the **fraction** of points in the training data that fall into the corresponding bin:

$$p_H(x) = \frac{1}{N} \cdot \frac{\# \text{ of } x^{(k)} \text{ in same bin as } x}{\text{width of bin}}$$

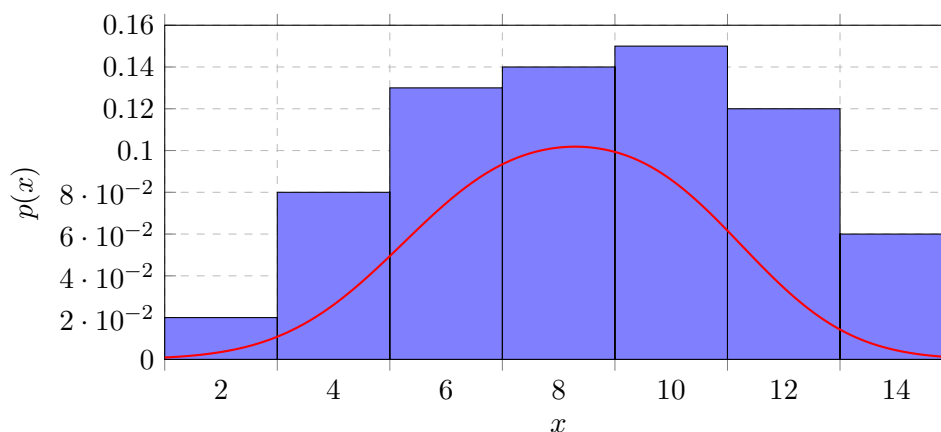
where N is the total number of training samples.

Observation. The histogram essentially counts how many data points fall into each bin and normalizes by the bin width and total number of samples to create a probability density estimate.

24.2.2 Histogram Parameters

The histogram requires two "parameters" to be defined:

1. **Bin width:** Determines the resolution of the density estimate
2. **Starting position** of the first bin: Determines where the bins are anchored



24.3 Drawbacks of Histograms

Despite their simplicity, histograms have several significant limitations:

1. **Dependence on starting position:** The density estimate depends on the starting position of the bins
 - Different starting positions can produce different density estimates
 - For multivariate data, the density estimate is also affected by the **orientation of the bins**
2. **Discontinuities:** The discontinuities of the estimate are not due to the underlying density; they are only an artifact of the chosen bin locations
 - These discontinuities make it very difficult to grasp the **structure** of the data
 - The true underlying distribution is typically smooth, but histograms create artificial jumps
3. **Curse of dimensionality:** A much more serious problem is the **curse of dimensionality**, since the number of bins grows exponentially with the number of dimensions
 - In high dimensions we would require a very large number of examples or else most of the bins would be empty
 - With d dimensions and k bins per dimension, we need k^d total bins

The Curse of Dimensionality

As the number of features or dimensions increases, the amount of data required to effectively cover the feature space increases **exponentially**.

24.4 The Curse of Dimensionality

Definition 24.2 (Curse of Dimensionality). The curse of dimensionality refers to the challenges and issues that arise when working with high-dimensional data in machine learning.

As the number of features or dimensions in a dataset increases, the amount of data required to effectively cover the feature space also increases exponentially.

24.4.1 Why It Matters

- **Exponential growth:** With d dimensions, if each dimension is divided into k bins, we need k^d bins total

- **Data sparsity:** In high dimensions, data points become increasingly sparse
 - Most bins will be empty unless we have an enormous amount of data
 - The distance between points increases, making it harder to find meaningful patterns
- **Sample size requirements:** To maintain the same density of samples, we need exponentially more data as dimensions increase

Example 24.1 (Exponential Growth of Bins). • 1D: 10 bins per dimension $\rightarrow$ 10 bins total

- 2D: 10 bins per dimension $\rightarrow$ 100 bins total
- 3D: 10 bins per dimension $\rightarrow$ 1,000 bins total
- 10D: 10 bins per dimension $\rightarrow$ 10,000,000,000 bins total!

24.4.2 Mitigating the Curse of Dimensionality

To mitigate the curse of dimensionality, various techniques are employed:

- **Feature selection:** Choose only the most relevant features
- **Dimensionality reduction:** Techniques like Principal Component Analysis (PCA) reduce the number of dimensions while preserving important characteristics
- **Regularization methods:** Add constraints to prevent overfitting in high dimensions
- **Domain knowledge:** Use expert knowledge to identify which features are truly important

Practical Implication

In practice, nonparametric methods like histograms work well for 1D or 2D data, but become impractical for high-dimensional data. This motivates the development of more sophisticated techniques.

24.5 Probability Density: Mathematical Foundation

Before we proceed with more advanced nonparametric techniques, let's return to the basic definition of probability to get a solid idea of what we are trying to accomplish.

Probability in a Region

The probability that a vector $\mathbf{x}$, drawn from a distribution $P(\mathbf{x})$, will fall in a region $\mathcal{R}$ of the sample space is:

$$P = \int_{\mathbf{x}' \in \mathcal{R}} P(\mathbf{x}') d\mathbf{x}'$$

where:

- $\mathbf{x}$ is a particular vector we are asking about ("drawn from the distribution")
- $\mathbf{x}'$ is a dummy variable used to sum/integrate over all vectors in $\mathcal{R}$
- $\mathcal{R}$ is a region in the feature space

Observation. This integral formulation is the foundation for all density estimation methods. The key insight is that if we can estimate the probability in small regions around each point, we can reconstruct the entire probability density function.

24.5.1 Key Concepts

- **Probability density function** $P(\mathbf{x})$: Describes how probability is distributed over the feature space
- **Integration over a region**: The probability of falling in region $\mathcal{R}$ is the integral of the density over that region
- **Dummy variable** $\mathbf{x}'$: Used for integration; represents all possible points in the region
- **Normalization**: The integral over the entire space must equal 1:

$$\int_{\text{all space}} P(\mathbf{x}') d\mathbf{x}' = 1$$

Note. For discrete distributions, the integral becomes a sum:

$$P = \sum_{\mathbf{x}' \in \mathcal{R}} P(\mathbf{x}')$$

This is the discrete analog of the continuous case and is what we use when working with categorical data.

24.6 Binomial Distribution Approach

Now let's develop a more rigorous approach to nonparametric density estimation based on probability theory.

24.6.1 Setup

Suppose now that N vectors $\{\mathbf{x}(1), \mathbf{x}(2), \dots, \mathbf{x}(N)\}$ are drawn **independently and identically distributed (i.i.d.)** from the distribution. The probability that k of these N vectors fall in $\mathcal{R}$ is given by the binomial distribution:

Binomial Distribution

$$P(k) = \binom{N}{k} P^k (1 - P)^{N-k}$$

where:

$$\binom{N}{k} = \frac{N!}{k!(N-k)!}$$

and the expected value for k is:

$$\mathbb{E}[k] = NP$$

Observation. Recall: The expectation is the average value we would get if we repeated drawing samples many times.

The binomial distribution tells us how likely it is to observe exactly k points in region $\mathcal{R}$ when we draw N samples, given that each point has probability P of falling in $\mathcal{R}$.

24.6.2 Mean and Variance of the Ratio

The mean and variance of the ratio k/N are:

$$\mathbb{E} \left[\frac{k}{N} \right] = P \quad (54)$$

$$\text{Var} \left[\frac{k}{N} \right] = \mathbb{E} \left[\left(\frac{k}{N} - P \right)^2 \right] = \frac{P(1-P)}{N} \quad (55)$$

Observation. Therefore, as $N \rightarrow \infty$, the distribution becomes **sharper**, the variance gets **smaller**, thus we can obtain a **better estimate** of the probability P from the mean fraction of the points that fall within $\mathcal{R}$:

$$P \cong \frac{k}{N}$$

This is the law of large numbers in action: with more samples, our estimate becomes more accurate.

24.7 Density Estimation Formula

24.7.1 Small Region Assumption

On the other hand, if we assume that $\mathcal{R}$ is **so small** that $P(\mathbf{x})$ **does not vary** appreciably within it, then:

$$\int_{\mathcal{R}} P(\mathbf{x}') d\mathbf{x}' \cong P(\mathbf{x})V$$

where V is the **volume enclosed** by region $\mathcal{R}$.

Observation. This approximation assumes that the probability density is approximately constant within the small region $\mathcal{R}$, so we can pull $P(\mathbf{x})$ out of the integral, leaving just the volume V .

24.7.2 Combining the Results

Merging with the previous result, we obtain:

$$P = \int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}' \cong p(\mathbf{x})V \quad (56)$$

$$P \cong \frac{k}{N} \quad (57)$$

Therefore:

$$p(\mathbf{x}) \cong \frac{k}{NV}$$

Key Result

This estimate becomes more accurate as we **increase the number of sample points** N and **shrink the volume** V .

24.8 The Trade-off Problem

In practice, the value of N (the total number of examples) is fixed.

24.8.1 The Dilemma

- In order to improve the accuracy of the estimate $P(\mathbf{x})$ we could let V to approach zero, but then the region $\mathcal{R}$ would become so small that it would enclose no examples
- This means that in practice, we will have to find a **compromise value of the volume V** :
 - **Large enough** to include enough examples within $\mathcal{R}$
 - **Small enough** to support the assumption that $P(\mathbf{x})$ is constant within $\mathcal{R}$

The Fundamental Trade-off

- Too large $V \rightarrow$ Poor approximation (density varies within region)
- Too small $V \rightarrow$ Too few samples (high variance in estimate)

24.9 General Nonparametric Density Estimation Formula

General Expression for Nonparametric Density Estimation

$$P(\mathbf{x}) \cong \frac{k}{NV}$$

where:

- V is the volume surrounding $\mathbf{x}$
- N is the total number of examples
- k is the number of examples inside V

24.10 Two Approaches to Nonparametric Density Estimation

When applying this result to practical density estimation problems, two basic approaches can be adopted:

1. Fix V and determine k from the data

- This leads to **kernel density estimation (KDE)**, e.g., Parzen window
- We define a fixed-size region around each point and count how many samples fall within it

2. Fix k and determine V from the data

- This gives rise to the **k -nearest-neighbor (k -NN) approach**
- We find the k nearest neighbors and measure the volume needed to enclose them

Convergence Property

Both k -NN and KDE converge to the true probability density as $N \rightarrow \infty$, provided that:

- V shrinks with N
- k grows with N appropriately

24.10.1 Comparison of Approaches

Observation. **KDE** produces smoother estimates but may miss structure in sparse regions, while **k -NN** adapts to local density but can produce less smooth estimates. The choice depends on the specific application and data characteristics.

Kernel Density Estimation (KDE)	k -Nearest Neighbor (k -NN)
Fix volume V Count samples k in fixed region	Fix number of neighbors k Adjust volume V to include k samples
Smooth density estimate Can be zero in sparse regions Examples: Parzen window, Gaussian kernel	Adaptive to local density Always uses k neighbors Examples: k -NN classifier, k -NN regressor

Table 8: Comparison of KDE and k -NN approaches

24.10.2 Choosing Parameters

- **For KDE:** Choose bandwidth (related to V)
 - Too small: noisy, overfitting
 - Too large: oversmoothing, missing details
- **For k -NN:** Choose k
 - Too small: sensitive to noise, high variance
 - Too large: oversmoothing, high bias

Note. Both methods require careful tuning of their parameters (bandwidth for KDE, k for k -NN). Cross-validation is commonly used to select these parameters in practice.

25 k-Nearest Neighbors (k-NN)

25.1 Visual Comparison: k-NN vs. Parzen Windows

Key Difference:

- **Parzen Window:** Fixed volume V (radius R), variable k
- **k-NN:** Fixed k (number of neighbors), variable volume V (radius R)

Observation. In the visual comparison:

- **Parzen Window:** Uses a fixed-size circle (radius R) around the query point (yellow cross). The number of points inside varies depending on local density.
- **k-NN (e.g., 3-NN):** The circle grows or shrinks to always include exactly $k = 3$ nearest neighbors. In dense regions, the circle is small; in sparse regions, it's large.

25.2 Parzen Window: Practical Considerations

When to Use Parzen Windows

Parzen window is great for **low-dimensional data** (2–3 features, maybe up to 5).

- In higher dimensions, the method becomes **computationally expensive**, and the density estimates are **unreliable** because of the curse of dimensionality
- For this reason, in A.A. 2025–2026, we focus on other methods better suited for high-dimensional data

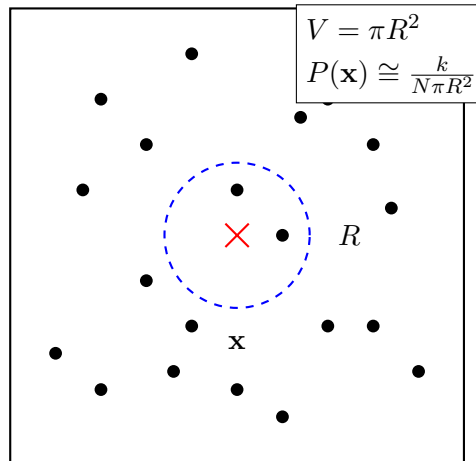
Note. While Parzen windows provide smooth density estimates, they suffer from the curse of dimensionality. As dimensions increase, the fixed window size becomes either too large (over-smoothing) or too small (too few samples), making the method impractical for modern high-dimensional datasets.

25.3 k-Nearest Neighbors Algorithm

The k-NN algorithm is one of the simplest and most intuitive machine learning algorithms.

25.3.1 Basic Principle

The volume is grown surrounding the estimation point $\mathbf{x}$, until it encloses a total of k data points.



k-NN Density Estimation

$$P(\mathbf{x}) \cong \frac{k}{NV}$$

where:

- V is the volume surrounding $\mathbf{x}$
- N is the total number of examples
- k is the number of examples inside V (fixed)

25.4 k-NN for Classification

25.4.1 1-Nearest Neighbor (1-NN)

Given an unknown point $\mathbf{x}^*$, we consider $s_i \in \{s_1, \dots, s_N\} = S$ the nearest point (NN) of the point $\mathbf{x}^*$ if:

$$d(\mathbf{x}^*, s_i) = \min_l d(\mathbf{x}^*, s_l), \quad l = 1 \dots N$$

where $d(\cdot)$ is a **distance measure** defined on the feature space.

Observation. If you interpret each point of set S as a prototype of a class, you can then see the equation above as a rule of classification **1-NN** associating the sample $\mathbf{x}$ to the class j to which the point s_i belongs.

Example 25.1 (Common Distance Measures). • **Euclidean distance:** $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$

- **Manhattan distance:** $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i|$
- **Minkowski distance:** $d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$

25.4.2 Generalizing to k-NN

Rule 1-NN can be generalised to define a **k-NN rule**, which consists of determining the k points belonging to the set S closest to the observation $\mathbf{x}$.

k-NN Classification Rule

The classification rule k -NN associates observation $\mathbf{x}$ with class i having the **largest number of elements** among the nearest k .
We call $U(\mathbf{x})$ the set of k points closer to $\mathbf{x}$.

25.4.3 Majority Voting

For example, with odd k and two classes, y_1 and y_2 , the decision rule can choose the class with the most samples in $U(\mathbf{x})$. This is called **majority voting**.

Majority Voting

Choose the class that appears most frequently among the k nearest neighbors.

Example 25.2 (3-NN Classification). Suppose we have $k = 3$ and two classes (red and blue):

- Query point $\mathbf{x}^*$ has 3 nearest neighbors: 2 red, 1 blue
- Majority voting: $\mathbf{x}^*$ is classified as **red**

25.5 Choosing k

The choice of k is critical for k-NN performance:

Value of k	Advantages	Disadvantages
Small k (e.g., 1-3) - More flexible decision boundaries - High variance - Overfitting	- Captures local structure - Sensitive to noise	
Large k - Smoother decision boundaries - Lower variance - High bias - Underfitting	- More robust to noise - May miss local patterns	
Odd k	- Avoids ties in binary classification	-

Table 9: Trade-offs in choosing k

Rule of Thumb

- Start with $k = \sqrt{N}$ where N is the number of training samples
- Use cross-validation to find the optimal k
- For binary classification, use odd k to avoid ties

25.6 Advantages and Disadvantages of k-NN

Advantages	Disadvantages
• Simple and intuitive • No training phase (lazy learning) • Naturally handles multi-class problems • Non-parametric (no assumptions about data distribution) • Can capture complex decision boundaries • Easy to update with new data	• Computationally expensive at prediction time • Requires storing all training data • Sensitive to irrelevant features • Sensitive to the scale of features • Curse of dimensionality • Choosing optimal k can be difficult

Table 10: Advantages and disadvantages of k-NN

25.7 Practical Considerations

25.7.1 Feature Scaling

Observation. k-NN is **highly sensitive to feature scaling** because it relies on distance calculations. Features with larger ranges will dominate the distance metric.

Solution: Always normalize or standardize features before applying k-NN:

- **Min-Max Normalization:** Scale to $[0, 1]$: $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$
- **Standardization:** Scale to mean 0, std 1: $x' = \frac{x - \mu}{\sigma}$

25.7.2 Computational Complexity

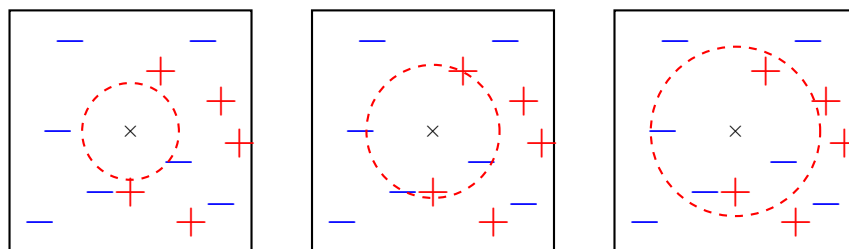
- **Training time:** $O(1)$ (no training, just store data)
- **Prediction time:** $O(Nd)$ where N is training set size, d is dimensionality
- For large datasets, use approximate nearest neighbor algorithms (e.g., KD-trees, Ball trees, LSH)

Note. k-NN is called a **lazy learner** or **instance-based learner** because it doesn't learn a model during training. Instead, it memorizes the training data and performs all computation at prediction time. This makes training fast but prediction slow, especially for large datasets.

25.8 Distance Metrics in k-NN

25.8.1 Euclidean Distance

To calculate the distance, we usually use **Euclidean distance**.



(m) 1-nearest neighbor (m) 2-nearest neighbor (m) 3-nearest neighbor

Euclidean Distance Formula

Given two points $\mathbf{p} = (p_1, p_2, \dots, p_n)$ and $\mathbf{q} = (q_1, q_2, \dots, q_n)$:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Observation. The visual shows how the radius grows as k increases:

- **1-NN:** Small circle, includes only the closest neighbor
- **2-NN:** Medium circle, includes 2 closest neighbors
- **3-NN:** Larger circle, includes 3 closest neighbors

The decision boundary depends on which class dominates within the circle.

25.9 Weighted k-NN

25.9.1 The Problem with Equal Weights

All the closest neighbors have the same weights, meaning that they are equally important to make a decision.

Issue: Therefore, the algorithm is very sensitive to the number of k .

25.9.2 Solution: Distance-Based Weighting

This can be reduced by **weighing the contribution of neighbors based on distance** $w = 1/d(\mathbf{x}^*, s_i)$, such that the **closest** neighbor would have more **importance**.

Weighted k-NN

Instead of simple majority voting, each neighbor contributes with weight:

$$w_i = \frac{1}{d(\mathbf{x}^*, s_i)}$$

The class with the highest weighted sum is chosen:

$$\text{class}(\mathbf{x}^*) = \arg \max_c \sum_{i \in U(\mathbf{x}^*)} w_i \cdot \mathbb{I}(y_i = c)$$

where $\mathbb{I}(y_i = c)$ is 1 if neighbor i belongs to class c , and 0 otherwise.

Example 25.3 (Weighted vs. Unweighted k-NN). Consider $k = 3$ with neighbors at distances 1, 2, and 5:

- **Unweighted:** Each neighbor gets weight 1
- **Weighted:** Neighbors get weights $w_1 = 1/1 = 1.0$, $w_2 = 1/2 = 0.5$, $w_3 = 1/5 = 0.2$

The closest neighbor has much more influence in the weighted version.

25.10 Importance of Feature Scaling (Revisited)

25.10.1 Why k is Sensitive

The choice of k is important because:

- If k is too **small**, the approach is **sensitive to noise**
- If k is too **large**, the neighborhood can include examples belonging to **other classes**

Critical Requirement

To operate correctly the attributes must have the **same value scale** and must therefore be **normalized** in the preprocessing phase.

Observation. Without normalization, features with larger ranges will dominate the distance calculation, making features with smaller ranges essentially irrelevant to the classification decision.

25.11 Mahalanobis Distance

25.11.1 When Euclidean Distance is Not Enough

Normalizing the scale may not be sufficient when there are different data distributions.

In such cases instead of using Euclidean distance, one can also try using **Mahalanobis distance**.

25.11.2 Definition

The Mahalanobis distance is a measure of the distance between **a point and a distribution**. It's a generalized form of the Euclidean distance that accounts for **correlations between variables** and **different variances** along different axes.

Mahalanobis Distance Formula

$$D = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

where:

- $\mathbf{x}$ is the n -dimensional vector representing the point's coordinates
- $\boldsymbol{\mu}$ is the n -dimensional vector representing the mean of the distribution
- $\boldsymbol{\Sigma}$ is the $n \times n$ covariance matrix of the distribution
- $\boldsymbol{\Sigma}^{-1}$ is the inverse of the covariance matrix

Observation. When the covariance matrix is the identity matrix (i.e., features are uncorrelated and have unit variance), the Mahalanobis distance reduces to the Euclidean distance.

25.11.3 When to Use Mahalanobis Distance

1. **Correlated Features:** Changes in one feature are associated with changes in another.
2. **Unequal Variances:** MD normalizes the distance based on the variances of the features, preventing features with larger variances from dominating the distance calculation.
3. **Outlier Detection:** Outliers often have a large MD from the rest of the data points, especially if they deviate from the normal covariance pattern.
4. **In high dimensional spaces:** ED can be problematic (*curse of dimensionality*). Incorporating information about the covariance structure, can mitigate this issue to some extent.

Example 25.4 (Mahalanobis Distance in Practice). Consider a 2D dataset where:

- Feature 1 has variance 1
- Feature 2 has variance 100

Euclidean Distance	Mahalanobis Distance
Assumes features are independent	Accounts for correlations between features
Treats all directions equally	Accounts for different variances along different axes
Sensitive to feature scaling	Automatically accounts for different scales
Simple and fast to compute	More computationally expensive (requires covariance matrix)
Works well when features are uncorrelated and have similar variance	Better when features are correlated or have different variances
Distance from point to point	Distance from point to distribution

Table 11: Euclidean vs. Mahalanobis Distance

- Features are positively correlated

Euclidean distance would be dominated by Feature 2 due to its large variance.

Mahalanobis distance would:

- Normalize for the different variances
- Account for the correlation structure
- Provide a more meaningful distance measure

Note. Computing Mahalanobis distance requires estimating the covariance matrix Σ , which can be challenging when:

- The number of features is large relative to the number of samples
- The covariance matrix is singular (not invertible)
- There is insufficient data to reliably estimate correlations

In such cases, regularization techniques or dimensionality reduction may be needed before applying Mahalanobis distance.

25.11.4 Visual Example: Euclidean vs. Mahalanobis Distance

Example 25.5 (Comparing Distance Metrics). Consider a dataset with a centroid at $(\bar{x}, \bar{y}) = (3.1, 3.0)$ and two data points:

- **Data point 1:** (5, 5) (yellow/orange point)
- **Data point 2:** (5, 1) (blue point)

Using Euclidean Distance:

Distance from centroid to Data point 1:

$$d_1 = \sqrt{(5 - 3.1)^2 + (5 - 3.0)^2} = \sqrt{1.9^2 + 2.0^2} = \sqrt{3.61 + 4.00} = 2.76$$

Distance from centroid to Data point 2:

$$d_2 = \sqrt{(5 - 3.1)^2 + (1 - 3.0)^2} = \sqrt{1.9^2 + (-2.0)^2} = \sqrt{3.61 + 4.00} = 2.76$$

Based on Euclidean distance, both points have the same distance to the centroid!

Observation. However, looking at the scatter plot, we can see that the green points (the cluster) are elongated along a diagonal direction. Data point 1 lies along this diagonal trend, while Data point 2 deviates significantly from it.

Euclidean distance treats all directions equally and doesn't account for the correlation structure of the data.

Example 25.6 (Using Mahalanobis Distance). When we apply Mahalanobis distance to the same scenario:

- $MD_1 = 2.26$ (Data point 1 to centroid)
- $MD_2 = 6.45$ (Data point 2 to centroid)

Why is MD_1 so different from MD_2 ?

Because MD takes into account the **correlation** in the data.

Interpretation:

- Data point 1 follows the natural diagonal trend of the cluster → smaller Mahalanobis distance
- Data point 2 deviates from the correlation pattern → larger Mahalanobis distance

Note. This example illustrates why Mahalanobis distance is superior when features are correlated:

- **Euclidean distance:** Treats both points as equally distant (2.76)
- **Mahalanobis distance:** Correctly identifies that point 2 is much farther from the distribution (6.45 vs 2.26)

Mahalanobis distance accounts for the shape and orientation of the data distribution, not just raw coordinate distances.

25.12 How to Select k in k-NN

Choosing the right value of k is crucial for k-NN performance. Here are practical strategies:

25.12.1 Common Heuristics

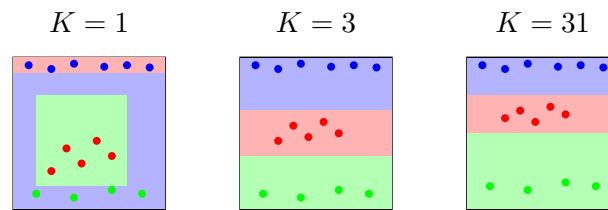
- **Common values:** Often 3, 5, 7
- **Choose an odd number** to avoid ties (*undetermined cases*)
- **Use your training and validation data** to decide k :
 - Change k from $k = 1$ until the validation performance decreases
 - Pay attention not to **underfit** and **overfit** the data
 - Finalize the decision of k and use the same k to classify the test data
- **A typical choice** is $k \approx \sqrt{N}$ where N is the number of training samples

Validation-Based Selection

The most reliable method is to use cross-validation:

1. Try different values of k (e.g., 1, 3, 5, 7, 9, 11, ...)
2. Evaluate performance on validation set for each k
3. Select the k that gives the best validation performance
4. Use this optimal k on the test set

25.12.2 Visual Understanding: k and Decision Boundaries



What is the role of k ? How does it relate to overfitting and underfitting?

- $k = 1$ (left): Very complex, jagged decision boundaries
 - High variance, low bias
 - **Overfitting:** Captures noise in the training data
 - Sensitive to outliers
- $k = 3$ (middle): Moderate complexity, smoother boundaries
 - Balanced bias-variance trade-off
 - Good generalization
- $k = 31$ (right): Very smooth, simple decision boundaries
 - Low variance, high bias
 - **Underfitting:** May miss important patterns
 - Too much smoothing

Observation. As k increases:

- Decision boundaries become smoother and simpler
- The model becomes less sensitive to individual data points
- Risk shifts from overfitting (small k) to underfitting (large k)

25.12.3 Handling Ties (Undetermined Cases)

If there are undetermined cases (ties), there are some alternatives:

1. **Arbitrary choice:** Randomly select one of the tied classes
2. **Nearest average sample:** Assign $\mathbf{x}$ to the class (among those "tie") that has the nearest average sample (calculated between the k_i samples)
3. **Least distance:** Assign $\mathbf{x}$ to the class that has the k_i samples with least distance from it

Example 25.7 (Handling a Tie). Suppose $k = 4$ and we have:

- 2 neighbors from class A at distances: 1.5, 2.0
- 2 neighbors from class B at distances: 1.2, 3.5

Solutions:

- **Arbitrary:** Randomly choose A or B
- **Nearest average:**
 - Class A average: $(1.5 + 2.0)/2 = 1.75$

- Class B average: $(1.2 + 3.5)/2 = 2.35$
- Choose class A (smaller average distance)
- **Least distance:** Choose class B (has the closest neighbor at 1.2)

Note. The best strategy for handling ties depends on your specific problem:

- **Nearest average:** Good when you want to consider all neighbors equally
- **Least distance:** Good when you believe the closest neighbor is most important
- **Best practice:** Use odd k to avoid ties altogether in binary classification

25.13 k-NN: Pros and Cons

25.13.1 Advantages (Pros)

- **Almost no assumptions about the data**
 - Non-parametric method
 - Doesn't assume any particular distribution
 - Works with any data structure
- **Non-parametric**
 - No model parameters to learn
 - Flexible and adaptive to data
- **There is no training phase, cost is 0**
 - Instant "training" - just store the data
 - Easy to update with new data
 - No convergence issues
- **Allow us to construct non-linear class boundaries and are therefore more flexible**
 - Can capture complex decision boundaries
 - Not limited to linear separability
 - Adapts to local data structure

k-NN gives locally defined decision boundaries between classes

Unlike global methods (e.g., linear classifiers), k-NN creates decision boundaries that adapt to the local structure of the data, allowing for complex, non-linear separations.

Observation. The figure shows how k-NN can create complex, non-linear decision boundaries (shown in red) that follow the natural clustering of the data. The decision boundary is not a simple line or hyperplane, but rather a complex curve that adapts to the local density and distribution of each class (label 1, 2, and 3).

On the right, we see the resulting decision regions where each color represents the area classified as a particular label. The boundaries are highly non-linear and follow the intricate patterns in the data.

25.13.2 Disadvantages (Cons)

- **Need to handle missing data**
 - Cannot compute distances with missing values
 - Requires imputation or special handling
- **The distance metric should be chosen carefully**
 - Different metrics can give very different results
 - Must consider feature correlations and scales
 - May need domain knowledge
- **Sensitive to class outliers** (e.g., mislabeled training instances)
 - Outliers can create incorrect local decision boundaries
 - No mechanism to detect or remove bad training examples
- **Sensitive to many irrelevant attributes** (which influence the distance)
 - All features contribute equally to distance
 - Irrelevant features add noise
 - Feature selection is crucial
- **Computationally expensive:**
 - **Poor scalability:** For each new example, we have to perform as many comparisons as there are labeled data
 - **It is necessary to store all data points:** Therefore, the method is more appropriate when the number of samples N is low
 - Memory requirements grow linearly with dataset size
 - Prediction time is $O(Nd)$ where N is training size, d is dimensionality

Scalability Issue

k-NN does not scale well to large datasets. For a dataset with millions of samples, making a single prediction requires computing distances to all training points, which becomes prohibitively expensive.

Note. The main trade-off with k-NN is between flexibility and computational cost:

- **Pros:** Simple, flexible, no training, handles non-linear boundaries
- **Cons:** Slow predictions, high memory, sensitive to irrelevant features and outliers

k-NN works best for:

- Small to medium-sized datasets
- Problems where decision boundaries are complex and non-linear
- Applications where training time is critical but prediction time is less important
- Situations where new data arrives frequently (easy to update)

25.14 How k-NN Was Born: PDF Estimation

k-NN has a theoretical foundation in probability density function (PDF) estimation. Let's explore this connection.

25.14.1 Going Back to PDF Estimation

Recall from our earlier discussion of density estimation:

- Instead of fixing V as in the case of kernels (Parzen windows), we **fix the value of k** and enlarge the radius of the sphere to include k samples
- Let's see the **a-priori probability**:
 - Given a class y_j , the frequency of occurrence of samples N_j of class j is generally simply measured in relation to the total number of samples N , i.e.,

$$P(y_j) \equiv \hat{p}(y_j) = \frac{N_j}{N}$$

Observation. This is the empirical estimate of the prior probability: the proportion of training samples that belong to class j .

25.14.2 Density Estimation and Nearest-Neighbor Rule

Consider 2 classes, y_i and y_j , containing N_i and N_j samples, $N = N_i + N_j$.

The local density estimation for y_i is calculated as (and similarly for y_j):

$$\hat{p}(\mathbf{x}|y_i) = \frac{1}{V} \frac{k_i}{N_i}$$

such that the ratio of k_i (k_j) points to the total N_i (N_j) belonging to the class y_i (y_j) contained in the volume V .

Deriving k-NN from Bayes Rule

The Bayes rule says $p(\mathbf{x}|y_i)P(y_i) > p(\mathbf{x}|y_j)P(y_j)$, then

$$\begin{aligned} \hat{p}(\mathbf{x}|y_i)\hat{p}(y_i) &> \hat{p}(\mathbf{x}|y_j)\hat{p}(y_j) \\ \Rightarrow \frac{1}{V} \frac{k_i}{N_i} \cdot \frac{N_i}{N} &> \frac{1}{V} \frac{k_j}{N_j} \cdot \frac{N_j}{N} \\ \Rightarrow \frac{1}{VN_i} k_i N_i &> \frac{1}{VN_j} k_j N_j \\ \Rightarrow k_i &> k_j \end{aligned}$$

The k-NN Rule

Classify $\mathbf{x}$ to class y_i if $k_i > k_j$, i.e., if there are more neighbors from class i than from class j in the volume V around $\mathbf{x}$.

Observation. This derivation shows that k-NN is actually implementing a Bayesian decision rule using local density estimates! The key insight is:

- We estimate $p(\mathbf{x}|y_i)$ locally using the k nearest neighbors

- We estimate $P(y_i)$ using the proportion of class i in the training set
- The Bayes rule simplifies to: choose the class with the most neighbors in the local region

Note. This theoretical foundation explains why k-NN works:

- It's approximating the optimal Bayes classifier
- It uses local density information to make decisions
- The quality of the approximation depends on:
 - Choice of k (too small = noisy estimates, too large = oversmoothing)
 - Density of training data (more data = better estimates)
 - Dimensionality (curse of dimensionality affects density estimation)

As the number of training samples $N \rightarrow \infty$ and $k \rightarrow \infty$ (but $k/N \rightarrow 0$), the k-NN classifier approaches the optimal Bayes classifier.

Part IV

Decision Trees and Ensemble Methods

26 Introduction to Decision Trees

Decision trees are fundamental machine learning models that provide both powerful predictive capabilities and high interpretability. They are tree-structured prediction models that make decisions by routing data through a hierarchy of tests.

26.1 Decision Tree Structure

A decision tree is composed of two types of nodes:

- **Non-terminal nodes:** These nodes have 2 or more children and implement a **routing function**. They perform tests on the input features to decide which branch to follow.
- **Leaf nodes (terminal nodes):** These nodes have no children and implement a **prediction function**. They provide the final output of the model.

Key Structural Properties

- There are no cycles in the tree structure
- All nodes have at most 1 parent
- There is exactly one node with no parent, called the **root node**
- The tree forms a directed acyclic graph (DAG)

26.2 How Decision Trees Work

A decision tree takes an input $\mathbf{x} \in \mathcal{X}$ and routes it through its nodes until it reaches a leaf node, where a prediction is made.

The process:

1. The input enters at the root node
2. Each node represents a test on a variable (feature)
3. An edge to a child node represents the outcome of the test
4. The routing continues until a leaf node is reached
5. At the leaf, a prediction is made based on the prediction function

26.3 Decision Tree Inference: Non-terminal Nodes

Each non-terminal node can be formally represented as $\text{Node}(\phi, t_L, t_R)$, where:

- $\phi : \mathcal{X} \rightarrow \{L, R\}$ is a **routing function**
- t_L is the left child subtree
- t_R is the right child subtree

When an input $\mathbf{x}$ reaches the node:

- If $\phi(\mathbf{x}) = L$, the input goes to the left child t_L

- If $\phi(\mathbf{x}) = R$, the input goes to the right child t_R

Note: We assume binary trees here, but the concept generalizes to trees with more than two children.

26.4 Decision Tree Inference: Leaf Nodes

Each leaf node can be represented as $\text{Leaf}(h)$, where:

- $h \in \mathcal{F}_{\text{task}}$ is a **prediction function** (typically a constant)
- Depending on the task:
 - For **classification**: $h \in \mathcal{Y}$ (a class label)
 - For **regression**: $h \in \mathbb{R}$ (a continuous value)

Once $\mathbf{x}$ reaches the leaf, the final prediction is given by $h(\mathbf{x})$.

26.5 Formal Decision Tree Function

The complete decision tree function can be defined recursively:

$$f_t(\mathbf{x}) = \begin{cases} h(\mathbf{x}) & \text{if } t = \text{Leaf}(h) \\ f_{t_{\phi(\mathbf{x})}}(\mathbf{x}) & \text{if } t = \text{Node}(\phi, t_L, t_R) \end{cases}$$

Interpretation:

- If we are at a leaf node, return the prediction $h(\mathbf{x})$
- If we are at a non-terminal node, apply the routing function $\phi(\mathbf{x})$ to determine which child to visit, then recursively apply the tree function to that child

26.6 Decision Tree Inference Example

Consider a 2D classification problem with classes $\xi \in \{+, \times, \circ, \triangle, \square\}$. We can build a decision tree with routing functions:

$$\phi_1(x, y) = \begin{cases} L & \text{if } y \geq 3 \\ R & \text{else} \end{cases}$$

$$\phi_2(x, y) = \begin{cases} L & \text{if } x \leq 3 \\ R & \text{else} \end{cases}$$

$$\phi_3(x, y) = \begin{cases} L & \text{if } x \leq 2 \\ R & \text{else} \end{cases}$$

$$\phi_4(x, y) = \begin{cases} L & \text{if } x \leq 4 \\ R & \text{else} \end{cases}$$

These routing functions partition the 2D space into axis-aligned rectangular regions, with each region corresponding to a leaf node that predicts a specific class.

27 Learning Decision Trees

27.1 The Learning Problem

Given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, we want to find a tree $t^* \in \mathcal{T}$ from the set of all possible decision trees $\mathcal{T}$ that best fits the data.

The Challenge

This optimization problem has many solutions and can be **NP-hard** in general. Thus, we need to impose constraints, such as:

- Finding the most compact tree
- Using greedy algorithms that find locally optimal splits
- Limiting tree depth or complexity

27.2 Tree Learning Framework

To learn a decision tree, we need to specify:

1. $\mathcal{H}_{\text{leaf}}$: A set of leaf prediction functions (e.g., constant functions)
2. Φ : A set of possible splitting functions $\Phi \subseteq \{L, R\}^{\mathcal{X}}$
3. **Tree-growing strategy**: A recursive algorithm that partitions the training set and decides whether to grow leaves or non-terminal nodes

Two Popular Algorithms:

- **ID3** (Iterative Dichotomiser 3): Developed by Ross Quinlan
- **CART** (Classification and Regression Trees): Developed by Breiman et al.

27.3 Tree Growing Strategy

The tree-growing process works recursively:

1. Start with all training examples at the root
2. Find the best splitting function $\phi \in \Phi$ that partitions the data
3. For each resulting subset:
 - If the subset is pure or meets stopping criteria, create a leaf node
 - Otherwise, recursively apply the splitting process

28 ID3 and CART Algorithms

28.1 Example: Playing Tennis Dataset

Consider a classic example with 14 training samples and 4 attributes:

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- 14 training samples
- 4 attributes (features)
- 9 samples with class “Yes”
- 5 samples with class “No”

28.2 Choosing the Best Attribute

The fundamental question in building a decision tree is: **How do we pick the best attribute to split on?**

Consider two possible splits on the root node:

Split 1: Outlook attribute

- Sunny: 2 yes / 3 no
- Overcast: 4 yes / 0 no (pure!)
- Rain: 3 yes / 2 no

Split 2: Wind attribute

- Weak: 6 yes / 2 no
- Strong: 3 yes / 3 no

Which split is better?

The **Outlook** split is better because:

- It produces one pure subset (Overcast: 4 yes / 0 no)
- Pure subsets mean we are completely certain about the classification
- The Weak wind subset (6 yes / 2 no) is better than Strong (3 yes / 3 no), but neither is pure

28.3 Measuring Purity

We need a formal way to measure the “purity” of a split. Key properties:

- **Pure set** (4 yes / 0 no): Completely certain (100% confidence)
- **Impure set** (3 yes / 3 no): Completely uncertain (50% confidence)

- The measure should be **symmetric**: 4 yes / 0 no is as pure as 0 yes / 4 no

There are different ways to measure purity, with **entropy** being the most popular.

29 Entropy and Information Gain

29.1 Entropy

For binary classification problems with positive and negative classes:

- $p(+)$ is the proportion of positives in a subset
- $p(-)$ is the proportion of negatives in a subset

Entropy Formula

The entropy of a subset S is:

$$H(S) = -p(+) \log_2 p(+) - p(-) \log_2 p(-)$$

Important properties:

- Entropy measures uncertainty or impurity
- Higher entropy = more impure = more uncertain
- Lower entropy = more pure = more certain

29.1.1 Entropy Examples

Impure subset (3 yes / 3 no):

$$\begin{aligned} p(+) &= 0.5, \quad p(-) = 0.5 \\ H(S) &= -0.5 \log_2(0.5) - 0.5 \log_2(0.5) \\ &= -0.5 \times (-1) - 0.5 \times (-1) \\ &= 0.5 + 0.5 = 1.0 \end{aligned}$$

Pure subset (4 yes / 0 no):

$$\begin{aligned} p(+) &= 1.0, \quad p(-) = 0.0 \\ H(S) &= -1.0 \log_2(1.0) - 0.0 \log_2(0.0) \\ &= -1.0 \times 0 - 0 \\ &= 0.0 \end{aligned}$$

Interpretation:

- Maximum entropy (1.0) for the impure set (50/50 split)
- Minimum entropy (0.0) for the pure set (100% one class)
- Higher numbers correspond to less pure subsets!

29.2 Information Gain

We want to have many items in pure sets after splitting. We calculate the **expected drop in entropy** after each split.

Information Gain Formula

The information gain of attribute A on set S is:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- $H(S)$ is the entropy of the original set
- $\text{Values}(A)$ are the possible values of attribute A
- S_v is the subset of S where $x_A = v$
- $|S_v|/|S|$ weights each subset by its size

Interpretation:

- Information gain is the **mutual information** between attribute A and class labels
- It measures how much uncertainty is reduced by knowing attribute A
- We want to **maximize information gain**
- Higher gain = better attribute for splitting

29.3 Using Information Gain for Attribute Selection

The ID3 algorithm uses information gain to decide which attribute to pick:

1. For every attribute in your data, compute the information gain
2. Select the attribute with the **highest information gain**
3. This attribute reduces uncertainty the most and leads to the purest possible split

Note: We consider one level split at a time, but the procedure is recursive. After splitting on an attribute, we recursively apply the same process to each resulting subset.

29.4 Problems with Information Gain

Information gain has a significant problem: it favors attributes with many values.

Example: Consider the “Day” attribute (D1, D2, ..., D14) in the tennis dataset:

- Each day is unique, so splitting on “Day” creates 14 subsets
- Each subset contains exactly one example
- All subsets are perfectly pure (optimal split according to information gain!)
- But this is useless! It won’t generalize to new data

If we encounter a new example “D15: Rain, High, Weak”, the tree cannot classify it because it only knows about days D1-D14.

The problem: Information gain can lead to overfitting by favoring attributes that perfectly separate the training data but generalize poorly to test data.

29.5 Gain Ratio: A Solution

One solution is to use **Gain Ratio** instead of information gain:

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}$$

Where:

$$\text{SplitInfo}(S, A) = - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

Effect:

- SplitInfo measures how broadly and uniformly the attribute splits the data
- Dividing by SplitInfo **penalizes attributes with many values**
- This helps prevent overfitting to attributes like “Day”

29.6 Advantages of Information Gain

Despite its limitations, information gain (and decision trees in general) has important advantages:

- **Interpretability:** Decision trees are easy to understand
- **Readable rules:** It’s possible to extract concise rules describing classification decisions
- **No “black box”:** The decision-making process is transparent
- **Important for users:** Explainability is crucial in many applications (medical diagnosis, loan approval, etc.)

Example rule from a decision tree:

IF Outlook = Overcast THEN PlayTennis = Yes

IF Outlook = Sunny AND Humidity = High THEN PlayTennis = No

30 Alternative Splitting Criteria

Besides entropy and information gain, there are other measures for selecting splits.

30.1 Gini Impurity

The **CART algorithm** uses Gini impurity instead of entropy.

Gini Impurity Formula

For a set S with C classes:

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2$$

Where p_i is the proportion of samples belonging to class i .

Properties:

- Gini impurity = 0 for a pure set (all samples from one class)
- Gini impurity = 0.5 for a binary classification with 50/50 split

- When building a decision tree, we choose features with the **least Gini index** as the root node

30.2 Comparison of Splitting Criteria

Criterion	Task	Formula	Description
Entropy	Classification	$-\sum_{i=1}^C p_i \log_2(p_i)$	p_i is the frequency of label i at a node, C is the number of unique labels. Maximize the Gain.
Gini Impurity	Classification	$1 - \sum_{i=1}^C p_i^2$	p_i is the frequency of label i at a node, C is the number of unique labels. Minimize the Gini.
Variance	Regression	$\frac{1}{N} \sum_{i=1}^N (y_i - \mu)^2$	y_i is the label for instance i , N is the number of instances, $\mu = \frac{1}{N} \sum_{i=1}^N y_i$. Minimize the variance.

31 Decision Trees and Overfitting

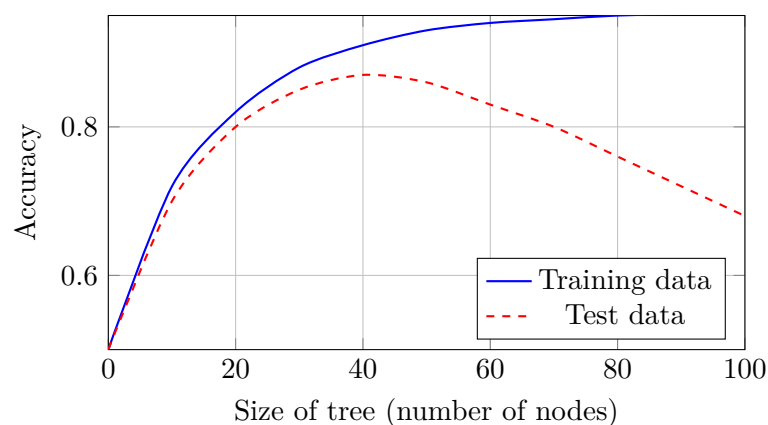
31.1 Why Decision Trees Overfit

Decision trees are **non-parametric models** with a structure determined by the data. As a result:

- They are very **flexible** and can easily fit the training set
- They have a **high risk of overfitting**
- Without constraints, they can create a leaf for every training example

31.2 Overfitting Example

Consider the relationship between tree size and accuracy:



Observations:

- Training accuracy continues to increase with tree size
- Test accuracy initially increases, then decreases
- The optimal tree size is around 30-40 nodes
- Larger trees overfit to the training data

31.3 Techniques to Prevent Overfitting

Standard machine learning techniques apply to decision trees:

1. **Early stopping:** Stop growing the tree before it perfectly fits the training data
 - Set a maximum depth
 - Require a minimum number of samples to split
 - Require a minimum information gain to split
2. **Pruning:** Build a full tree, then remove branches that don't improve validation performance
3. **Regularization:** Add penalties for tree complexity
4. **Data augmentation:** Increase training data
5. **Ensemble methods:** Combine multiple trees (Random Forest, boosting)

31.4 Pruning (Post-processing)

Pruning is a technique to reduce complexity *after* tree building:

1. **Build the full tree:** Grow the tree until each leaf node is pure or a stopping criterion is met
2. **Evaluate nodes from bottom to top:** Starting from the bottom, evaluate each node to see if pruning it would improve performance on a validation set
3. **Define cost-complexity measure:** Consider both the accuracy of the node and the size of the subtree rooted at that node
4. **Prune nodes iteratively:** Identify the node with the smallest cost-complexity measure and prune its subtree (turn it into a leaf labeled with the majority class)
5. **Repeat until done:** Continue until a suitable tree is achieved or further pruning doesn't improve performance

32 Decision Trees for Different Data Types

32.1 Continuous Attributes

So far, we have focused on categorical attributes, but decision trees can also handle continuous attributes.

Strategy: Pick a threshold to create a binary split.

For example, for a continuous attribute like temperature:

$$\phi(\text{temperature}) = \begin{cases} L & \text{if temperature} > 77.8 \\ R & \text{else} \end{cases}$$

How to choose the threshold?

- Consider all possible split points (or a sample of them)
- For each candidate threshold, compute the information gain (or Gini impurity reduction)
- Choose the threshold that maximizes information gain

Geometric interpretation:

- Each split creates an axis-aligned boundary in feature space
- For 2D data with continuous features x_1 and x_2 , splits create rectangular regions
- Example splits: $x_1 > \theta_1$, $x_2 < \theta_2$, $x_2 > \theta_3$, $x_1 < \theta_4$

32.2 Multi-class Classification

Decision trees naturally extend to multi-class problems (k different classes):

- At each leaf, predict the **most frequent class** in that subset
- Entropy for multi-class:

$$H(S) = - \sum_{c=1}^k p(c) \log_2 p(c)$$

Where $p(c)$ is the proportion of examples of class c in subset S

- Information gain and Gini impurity formulas extend naturally to k classes

33 Decision Trees: Summary**33.1 Advantages (Pros)**

- **Interpretable:** Humans can understand the reason for decisions
- **Handles irrelevant data:** Irrelevant features have Gain = 0 and won't be selected
- **Handles missing data:** Can be adapted to deal with missing values
- **Very compact:** After pruning, # nodes $\ll$ # training data
- **Fast at test time:** $O(\text{depth})$, where depth $\ll$ # training data
- **No feature scaling needed:** Works with features on different scales

33.2 Disadvantages (Cons)

- **Greedy algorithm:** May not find the globally optimal tree (locally optimal choices at each split)
- **NP-hard problem:** Finding the optimal tree is exponentially difficult (exponentially many possible trees)
- **Only axis-aligned splits:** Cannot naturally capture diagonal decision boundaries
 - Linear classifiers can have diagonal boundaries
 - Decision trees create rectangular partitions
 - This limits expressiveness for some problems
- **High variance:** Small changes in training data can lead to very different trees
- **Prone to overfitting:** Especially with deep trees

33.3 Key Takeaways

- **ID3 algorithm:** Grows decision tree from root down, greedily selecting the next best attribute using information gain
- **Entropy:** Measures uncertainty (impurity) in a set
- **Information Gain:** Measures the reduction in uncertainty following a split
- **Complete hypothesis space:** Searches the entire space of possible trees
- **Preference:** Prefers smaller trees and high information gain at the root
- **Overfitting:** Addressed by post-pruning (remove nodes while accuracy increases on validation set)
- **Performance:** Fast, compact, and interpretable

34 Random Forests

Random forests address many limitations of individual decision trees by combining multiple trees into an ensemble.

34.1 What is a Random Forest?

Random forests are ensembles of decision trees.

- Each tree is trained on a **bootstrapped version** of the training set (sampled with replacement)
- This creates diversity among the trees
- Different trees make different errors
- Combining their predictions reduces overall error

34.1.1 Sampling with Replacement (Bootstrapping)

Given a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$:

1. For each tree t_i , create a new dataset $\mathcal{D}_i$ by:
 - Randomly sampling n examples from $\mathcal{D}$ **with replacement**
 - Some examples may appear multiple times
 - Some examples may not appear at all (approximately 37% are left out)
2. Train tree t_i on dataset $\mathcal{D}_i$
3. Repeat for all T trees in the forest

34.2 Random Feature Selection

In addition to bootstrap sampling, random forests introduce randomness in feature selection:

- At each node, instead of considering all features for splitting, only a **random subset** of features is considered
- Typical choices:
 - For classification: $\sqrt{d}$ features, where d is the total number of features

- For regression: $d/3$ features
- This helps to **decorrelate** decision trees
- **Extremely randomized trees** take this further by sampling split thresholds randomly instead of optimizing them

34.3 Random Forest Prediction

The final prediction of a random forest is obtained by **aggregating** the predictions of all trees:

Random Forest Prediction

For classification:

$$\hat{y}_{\text{RF}}(\mathbf{x}) = \text{mode}\{f_{t_1}(\mathbf{x}), \dots, f_{t_T}(\mathbf{x})\}$$

(Majority vote across all trees)

For regression:

$$\hat{y}_{\text{RF}}(\mathbf{x}) = \frac{1}{T} \sum_{i=1}^T f_{t_i}(\mathbf{x})$$

(Average of all tree predictions)

34.4 Why Random Forests Work

Random forests work well because of several key principles:

1. **Variance reduction:** Averaging multiple models reduces variance without increasing bias
2. **Decorrelation:** Random feature selection ensures trees make different types of errors
3. **Robustness:** Errors of individual trees tend to cancel out
4. **Overfitting resistance:** Individual trees may overfit, but the ensemble typically does not

Bias-Variance Trade-off:

- Individual deep trees have low bias but high variance
- Averaging many trees keeps bias low while reducing variance
- Result: Better generalization performance

35 Ensemble Methods

Random forests are one example of a broader class of techniques called **ensemble methods**.

35.1 What are Ensemble Methods?

Ensemble learning combines different models to obtain more accurate predictions than any individual model.

Ensemble Learning Principle

“Wisdom of crowds”: Aggregating predictions from multiple models often yields better performance than using a single model.

35.2 Types of Ensemble Methods

There are three main types of ensemble methods:

1. **Bagging** (Bootstrap Aggregating)
2. **Boosting**
3. **Stacking**

36 Bagging (Bootstrap Aggregating)

36.1 The Bagging Algorithm

Bagging is a technique where we train T models on T different samplings of the dataset.

Procedure:

1. Take the original dataset $\mathcal{D}$
2. Create T bootstrap samples:
 - $\mathcal{D}_1$: Sample n examples from $\mathcal{D}$ with replacement
 - $\mathcal{D}_2$: Sample n examples from $\mathcal{D}$ with replacement
 - ...
 - $\mathcal{D}_T$: Sample n examples from $\mathcal{D}$ with replacement
3. Train a model on each bootstrap sample:
 - Model 1 trained on $\mathcal{D}_1$
 - Model 2 trained on $\mathcal{D}_2$
 - ...
 - Model T trained on $\mathcal{D}_T$
4. **Aggregate** predictions:
 - Classification: Majority vote
 - Regression: Average

36.2 Benefits of Bagging

- **Reduces variance:** Different bootstrap samples lead to different models; averaging reduces variance
- **Doesn't increase bias:** Each model is trained on a dataset of the same size as the original
- **Robust to overfitting:** Individual models may overfit to their bootstrap samples, but averaging helps
- **Parallel training:** All models can be trained independently and in parallel

36.3 When to Use Bagging

Bagging works best with:

- **High-variance, low-bias models:** e.g., deep decision trees

- Models that are sensitive to small changes in training data
- Situations where you want to reduce overfitting

37 Boosting

Boosting is a different ensemble strategy that trains models **sequentially**, with each new model focusing on the errors of previous models.

37.1 Key Differences from Bagging

Aspect	Bagging	Boosting
Training	Parallel (independent)	Sequential (dependent)
Sampling	Uniform (with replacement)	Weighted (focus on errors)
Model weights	Equal	Based on performance
Goal	Reduce variance	Reduce bias and variance

37.2 Boosting Intuition

- Trained models are **weighted based on their performance**
- Weak classifiers are learned **sequentially**
- The more mistakes the previous classifier makes on a dataset, the more those misclassified examples are **weighted for the next classifier**
- Each new model focuses on the “hard” examples that previous models got wrong

37.3 Weak Learners

Ensemble methods often use **weak learners** rather than complex models.

Definition 37.1 (Weak Learner). A weak learner is a classification model that performs **slightly better than random guessing**.

For binary classification with balanced classes:

- Random guessing: 50% accuracy
- Weak learner: slightly better than 50% (e.g., 55%)

Examples of weak learners:

- **Decision stump:** A decision tree with only one split (depth = 1)
- **1-Nearest Neighbor:** k-NN with k=1 on a subset of features
- Simple linear classifiers on a single feature

Why use weak learners?

- Fast to train
- Less likely to overfit individually
- Ensemble of many weak learners can create a strong classifier
- “Two heads are better than one” principle

37.4 AdaBoost (Adaptive Boosting)

AdaBoost is one of the most popular boosting algorithms.

AdaBoost Algorithm

Step 1: Initialize weights uniformly for all training samples:

$$w_i^{(1)} = \frac{1}{n} \quad \text{for } i = 1, \dots, n$$

Step 2: For $t = 1$ to T :

1. Train a weak classifier h_t using the current weights $w^{(t)}$
2. Compute the weighted error:

$$\epsilon_t = \sum_{i: h_t(\mathbf{x}_i) \neq y_i} w_i^{(t)}$$

3. Compute classifier weight:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

4. Update sample weights:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(\mathbf{x}_i))$$

(Increase weight for misclassified samples, decrease for correctly classified)

5. Normalize weights: $w^{(t+1)} \leftarrow w^{(t+1)} / \sum_i w_i^{(t+1)}$

Step 3: Final classifier:

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$$

(Weighted vote of all weak classifiers)

37.5 AdaBoost: Key Steps Explained

Step 1: Initialize with equal weights

- All training samples start with equal importance
- Build the first weak classifier on this uniformly weighted dataset
- Check which samples are correctly classified and which are not

Step 2: Increase weights for misclassified samples

- Samples that were misclassified get higher weights
- This makes classifying them correctly more important for the next model
- The next weak classifier will focus more on these "hard" examples

Step 3: Iterate until convergence

- Repeat until all data points are correctly classified
- OR until a maximum number of iterations is reached
- Each iteration focuses on the remaining difficult examples

37.6 AdaBoost Properties

- **Adaptive:** Adapts to the errors of previous weak learners
- **Reduces bias and variance:** Combines weak learners to create a strong learner
- **Sensitive to noisy data:** Can overfit if there are many outliers (keeps increasing their weights)
- **No need for feature selection:** Weak learners can be based on individual features
- **Fast training:** Each weak learner is simple and fast to train

38 Stacking

Stacking (stacked generalization) is a more sophisticated ensemble method that uses a **meta-model** to combine base models.

38.1 What is Stacking?

Stacking combines predictions of multiple ML models (called **base models**) to improve overall performance.

Key differences from bagging and boosting:

- Base models are typically **heterogeneous** (different types of models)
- Uses a **meta-model** to learn how to combine base model predictions
- Can be used for both regression and classification
- Often more powerful than bagging or boosting alone

38.2 The Stacking Process

Stacking Algorithm

Level 0 (Base Models):

1. Train multiple diverse base models on the training set:
 - Model 1: e.g., Decision Tree
 - Model 2: e.g., Logistic Regression
 - Model 3: e.g., Support Vector Machine
 - Model 4: e.g., Neural Network
2. Generate predictions from each base model on the training set

Level 1 (Meta-Model):

1. Use the predictions from base models as **features**
2. Train a **meta-model** that learns how to combine these predictions
3. The meta-model learns which base models to trust for which types of inputs

Prediction:

1. For a new input $\mathbf{x}$:
 - Get predictions from all base models
 - Feed these predictions to the meta-model
 - Meta-model outputs the final prediction

38.3 Why Stacking Works

- **Diversity:** Different models capture different patterns in the data
- **Complementary strengths:** Each model may excel in different regions of the input space
- **Learned combination:** The meta-model learns the optimal way to combine predictions (better than simple averaging)
- **Flexibility:** Can combine any types of models

38.4 Stacking Best Practices

- **Use diverse base models:** Different model types, different hyperparameters
- **Avoid data leakage:** Use cross-validation to generate training predictions for the meta-model
- **Keep meta-model simple:** Often a simple linear model or logistic regression works well
- **Regularization:** Add regularization to prevent the meta-model from overfitting

38.5 Comparison of Ensemble Methods

Method	Base Models	Training	Combination
Bagging	Same type (homogeneous), trained on bootstrap samples	Parallel, independent	Equal weight voting/averaging
Boosting	Same type (weak learners), trained sequentially	Sequential, each focuses on previous errors	Weighted voting based on model performance
Stacking	Different types (heterogeneous)	Parallel for base models, then meta-model	Meta-model learns optimal combination

38.6 Key Takeaways

Main Lessons

1. **Single models vs. Ensembles:** Ensembles generally outperform single models
2. **Bias-Variance tradeoff:** Different ensemble methods address different aspects
 - Bagging reduces variance
 - Boosting reduces both bias and variance
 - Stacking leverages diverse model strengths
3. **Interpretability vs. Performance:** Trade-off between understanding and accuracy
4. **Computational cost:** Ensembles are slower to train and predict than single models
5. **Real-world impact:** Random forests and boosting (e.g., XGBoost, LightGBM) are among the most successful ML methods in practice

Part V

Linear Discriminant Functions and Classification Methods

39 Linear Discriminant Functions

The objective of this lecture is to present methods for learning **linear discriminant** functions of the form:

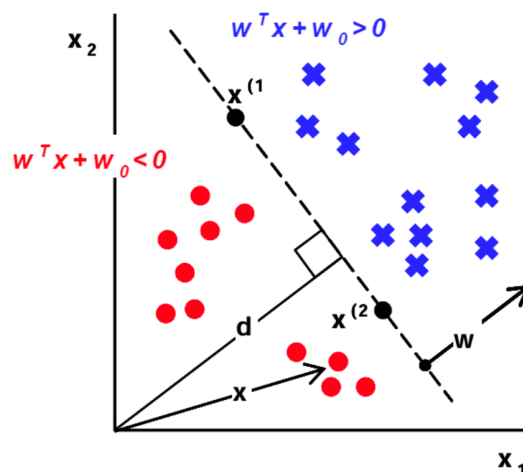
$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

where:

- w is the **weight vector**
- w_0 is the **threshold weight** or **bias**
- $\mathbf{x} = [x_1, \dots, x_n]$, $\mathbf{w} = [w_1, \dots, w_n]$

For a problem with 2 classes, the goal is to create a **linear function** $g(x)$ that separates the examples belonging to the two classes.

Note: $g(x) = 0 \Rightarrow$ either class



39.1 Learning the Linear Discriminant Function

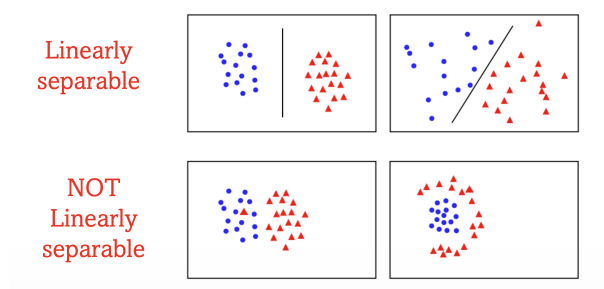
Suppose we have a set of m samples $\mathbf{x}_1, \dots, \mathbf{x}_m$, some tagged ω_1 and others tagged ω_2 .

We want to use these samples to determine the weights $\mathbf{w}$ and w_0 of the linear discriminating function.

Suppose also that there is a solution for which the probability of error is very low.

Thus, a reasonable approach is to look for a weight vector that classifies all of the samples correctly or that the probability of making a mistake is zero.

If such a **weight vector** exists, the samples are said to be **linearly separable**.

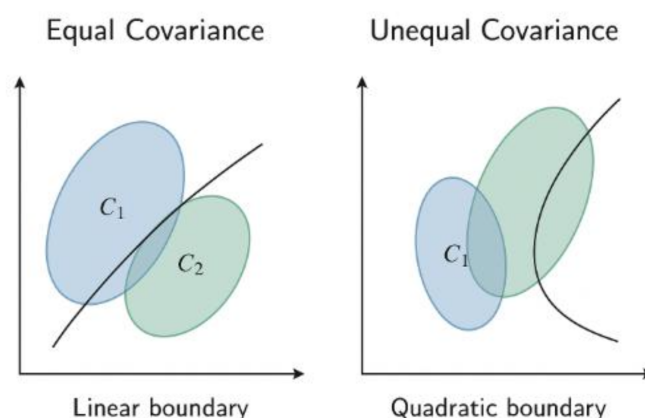


39.2 Linear Separability

Data are **linearly separable** if there exists a linear boundary (hyperplane) that perfectly separates the two classes.

Examples:

- **Linearly separable:** Classes that can be separated by a straight line (in 2D) or hyperplane (in higher dimensions)
- **NOT Linearly separable:** Classes that cannot be separated by any linear boundary

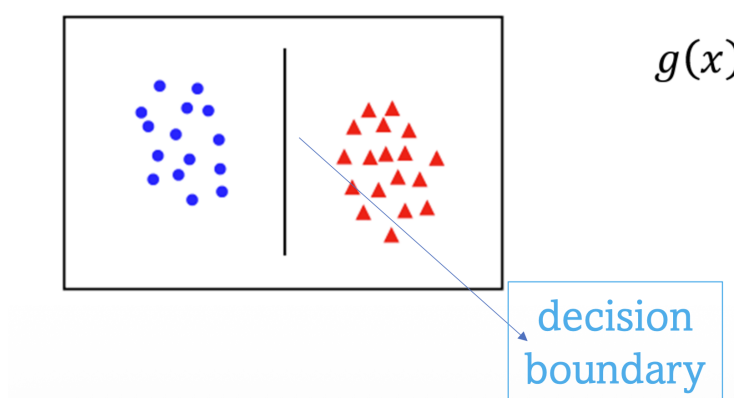


39.3 Properties of Linear Discriminant Functions

Linear discriminant functions are:

- **Optimal for Gaussian distributions with equal covariance:** When the data follows Gaussian distributions with equal covariance matrices, the optimal decision boundary is linear
- **May not be optimal for other data distributions,** but they are very simple to use
- **Knowledge of class densities is not required** when using linear discriminant functions
 - Therefore, we can say that this is a **non-parametric approach**

Note: With unequal covariance, the optimal decision boundary is quadratic rather than linear.



40 Linear Classifiers

Every linear discriminant function defines a linear classifier.

But not every linear classifier is necessarily described in the form of a "discriminant function" (although in practice, the terms are often used interchangeably).

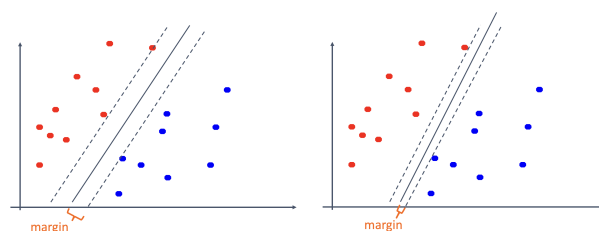
40.1 Decision Boundary

The linear discriminant function is defined as:

$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

The **decision boundary** is defined by $g(x) = w^T x + w_0 = 0$:

- In 2D, is a **line**
- In 3D, is a **plane**
- In nD, is a **hyperplane**



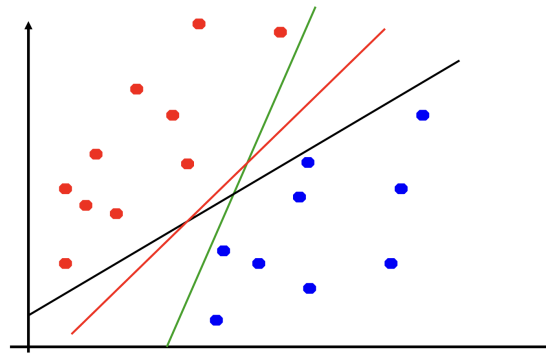
40.2 The Role of Weights

The training data is used to learn $\mathbf{w}$.

Important: Only $\mathbf{w}$ is needed to classify the new data point.

The equation $\mathbf{w}^T \mathbf{x}_i = 0$ defines a hyperplane passing through the origin of the feature space, having $\mathbf{w}$ as a normal vector.

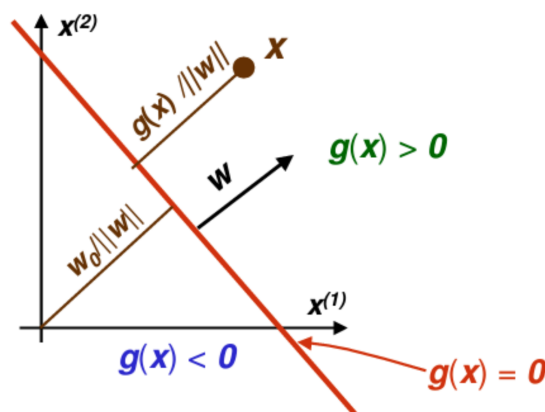
- $\mathbf{w}$ determines the orientation of the decision hyperplane
- w_0 determines the location of the decision surface



40.3 Which Hyperplanes?

If a solution vector exists (if there is a hyperplane) then:

- Which hyperplane a linear classifier choose when the data is linearly separable?
- The solution vector is not unique!



40.4 Optimization Approaches

There are different approaches to find the optimal weight vector:

40.4.1 Unit-Length Weight Vector

One possibility is to seek a **unit-length weight vector** that maximizes the minimum distance from the samples to the separating plane.

40.4.2 Minimum-Length Weight Vector

Another possibility is to seek the **minimum-length weight vector** satisfying:

$$\mathbf{w}^T \mathbf{x}_i \geq b, \quad \forall i$$

where b is a positive constant called the **margin**.

These techniques attempt to find a solution vector closer to the "middle" of the solution region:

- In this way, the resulting solution is more likely to classify new test samples correctly

40.5 The Margin

Definition 40.1 (Margin). The **margin** of a classifier is the distance to the closest points of either class.

A larger margin generally leads to better generalization performance on unseen data.

41 Multi-Class Problems

In case of a multi-class (C-class) problem:

Multi-Class Classification Strategies

One-vs-Rest Classifiers

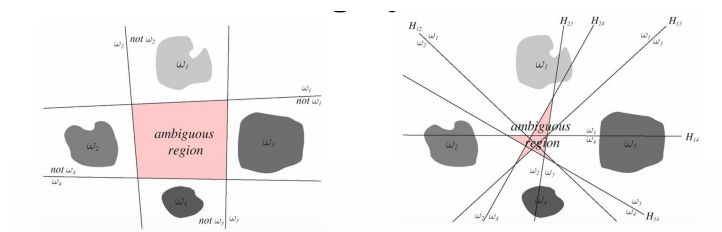
C-1 classifiers $g_i(\mathbf{x})$ are constructed, one for each class C_i versus non- C_i .

One-vs-One Classifiers

$C(C-1)/2$ binary classifiers (one-vs-one), and then classify a test sample following a **majority voting**.

Ambiguity Regions

Both approaches lead to areas of **ambiguity**.



42 LDF: Augmented Feature Vector

LDF: $g(x) = w^T x + w_0$

Augmented Feature Vector Transformation

We can write it as:

$$g(\mathbf{x}) = [w_0 \quad w^T] \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} = \mathbf{a}^T \mathbf{y} = g(\mathbf{y})$$

where:

- $\mathbf{a}$ is the **new weight vector**
- $\mathbf{y}$ is the **new feature vector** (called the **augmented feature vector**)

We added a dummy dimension to get a completely equivalent new **homogeneous** problem.

Old problem: $g(\mathbf{x}) = w^T \mathbf{x} + w_0$, with $\mathbf{x} = [x_1, \dots, x_d]$

New problem: $g(\mathbf{y}) = \mathbf{a}^T \mathbf{y}$, with $\mathbf{y} = [1, x_1, \dots, x_d]$

Feature augmenting is done only for simpler notation.

Note: Throughout these lecture notes we replace $\mathbf{a}$ with $\mathbf{w}$ for simplicity, since they both refer to weights.

43 LDF: Training Error

For the rest of the lecture, assume we have 2 classes.

We will use samples (training data) to determine weights $\mathbf{a}$ (or indicated with $\mathbf{w}$) in the discriminant function $g(\mathbf{y}) = \mathbf{a}^T \mathbf{y}$.

43.1 Criterion for Determining Weights

What should be our criterion for determining $\mathbf{a}$?

Suppose, for now that we want to minimize the training error (the number of misclassified samples $y_1, \dots, y_n$).

Zero Training Error Conditions

Recall that:

- $g(y_i) > 0 \Rightarrow y_i$ classified as class 1
- $g(y_i) < 0 \Rightarrow y_i$ classified as class 2

Thus, training error is 0 if:

$$\begin{aligned} g(y_i) &> 0 \quad \forall y_i \in c_1 \\ g(y_i) &< 0 \quad \forall y_i \in c_2 \end{aligned}$$

This is equivalent to:

$$\begin{aligned} \mathbf{a}^T y_i &> 0 \quad \forall y_i \in c_1 \\ \mathbf{a}^T y_i &< 0 \quad \forall y_i \in c_2 \end{aligned}$$

Which can be rewritten as:

$$\begin{aligned} \mathbf{a}^T y_i &> 0 \quad \forall y_i \in c_1 \\ \mathbf{a}^T (-y_i) &> 0 \quad \forall y_i \in c_2 \end{aligned}$$

44 LDF: Normalization

This suggests a so-called "**normalization**":

Normalization Procedure

- Replace all examples from class 2 by their negatives:

$$y_i \rightarrow -y_i \quad \forall y_i \in c_2$$

- Seek weight vector $\mathbf{a}$ such that:

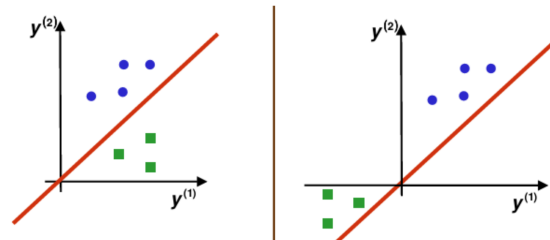
$$\mathbf{a}^T y_i > 0 \quad \forall y_i$$

- If such $\mathbf{a}$ exists, it is called a **separating** or **solution vector**
- Original samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ can indeed be separated by a line in that way

44.1 Before and After Normalization

Before normalization: Seek a hyperplane that separates patterns from different categories.

After normalization: Seek a hyperplane that puts normalized patterns on the same positive side.



45 LDF: Solution Region

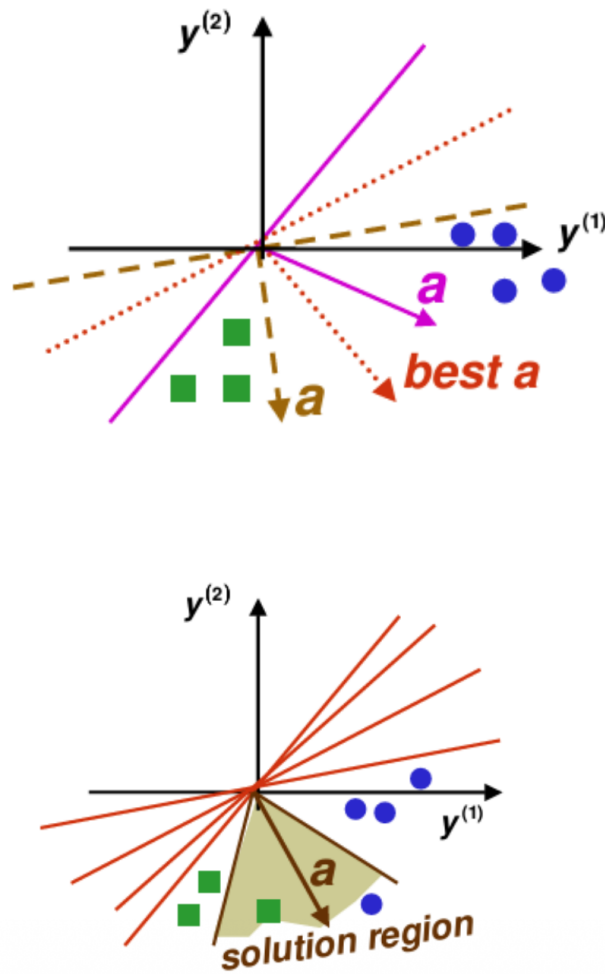
Coming back to the "which hyperplane" question...

We can see that many solutions for the weight vector $\mathbf{a}$ exist:

$$\mathbf{a}^T y_i = \sum_{k=0}^d a_k y_i^{(k)} > 0$$

Solution Region

The **solution region** corresponding to $\mathbf{a}$ consists of all points classified on one side of the hyperplane, where the hyperplane is perpendicular to the **normal vector** $\mathbf{a}$.

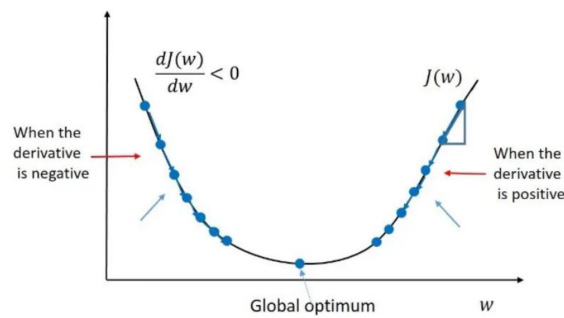


46 Gradient Descent

Gradient Descent is one of the simplest approaches to calculating a discriminant function. It is an iterative method for progressively adjusting the **weights**.

Key Principle

The gradient vector in space $\mathbf{W}$ points to the direction of **maximum deviation** of a function to be maximized/minimized.



46.1 Gradient Descent Update Rule

The procedure consists of updating the value of the weight vector at the $k + 1$ step with a contribution proportional to the gradient module calculated at the previous step:

Gradient Descent Formula

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \rho_k \nabla J(\mathbf{w})|_{\mathbf{w}=\mathbf{w}(k)}$$

where $J(\mathbf{w})$ is an **evaluation function** (*loss function*) that must be **minimized**.

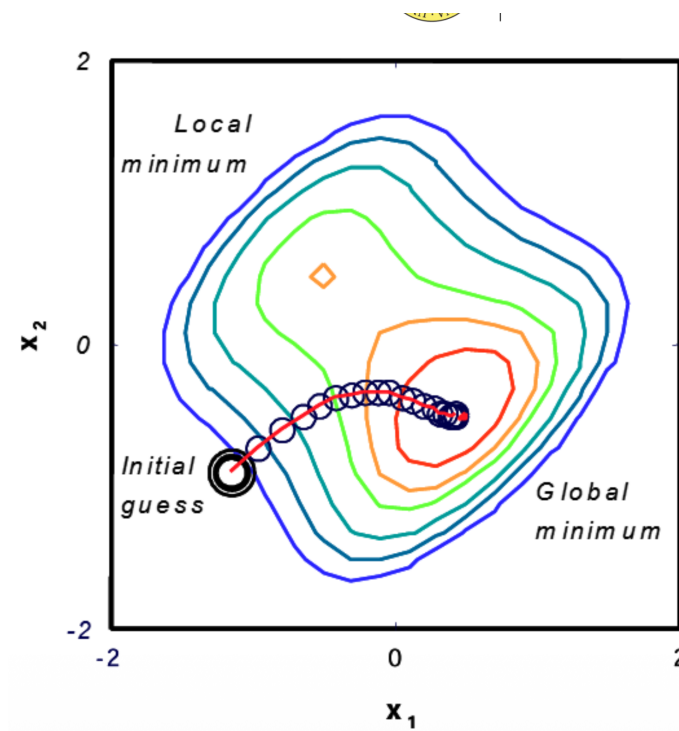
- $J(\mathbf{w})$ is chosen in such a way as to reach **the minimum** when $\mathbf{w}$ approaches the **optimal solution**
- That is, $J(\mathbf{w})$ should be convex, and $\mathbf{w}$ is the solution vector

where:

- ρ_k is the **learning rate** (step size) at iteration k
- $\nabla J(\mathbf{w})$ is the **gradient** of the loss function

46.2 Local vs Global Minimum

- The minimum of $J(\mathbf{w})$ is obtained by moving $\mathbf{w}$ in the direction opposite to the gradient (in the direction of steepest descent)
- Gradient descent is guaranteed to find only a **local minimum**
- Very popular, as it is simple and applicable to any function



46.3 Gradient Operator

∇ is the gradient operator symbol, given by:

$$\nabla J = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \vdots \\ \vdots \\ \frac{\partial J}{\partial w_n} \end{bmatrix}$$

The update rule remains:

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \rho_k \nabla J(\mathbf{w})|_{\mathbf{w}=\mathbf{w}(k)}$$

ρ_k (**learning rate**) is an appropriate scalar value that varies with the iteration k , fixing the "magnitude" of the correction.

46.4 Gradient Descent Summary

Gradient Descent Algorithm

To summarize, at the k -th iteration:

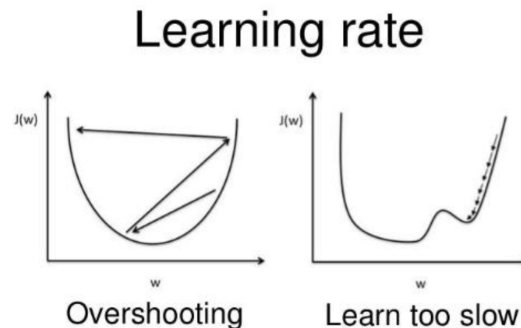
1. Start with a chosen weight vector $\mathbf{w}(1)$ and calculate its gradient $J(\mathbf{w}(1))$
2. Then, move from $\mathbf{w}(1)$ in the direction of the steepest descent (opposite to the gradient) by a fixed step size ρ_k to obtain the new vector $\mathbf{w}(2)$

Main issue: How to determine the optimal choice of ρ_k

46.5 Learning Rate Selection

The choice of learning rate ρ_k is critical:

- If ρ_k is too **large**, gradient descent may overshoot the minimum and possibly never find the minimum
- If ρ_k is too **small**, it needs too many iterations



47 Perceptron

Let's learn a **binary classifier** with linear discriminant function.

Important Note

From this point to the rest, $\mathbf{w}$ and $\mathbf{x}$ are used for an **augmented feature vector** for simplicity, instead of using $\mathbf{a}$ and $\mathbf{y}$.

To do so, we need to choose a $J(\mathbf{w})$ such that the gradient descent can solve it, in this way we can find the minimum of $J(\mathbf{w})$.

47.1 Perceptron Criterion Function

Perceptron Criterion Function

A good choice is the **Perceptron criterion function**:

$$J(\mathbf{w}) = - \sum_{i \in X} \mathbf{w}^T \mathbf{x}_i$$

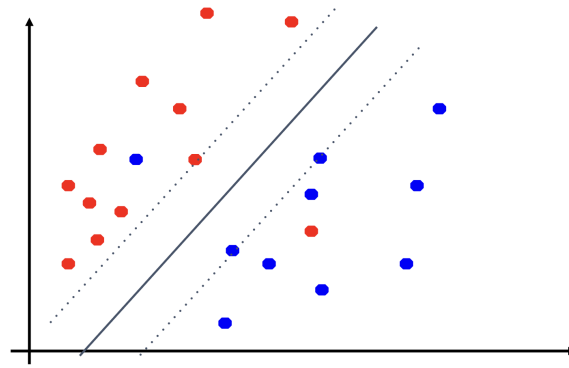
where X is a set of **samples misclassified** by $\mathbf{w}$.

Key property: $J(\mathbf{w})$ is non-negative since $\mathbf{w}^T \mathbf{x}_i < 0$ for all misclassified samples.

Important: Do not forget to "normalize" the training set by replacing all examples from class ω_2 with their negatives.

47.2 Convergence Limitation

We want to use gradient descent to solve the $J(\mathbf{w})$ so that it can converge; however, for such data (not linearly separable), we cannot converge.



47.3 Perceptron Gradient Descent

To find the minimum of $J(\mathbf{w})$, we use gradient descent, which is defined by:

- i -th component of the gradient of $J(\mathbf{w})$ is equal to $\partial J / \partial w_i$, thus:

$$J(\mathbf{w}) = - \sum_{i \in X} \mathbf{w}^T \mathbf{x}_i \quad \Rightarrow \quad \nabla J = - \sum_{i \in X} \mathbf{x}_i$$

Note that this does not depend on $\mathbf{w}$.

Perceptron Rule

The update rule becomes:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k \cdot \sum_{i \in X} \mathbf{x}_i, \quad \text{when } \rho_k = \frac{1}{k}$$

This is known as the **Perceptron rule**.

Historical note: Perceptron is the first **Neural Network**, which was then evolved to more complex versions such as **Multilayer Perceptron**.

47.4 Perceptron Example (Non-Separable Scenario)

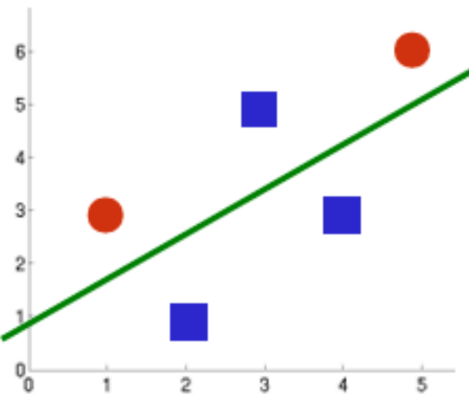
Suppose we have 2 features and 5 samples for 2 classes:

- Class 1: $[2, 1]$, $[4, 3]$, $[3, 5]$
- Class 2: $[1, 3]$, $[5, 6]$

These samples are not separable with a line, still we will try to get approximate separation by a line.

Obtain the augmented features and normalize the data, by doing so we obtain:

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}, \begin{bmatrix} -1 \\ -1 \\ -3 \end{bmatrix}, \begin{bmatrix} -1 \\ -5 \\ -6 \end{bmatrix}$$



47.5 Applying the Perceptron Method

Let's apply the perceptron method:

- Initial weights $[1 \ 1 \ 1]$
- This is a line $x_1 + x_2 + 1 = 0$
- Fixed learning rate $\rho(k) = 1$

Iteration 1:

$$\mathbf{w}_1^T \mathbf{x}_1 \Rightarrow [1 \ 1 \ 1][1 \ 2 \ 1]^T > 0$$

$$\mathbf{w}_1^T \mathbf{x}_2 \Rightarrow [1 \ 1 \ 1][1 \ 4 \ 3]^T > 0$$

$$\mathbf{w}_1^T \mathbf{x}_3 \Rightarrow [1 \ 1 \ 1][1 \ 3 \ 5]^T > 0$$

$$\mathbf{w}_1^T \mathbf{x}_4 \Rightarrow [1 \ 1 \ 1][-1 \ -1 \ -3]^T = -5 < 0$$

$$\text{Update } \mathbf{w}!!! \ \mathbf{w}_2 = [1 \ 1 \ 1] + [-1 \ -1 \ -3] = [0 \ 0 \ -2]$$

Continue iteration:

$$\mathbf{w}_2^T \mathbf{x}_5 \Rightarrow [0 \ 0 \ -2][-1 \ -5 \ -6]^T > 0$$

Iteration 2:

$$\mathbf{w}_2^T \mathbf{x}_1 \Rightarrow [0 \ 0 \ -2][1 \ 2 \ 1]^T < 0$$

Update $\mathbf{w}$ again!!!

$$\mathbf{w}_3 = [0 \ 0 \ -2] + [1 \ 2 \ 1] = [1 \ 2 \ -1]$$

...

...

Non-Convergence Result

We can continue this forever since there is no solution vector $\mathbf{w}$ satisfying the perceptron rule.

The data is **not linearly separable**, so the perceptron algorithm will not converge to a solution.

47.6 Need for Relaxation

We want to learn something like this, but our Perceptron constraint does not allow!

Solution: Let's apply some **relaxation**.

47.7 Perceptron Summary

Perceptron Convergence Properties

- If classes are **linearly separable**, the perceptron rule is **guaranteed to converge** to a valid solution
 - Some versions of the perceptron rule use a **variable learning rate** $\rho(k)$; for now, we fixed it as $1/k$
- However, if the two classes are **not linearly separable**, the perceptron rule will **not converge**
 - Since no weight vector $\mathbf{w}$ can correctly classify every sample in a non-separable dataset, the corrections in the perceptron rule will never cease
 - One ad-hoc solution to this problem is to enforce convergence by using variable learning rates $\rho(k)$ that approach zero as $k \rightarrow \infty$
 - We will see once we introduce NN architectures

48 The Relaxation Method

48.1 Relaxation Criterion Function (1)

The $J(\mathbf{w})$ takes the form:

$$J_q(\mathbf{w}) = \sum_{i \in X} (\mathbf{w}^T \mathbf{x}_i)^2$$

- Also, this function takes into account the **misclassified samples**, and it is minimized when $\mathbf{w}$ is the solution vector
- Its main difference is that its **gradient is now continuous**, whereas the gradient of the perceptron function J is not
- So, such a function is suitable for finding smooth surfaces, but this may lead to problems
 - In fact, in this way, the points very far from the origin $(0, 0, \dots, 0)$ (i.e., with large feature magnitudes == outliers) are weighted a lot

48.2 Relaxation Criterion Function (2)

Another option for $J(\mathbf{w})$ is:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i \in X} \left\{ \frac{(\mathbf{w}^T \mathbf{x}_i - b)^2}{\|\mathbf{x}_i\|^2} \right\}$$

where $b \geq 0$ margin (e.g., $b = 1$) and $\mathbf{w}^T \mathbf{x}_i \leq b$.

- Dividing by $\|\mathbf{x}_i\|^2$ ensures that samples **contribute roughly equally**, regardless of their distance from the origin

The gradient descent becomes:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \rho_k \sum_{i \in X} \left\{ \frac{(\mathbf{w}^T \mathbf{x}_i - b) \mathbf{x}_i}{\|\mathbf{x}_i\|^2} \right\}$$

with $\mathbf{x}_i$ that verify $\mathbf{w}^T \mathbf{x}_i \leq b$ and $\mathbf{w}$ is calculated with the usual rule.

$J(\mathbf{w}) > 0$ always, and $J(\mathbf{w}) = 0$ if $\mathbf{w}^T \mathbf{x}_i \geq b \forall \mathbf{x}_i$

49 Minimum Squared Error (MSE) Solution

The classical MSE criterion provides an alternative to the perceptron rule.

Key Difference: Perceptron vs MSE

Perceptron rule: Only considers **misclassified samples**, since these are the only ones that violate the $J(\mathbf{w})$.

MSE criterion: Looks for a solution involving **all of the samples** (not just the misclassified ones).

- The goal is to verify $\mathbf{w}^T x_i = b_i$, where the b_i are some arbitrarily specified **positive constants**

49.1 Problem Transformation

We have replaced the problem of finding the solution to a set of linear inequalities (*perceptron*), now, we have the problem of finding the solution for a set of linear equations (*MSE*).

- **Perceptron:** $\mathbf{w}^T x_i > 0$ for all samples x_i , this solves the system of linear inequalities
- **MSE:** $\mathbf{w}^T x_i = b_i$ for all samples x_i , this solves the system of linear equations

49.2 Comparison: Perceptron Rule vs MSE/Relaxation Method

Aspect	Perceptron Rule	MSE / Relaxation Method
Loss function	Step / misclassification-based	Continuous quadratic (MSE)
Update	Only for misclassified samples	Gradient descent on all samples
Differentiability	Non-differentiable	Differentiable
Surface	Piecewise constant	Smooth (quadratic)

49.3 MSE Procedure

MSE Procedure

1. Choose positive constants $b_1, b_2, \dots, b_n$
2. Try to find a weight vector $\mathbf{w}$ such that $\mathbf{w}^T \mathbf{x}_i = b_i$ for all samples x_i
3. If we can find a weight vector $\mathbf{w}$ such that $\mathbf{w}^T \mathbf{x}_i = b_i$ for all samples x_i , then $\mathbf{w}$ is a solution because b_i 's are all positive
4. Consider all the samples

49.4 Normalization and Dataset Setup

For consistency, we will continue assuming that examples from class ω_2 have been replaced by their **negative vector**, although this is not a requirement for the MSE solution. Thus, we have:

$$x_{i,n+1} = 1 \text{ for } \omega_1 \quad \text{and} \quad x_{i,n+1} = -1 \text{ for } \omega_2$$

Suppose we have the following dataset:

$$\begin{aligned} \text{1st sample: } & x_{1,1}w_1 + x_{1,2}w_2 + \dots + x_{1,n}w_n + x_{1,n+1}w_{n+1} = b_1, \quad b_1 \geq 0 \\ \text{2nd sample: } & x_{2,1}w_1 + x_{2,2}w_2 + \dots + x_{2,n}w_n + x_{2,n+1}w_{n+1} = b_2, \quad b_2 \geq 0 \\ & \dots \\ \text{N-th sample: } & x_{N,1}w_1 + x_{N,2}w_2 + \dots + x_{N,n}w_n + x_{N,n+1}w_{n+1} = b_N, \quad b_N \geq 0 \end{aligned}$$

49.5 Matrix Formulation

We then obtain N equations (equal to the training samples, N_1 for ω_1 and N_2 for ω_2), with $N = N_1 + N_2$.

We need to solve N equations.

In matrix notation we have: $\mathbf{X} \cdot \mathbf{w} = \mathbf{b}$ where:

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,n} & \pm 1 \\ x_{2,1} & & \ddots & & \vdots \\ \vdots & & & \ddots & \vdots \\ x_{N,1} & \dots & \dots & x_{N,n} & \pm 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \\ w_{n+1} \end{bmatrix}$$

Thus, we need to solve a linear system: $\mathbf{X} \cdot \mathbf{w} = \mathbf{b}$

49.6 Error Vector and MSE Criterion

Generally, $N \gg n + 1$, so there are more equations than unknowns.

$$\mathbf{X} \cdot \mathbf{w} = \mathbf{b} \Rightarrow \mathbf{w} = \mathbf{X}^{-1}\mathbf{b}$$

Let's now inject the **error vector** (needed to obtain the loss function), defined as:

$$\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{b}$$

(imagine this as $\mathbf{b}$ is the real class label and $\mathbf{X}\mathbf{w}$ is your predictions)

MSE finds $\mathbf{w}$ which minimises the length of the error vector $\mathbf{e}$. Thus, minimise the **minimum squared error** criterion function $J(\mathbf{w})$:

$$J_M(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{b})^T(\mathbf{X}\mathbf{w} - \mathbf{b}) = \|\mathbf{X}\mathbf{w} - \mathbf{b}\|^2$$

49.7 Analytical Solution

Unlike the perceptron criterion, the MSE function is quadratic and differentiable.

Therefore, we can find its minimum **analytically** by setting the gradient to zero.

The gradient of the objective function $J_M(\mathbf{w})$:

- Find derivative of $J_M(\mathbf{w})$ wrt. to $\mathbf{w}$

Given $J_M(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{b})^T(\mathbf{X}\mathbf{w} - \mathbf{b}) = \|\mathbf{X}\mathbf{w} - \mathbf{b}\|^2$ in matrix format, we differentiate the expression $(\mathbf{X}\mathbf{w} - \mathbf{b})^T(\mathbf{X}\mathbf{w} - \mathbf{b})$ with respect to $\mathbf{w}$.

Let's denote $Y = \mathbf{X}\mathbf{w} - \mathbf{b}$, then $J_M(\mathbf{w}) = Y^TY$

49.8 Gradient Derivation

Now, differentiating $J_M(\mathbf{w})$ wrt. $\mathbf{w}$, we have:

$$\frac{dJ_M(\mathbf{w})}{d\mathbf{w}} = \frac{d}{d\mathbf{w}}(Y^T Y)$$

Remember the chain rule, if not clear to you:

$$= 2Y^T \frac{dY}{d\mathbf{w}} \Rightarrow \frac{dY}{d\mathbf{w}} = \frac{d}{d\mathbf{w}}(\mathbf{X}\mathbf{w} - \mathbf{b}) = \mathbf{X}$$

Therefore,

$$\frac{dJ_M(\mathbf{w})}{d\mathbf{w}} = 2Y^T \mathbf{X} = 2(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X}$$

49.9 Finding the Optimal Solution

Pseudoinverse Solution

By setting the **gradient to zero**, this becomes:

$$2(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X} = 0 \Rightarrow \mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{b}$$

(the proof can be found below)

If we multiply by $[\mathbf{X}^T \mathbf{X}]^{-1}$, we get:

$$\mathbf{w} = [\mathbf{X}^T \mathbf{X}]^{-1} \mathbf{X}^T \mathbf{b} = \mathbf{X}^\# \mathbf{b}$$

where $\mathbf{X}^\# = [\mathbf{X}^T \mathbf{X}]^{-1} \mathbf{X}^T$ is the **pseudoinverse** of $\mathbf{X}$.

Consequently, $\mathbf{w} = \mathbf{X}^\# \mathbf{b}$ is the optimal solution according to the MSE method.

49.10 Proof of the Gradient Result

Starting from:

$$2(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X} = 0$$

Step 1: Since we have a factor of 2 on the left side, divide both sides by 2:

$$(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X} = 0$$

Step 2: The expression $(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X} = 0$ can be rewritten by expanding the transpose:

$$(\mathbf{X}\mathbf{w})^T \mathbf{X} - \mathbf{b}^T \mathbf{X} = 0$$

Step 3: We know that $(\mathbf{X}\mathbf{w})^T \mathbf{X} = \mathbf{w}^T \mathbf{X}^T \mathbf{X}$ (since $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$):

$$\mathbf{w}^T \mathbf{X}^T \mathbf{X} - \mathbf{b}^T \mathbf{X} = 0$$

Step 4: This equality holds if:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{b}$$

This is the desired result, so we have shown that:

$$2(\mathbf{X}\mathbf{w} - \mathbf{b})^T \mathbf{X} = 0 \Rightarrow \mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{b}$$

50 MSE Properties

Dependence on Margin Vector

$\mathbf{w} = \mathbf{X}^\# \mathbf{b}$ shows that the MSE solution depends on the **margin vector $\mathbf{b}$** .

- Different choices of $\mathbf{b}$ lead to different properties of the solution
- If $\mathbf{b}$ is arbitrarily fixed, there is no evidence that the MSE solution gives a separator vector in the case of linear separability
- So, how to choose $\mathbf{b}$ in the MSE procedure?
 - Good choice is $b_1 = b_2 = \dots = B_n = 1$. In this case, the MSE solution is identical to Fischer's linear discriminant solution (*next lesson*)

50.1 Sensitivity to Outliers

Warning: Outlier Sensitivity

MSE pays too much attention to **outlier examples!**

- Because MSE squares the error, **large errors (outliers)** have a disproportionately large influence on the solution
- For classification, where we only care about *which side* of the boundary a sample is on, this sensitivity is undesirable

50.2 Computational Challenges

$\mathbf{X}^\#$ is difficult to obtain:

- As this requires inverting a large matrix, which is:
 - **expensive** for high-dimensional data
 - **impossible** if $\mathbf{X}^T \mathbf{X}$ is **singular** (non-invertible)
- The analytical solution is *computationally heavy* and sometimes *ill-defined*

50.3 Lack of Flexibility

It is not an iterative process, indeed, $\mathbf{w}$ is obtained directly through mathematical computations in **one step**.

- Even if this sounds good,
- it means we **lose flexibility**
 - E.g., no gradual correction
 - online learning

51 Example: Perceptron vs MSE

Compute the **perceptron** and **MSE** solution for the dataset:

Class 1 ($\mathbf{X}_1$): [(1, 6), (7, 2), (8, 9), (9, 9)]

Class 2 ($\mathbf{X}_2$): [(2, 1), (2, 2), (2, 4), (7, 1)]

51.1 Perceptron Learning

Assume $\rho_k = 0.1$ and an online update rule.

Assume $\mathbf{w}_1 = [0.1, 0.1, 0.1]$

Step 1: Normalize the dataset (augment features and replace class 2 with negatives):

$$\begin{bmatrix} 1 & 1 & 6 \\ 1 & 7 & 2 \\ 1 & 8 & 9 \\ 1 & 9 & 9 \\ -1 & -2 & -1 \\ -1 & -2 & -2 \\ -1 & -2 & -4 \\ -1 & -7 & -1 \end{bmatrix}$$

51.2 Perceptron Iteration

Step 2: Iterate through all the examples ($\mathbf{w}^T \mathbf{x}_i$) and update $\mathbf{w}(k)$:

Data point 1: $[0.1 \ 0.1 \ 0.1]^T [1 \ 1 \ 6]$ this is > 0 so correctly classified, therefore there is no update

Data point 2: $[0.1 \ 0.1 \ 0.1]^T [1 \ 7 \ 2]$ this is > 0 so correctly classified, therefore there is no update

...

Data point 5: $[0.1 \ 0.1 \ 0.1]^T [-1 \ -2 \ -1]$ this is < 0 so we have to apply an update:

$$\mathbf{w}(2) = \mathbf{w}(1) + \rho_1 [-1 \ -2 \ -1] = [0 \ -0.1 \ 0]$$

...

Data point 1: $[0 \ -0.1 \ 0]^T [1 \ 1 \ 6]$ this is < 0 so update:

$$\mathbf{w}(3) = \mathbf{w}(2) + \rho_1 [1 \ 1 \ 6] = [0 \ -0.1 \ 0] + 0.1 [1 \ 1 \ 6] = [0.1 \ 0 \ 0.6]$$

Data point 2: $[0.1 \ 0 \ 0.6]^T [1 \ 7 \ 2] > 0 \Rightarrow$ no update

...

The perceptron with this data and setup converges after 175 iterations, when $\mathbf{w} = [-3.5 \ 0.3 \ 0.7]$

51.3 MSE Solution

MSE: This solution will be found in one shot $\mathbf{w} = [\mathbf{X}^T \mathbf{X}]^{-1} \mathbf{X}^T \mathbf{b}$

For the choice of $\mathbf{b} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$, the solution is:

$$\mathbf{w} = [-1.1870 \ 0.0746 \ 0.1959]$$

Dataset:

- $\mathbf{X}_1 = [(1, 6), (7, 2), (8, 9), (9, 9)]$
- $\mathbf{X}_2 = [(2, 1), (2, 2), (2, 4), (7, 1)]$

As seen in the figure, the MSE misclassifies one of the samples.

52 Perceptron vs. MSE (Summary)

Comparison Summary

Perceptron rule: The perceptron rule always finds a solution if the classes are linearly separable, but does not converge if the classes are non-separable.

MSE criterion: The MSE solution has guaranteed convergence, but it may not find a separating hyperplane if classes are linearly separable.

- Notice that MSE tries to minimize the sum of the squares of the distances of the training data to the separating hyperplane, as opposed to finding this hyperplane

52.1 Gradient Descent Alternative for MSE

The objective function $J_M(\mathbf{w})$ can also be minimized using a gradient descent procedure. In this way, we avoid:

- the problems that arise when $\mathbf{X}^T\mathbf{X}$ is singular (happens if samples are correlated)
- the need for working with large matrices

Since $\nabla J_M(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{b})$, the gradient descent algorithm is:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \rho_k \mathbf{X}^T(\mathbf{X}\mathbf{w}_k - \mathbf{b}) \quad \text{with } \mathbf{w}_1 \text{ arbitrary}$$

[$\mathbf{X}$ and $\mathbf{b}$ are in matrix and vector form, written for all data]

Convergence Property

It can be shown that if $\rho_k = \rho_1/k$ where ρ_1 is any positive constant, this rule generates a sequence of weight vectors that converge to a solution to:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{b}) = 0$$

The gradient descent algorithm leads to the solution **regardless of whether $\mathbf{X}^T\mathbf{X}$ is singular or not.**

53 The Least MSE (LMSE = Widrow-Hoff Method)

Considering **one sample at a time**, we get:

LMS Rule

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k (b_k - \mathbf{w}_k^T \mathbf{x}_k) \mathbf{x}_k$$

such that b_k is the value for the k -th sample of the margin $\mathbf{b}$ ($b_k \geq 0$), i.e., it depends on the sample $\mathbf{x}_k$.

This method is called **Widrow-Hoff, least-mean-squares (LMS) or delta rule**:

- reduces **storage requirements** by considering single samples **sequentially**

53.1 Correction Mechanism

Widrow-Hoff rule "corrects" $\mathbf{w}_k$ whenever $\mathbf{w}_k^T \mathbf{x}_k \neq b_k$.

Why?

The weight change is:

$$\Delta \mathbf{w}_k = \mathbf{w}_{k+1} - \mathbf{w}_k = \rho_k (b_k - \mathbf{w}_k^T \mathbf{x}_k) \mathbf{x}_k$$

Case 1: Prediction is correct

$$\mathbf{w}_k^T \mathbf{x}_k = b_k$$

then

$$\Delta \mathbf{w}_k = \rho_k (b_k - \mathbf{w}_k^T \mathbf{x}_k) \mathbf{x}_k = \rho_k (0) \mathbf{x}_k = 0$$

so $\mathbf{w}_{k+1} = \mathbf{w}_k \rightarrow$ **no correction happens.**

53.2 Case 2: Prediction is Wrong

$$\mathbf{w}_k^T \mathbf{x}_k \neq b_k$$

then

$$\Delta \mathbf{w}_k = \rho_k (b_k - \mathbf{w}_k^T \mathbf{x}_k) \mathbf{x}_k \neq 0$$

so $\mathbf{w}_{k+1} \neq \mathbf{w}_k \rightarrow$ **the weight vector is corrected.**

- The advantage of "correcting only when $\mathbf{w}_k^T \mathbf{x}_k \neq b_k$ " is that learning is **efficient, stable, proportional, and error-driven**
- We adjust weights **exactly when and as much as needed**

53.3 Widrow-Hoff Method: Summary**Widrow-Hoff Advantages**

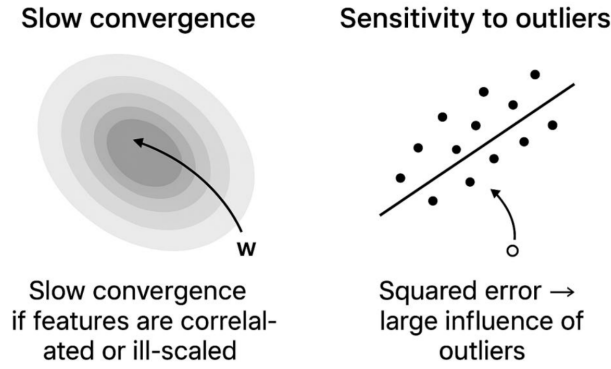
Iterative, gradient-descent-based update:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k (b_k - \mathbf{w}_k^T \mathbf{x}_k) \mathbf{x}_k$$

- Can update **sample-by-sample**
 - Low computational cost per iteration, scalable to large datasets
- Learning rate controls convergence speed
- Adapts to new data (suitable for online learning)

53.4 Widrow-Hoff Method: Disadvantages

- Can converge slowly
 - Sensitive to correlated features and outliers
- Needs careful learning rate
- **Requires multiple iterations; not one-step like analytical MSE**
- May struggle with non-linear or non-separable data



54 The Ho-Kashyap Method

MSE Limitation

The main limitation of the MSE criterion is the **lack of guarantees** that a separating hyperplane will be found in the linearly separable case.

All we can say about the MSE rule is that it minimises $\|\mathbf{X}\mathbf{w} - \mathbf{b}\|^2$ when $\mathbf{b}$ is chosen arbitrarily.

This means that whether MSE finds a separating hyperplane or not depends on how properly the margin $\mathbf{b}$ is selected (notice that in some sources $\mathbf{b}$ is referred to as *target outputs*).

54.1 Ho-Kashyap Approach

If the training samples are linearly separable, then there must be vectors $\mathbf{w}^*$ and $\mathbf{b}^*$ such that $\mathbf{X}\mathbf{w}^* = \mathbf{b}^* > 0$ where $\mathbf{b}^* > 0$, meaning that every component of $\mathbf{b}^*$ is **positive**.

- If $\mathbf{b}^*$ were **known**, one could simply use MSE solution to compute separating hyperplane
- **Idea**: find both $\mathbf{w}^*$ and $\mathbf{b}^*$
- Minimize the following criterion function, restricting to positive $\mathbf{b}$:

$$J_{HK}(\mathbf{w}, \mathbf{b}) = \|\mathbf{X}\mathbf{w} - \mathbf{b}\|^2$$

- $J_{HK}(\mathbf{w}^*, \mathbf{b}^*) = 0$

54.2 Gradient Descent Optimization

$$J_{HK}(\mathbf{w}, \mathbf{b}) = \|\mathbf{X}\mathbf{w} - \mathbf{b}\|^2$$

As usual, take partial derivatives with respect to $\mathbf{w}$ and $\mathbf{b}$:

$$\nabla_{\mathbf{w}} J_{HK} = -2\mathbf{W}^T(\mathbf{X}\mathbf{w} - \mathbf{b}) = 0$$

$$\nabla_{\mathbf{b}} J_{HK} = -2(\mathbf{X}\mathbf{w} - \mathbf{b}) = 0$$

Use gradient descent to find a minimum $J_{HK}(\mathbf{w}, \mathbf{b})$.

Alternate the two steps below until convergence:

- Fix $\mathbf{b}$ and minimize $J_{HK}(\mathbf{w}, \mathbf{b})$ with respect to $\mathbf{w}$
- Fix $\mathbf{w}$ and minimize $J_{HK}(\mathbf{w}, \mathbf{b})$ with respect to $\mathbf{b}$

54.3 Ho-Kashyap Rule

Ho-Kashyap Rule

The resulting iterative **Ho-Kashyap rule** is:

$$\mathbf{b}_{k+1} = \mathbf{b}_k - \frac{1}{2}\rho_k[\nabla_{\mathbf{b}}J_{HK}(\mathbf{w}_k, \mathbf{b}_k) - |\nabla_{\mathbf{b}}J_{HK}(\mathbf{w}_k, \mathbf{b}_k)|]$$

where $\nabla_{\mathbf{b}}J_{HK} = -2(\mathbf{X}\mathbf{w} - \mathbf{b}) = 0$

Let $\mathbf{e}_k = \mathbf{X}\mathbf{w}_k - \mathbf{b}_k = -\frac{1}{2}[\nabla_{\mathbf{b}}J_{HK}(\mathbf{w}_k, \mathbf{b}_k)]$

Then:

$$\begin{aligned}\mathbf{b}_{k+1} &= \mathbf{b}_k - \frac{1}{2}\rho_k[-2\mathbf{e}_k - |2\mathbf{e}_k|] \\ &= \mathbf{b}_k + \rho_k[\mathbf{e}_k + |\mathbf{e}_k|]\end{aligned}$$

54.4 Ho-Kashyap Algorithm

Ho-Kashyap Algorithm

(0) Start with arbitrary $\mathbf{b}_1 > 0$ and $\mathbf{w}_1$, let $k = 1$

Repeat steps (1) to (4):

(1) $\mathbf{e}_k = \mathbf{X}\mathbf{w}_k - \mathbf{b}_k$

(2) Solve for $\mathbf{b}_{k+1}$ using $\mathbf{w}_k$ and $\mathbf{b}_k$:

$$\mathbf{b}_{k+1} = \mathbf{b}_k + [\mathbf{e}_k + |\mathbf{e}_k|]$$

(3) Solve for $\mathbf{w}_{k+1}$ using $\mathbf{b}_{k+1}$:

$$\mathbf{w}_{k+1} = (\mathbf{w}^T \mathbf{w})^{-1} Y^T \mathbf{b}_{k+1}$$

(4) $k = k + 1$

until $\mathbf{e}_k = 0$ or $k > k_{max}$ or $\mathbf{b}_{k+1} = \mathbf{b}_k$ (for convergence fix learning rate as $0 < \rho_k < 1$)

54.5 Convergence Properties

Ho-Kashyap Convergence

In the linearly separable case:

- $\mathbf{e}_k = 0$, solution is found, STOP!
- One of the components of $\mathbf{e}_k$ is positive, algorithm continues

In non-separable case:

- $\mathbf{e}_k$ will have only negative components eventually, thus found proof of non-separability
- No bound on how many iterations is needed for the proof of non-separability

55 Summary: Linear Discriminant Functions

Comparison of Methods

Perceptron Procedure

- Find a separating hyperplane in the linearly separable case
- Do not converge in the non-separable case
- Can force convergence by using a decreasing learning rate, but there is no guarantee of a reasonable stopping point

MSE Procedure

- Converge in separable and non-separable cases
- May not find a separating hyperplane if classes are linearly separable
- Use pseudoinverse if $\mathbf{X}^T \mathbf{X}$ is not singular and not too large
- Use gradient descent (Widrow-Hoff procedure) otherwise

Ho-Kashyap Procedure

- Always converge
- Find a separating hyperplane in the linearly separable case
- More costly

Part VI

Linear Transformations and Dimensionality Reduction

56 Problems of High-Dimensional Data

In practical problems, training data can appear in various forms:

- Few samples
- Few samples and many features
- Fixed features that cannot be selected
- Not-independent features

There are two important things to consider when designing a classifier:

1. **Computational complexity** of the system
2. The **influence of the dimensionality** (the number of features) of the training set on the performance of the classifier

If the features are statistically **independent**, then it is more likely to achieve optimal performance. In general, if the performance is not at the desired level, one can add features, but this increases the complexity of both the classifier and the feature extractor.

If adding features leads to **worse** performance, then this could be because:

- Choosing an **incorrect model** (e.g., Gaussian hypothesis or conditional probability hypothesis)
- **Insufficient number of samples**

56.1 Computational Complexity

Complexity (in terms of running time) increases when the number of features, i.e., dimensionality d , increases. Many methods already have at least $O(nd^2)$ complexity, where n is the number of samples (e.g., covariance matrix calculation). As d increases, this cost may become prohibitive.

56.2 Curse of Dimensionality

Definition 56.1 (Curse of Dimensionality). If the dimensionality d is large and the number of samples n is too small, the problem of the **curse of dimensionality** appears: we cannot perform accurate parameter estimation, classification, etc.

For example, in the case of covariance matrix calculation, for accurate estimation, n should be much bigger than d^2 . Otherwise, the model would be too complicated for the data and **overfitting** occurs.

Example with k -NN ($k = 1$):

- Suppose we have 1 feature and 9 samples. The feature may not be discriminative.
- If we add a 2nd feature (same 9 samples), the feature space becomes too sparse for 1-NN to work well. To maintain the same density as in 1D, we would need $9^2 = 81$ samples.

- With 3 features, we would need $9^3 = 729$ samples!

Key Rule

If n samples is dense enough in 1D, then in d dimensions we need n^d samples to maintain the same density. The quantity n^d grows extremely fast as a function of d .

56.3 Overfitting in High Dimensions

Overfitting occurs when a model learns to **perform well on the training data** but **fails to generalize to unseen data**. When d is high, the data become increasingly sparse and spread out in the high-dimensional space. This can lead to overfitting because:

- The model may learn patterns that are specific to the training data
- The model has more freedom to fit the noise (e.g., irrelevant features) rather than the underlying patterns

For a fixed number of samples, as we add features, the training error keeps decreasing but the validation error eventually increases — this is overfitting.

Mitigating Overfitting and the Curse of Dimensionality

- **Reduce the dimensionality** of features (this lecture)
- Combine features: **feature extraction**
- Apply **feature selection**
- Apply **regularization**

It is important to **reduce the dependence** between features.

57 Dimensionality Reduction

Aim: Represent samples with fewer features, while preserving the structure of the data as much as possible.

- **Discriminative:** Keep only the structure that affects class separability
- We construct a new set of dimensions, e.g., by the **linear combination** of original dimensions
- Ideally the new feature vector should retain all important information from the old feature vector

Definition 57.1 (Dimensionality Reduction — Formal Definition). We look for a transformation $\mathbf{W}$ that maps the original features $\mathbf{y}$ into a new simpler, but more discriminant feature set $\mathbf{x}$ for classification:

$$\mathbf{x} = \mathbf{W} \mathbf{y}$$

where:

- $\mathbf{X}$ represents the new features with k data points ($k \times m$)
- $\mathbf{Y}$ represents the original features with k data points ($k \times n$)
- $n \gg m$

58 Principal Component Analysis (PCA)

PCA is perhaps the most frequently used method to reduce the dimensionality.

PCA — Core Idea

PCA defines a set of **principal components**:

1. **1st component**: Direction of the greatest variability (largest variance) in the data
2. **2nd component**: Perpendicular to 1st, greatest variability of what is left
3. ... and so on until the original dimensionality

Once the new feature representations (new dimensions) are obtained:

- Change the coordinates of every data point to these new dimensions, then apply the ML algorithm
- The new features have the same total variance as the old features. *PCA preserves the largest variances in the data.*

58.1 When to Apply PCA?

PCA is a good method to use if you answer “yes” to all three questions:

1. Do you want to reduce the number of variables, but aren’t able to identify variables to completely remove from consideration?
2. Do you want to ensure your variables are independent of one another?
3. Are you comfortable making your independent variables less interpretable?

If you answered “no” to question 3, you **should not** use PCA.

58.2 PCA: Direction of Largest Variance

What is the direction of largest variance in data?

- If $\mathbf{x}$ has multivariate distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, the direction of largest variance is given by the **eigenvector** corresponding to the **largest eigenvalue** of $\boldsymbol{\Sigma}$.
- Therefore, we need to look at the **covariance matrix** of the data.
- PCA can be applied to distributions also other than Gaussian.

58.3 PCA Algorithm

PCA Algorithm — Step by Step

1. **Standardize** the given data: subtract the mean of each feature from the data (the new data has zero mean).
2. **Calculate the covariance matrix** of the standardized data.
3. **Compute the eigenvectors and eigenvalues** of the covariance matrix.
4. **Sort** the eigenvalues in descending order and choose the top k eigenvectors as principal components.
5. **Project**: Multiply the standardized data matrix (from Step 1) by the selected principal components matrix. This yields the new features with k dimensions.

58.4 PCA: Mathematical Formulation

Given training data $\mathbf{x} = \{\mathbf{x}_1^T, \mathbf{x}_2^T, \dots, \mathbf{x}_M^T\}$ with M samples in n dimensions.

Mean:

$$\mathbf{m}_x = E\{\mathbf{x}\} = \frac{1}{M} \sum_{k=1}^M \mathbf{x}_k$$

Covariance matrix:

$$\mathbf{C}_x = E\{(\mathbf{x} - \mathbf{m}_x)(\mathbf{x} - \mathbf{m}_x)^T\} = \frac{1}{M} \sum_{k=1}^M (\mathbf{x}_k - \mathbf{m}_x)(\mathbf{x}_k - \mathbf{m}_x)^T = \frac{1}{M} \sum_{k=1}^M \mathbf{x}_k \mathbf{x}_k^T - \mathbf{m}_x \mathbf{m}_x^T$$

- $\mathbf{C}_x$ has size $n \times n$
- Diagonal elements c_{ii} are the **variance** of the i -th component
- Off-diagonal elements c_{ij} are the **covariances** between components i and j

Properties of $\mathbf{C}_x$:

- $\mathbf{C}_x$ is **real and symmetrical**
- If x_i and x_j are unrelated, then $c_{ij} = c_{ji} = 0$
- Since $\mathbf{C}_x$ is real and symmetrical, one can always find a set of n **orthonormal eigenvectors**

Let $\mathbf{e}_i$ and λ_i , $i = 1, 2, \dots, n$, be the eigenvectors and eigenvalues of $\mathbf{C}_x$, ordered in descending order: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$.

58.5 PCA: The Transformation

Let $\mathbf{A}$ be a matrix whose rows are the eigenvectors of $\mathbf{C}_x$ ordered as above. The transformation is:

$$\mathbf{y} = \mathbf{A}(\mathbf{x} - \mathbf{m}_x)$$

The resulting vectors $\mathbf{y}$ have:

- **Zero mean:** $\mathbf{m}_y = \mathbf{0}$
- **Diagonal covariance matrix:** $\mathbf{C}_y = \mathbf{A} \mathbf{C}_x \mathbf{A}^T = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & \lambda_n \end{bmatrix}$

The elements of $\mathbf{y}$ are therefore **decorrelated**.

Since the rows of $\mathbf{A}$ are orthonormal, $\mathbf{A}^{-1} = \mathbf{A}^T$, and we can reconstruct $\mathbf{x}$:

$$\mathbf{y} = \mathbf{A}(\mathbf{x} - \mathbf{m}_x) \Rightarrow \mathbf{x} = \mathbf{A}^T \mathbf{y} + \mathbf{m}_x$$

58.6 PCA: Dimensionality Reduction

Suppose we build $\mathbf{A}_k$ with only the first k (largest) eigenvectors, where $k < n$. Then:

- $\mathbf{A}_k$ is a matrix of dimension $k \times n$
- Vectors $\mathbf{y}$ have dimension k
- Reconstructed vectors $\hat{\mathbf{x}}$ will no longer be exact and are given by:

$$\hat{\mathbf{x}} = \mathbf{A}_k^T \mathbf{y} + \mathbf{m}_x$$

The smaller eigenvalues carry less significance; by discarding them we lose some information, but not much if the retained eigenvalues capture most of the total variance.

58.7 How Many Principal Components?

- Depends on the goal and the application
- No means of validating it, unless we use it in the context of a supervised method (e.g., reduce the input dimensionality for a supervised algorithm)
- We can compute the **cumulative proportion of eigenvalues** for the first k principal components to estimate the amount of information loss:

$$\text{Explained variance ratio} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^n \lambda_i}$$

As a threshold, one can use 90%, 95%, etc.

58.8 PCA Drawbacks

PCA Drawbacks

- It assumes that the variables are **linearly correlated**; in many cases, we already know that they are not
- Only the axes with the **greatest variance** are considered; there are some cases in which this involves huge mistakes (e.g., when variance does not correspond to discriminative power)
- PCA is **unsupervised**: it does not take class labels into account

59 Fisher Linear Discriminant (LDA)

Also called **Linear Discriminant Analysis (LDA)**.

FDA — Core Idea

Main idea: Find a projection to a line such that data samples from different classes are **well separated**.

Unlike PCA (which maximizes variance of the whole set), LDA maximizes the **distance between groups**.

59.1 Setup

Suppose we have 2 classes and d -dimensional samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ where:

- n_1 samples come from the first class ω_1
- n_2 samples come from the second class ω_2

Consider projection on a line with direction given by the unit vector $\mathbf{w}$. The scalar $\mathbf{w}^T \mathbf{x}_i$ is the projection of $\mathbf{x}_i$ onto the line (a one-dimensional subspace).

59.2 Separation Measure

We need to measure the separation between projections of different classes. Consider the class means **before** and **after** the transformation:

Before transformation (original space):

$$\mathbf{m}_1 = \frac{1}{N_1} \sum_{i=1}^{N_1} \mathbf{x}_i^{(1)}, \quad \mathbf{m}_2 = \frac{1}{N_2} \sum_{i=1}^{N_2} \mathbf{x}_i^{(2)}$$

After transformation (projected):

$$\tilde{m}_1 = \mathbf{w}^T \mathbf{m}_1, \quad \tilde{m}_2 = \mathbf{w}^T \mathbf{m}_2$$

A first idea: use $|\tilde{m}_1 - \tilde{m}_2|$ as a measure of separation. The larger this value, the better the expected separation.

Problem: $|\tilde{m}_1 - \tilde{m}_2|$ does not consider the **variance** (spread) of the classes! Two classes can have well-separated means but still heavily overlap if their variances are large.

59.3 Fisher's Criterion Function

We want the difference between the means of the two projected classes to be **large** compared to the **standard deviation** of each class. This leads to the Fisher criterion:

Fisher's Criterion

$$J(\mathbf{w}) = \frac{|\tilde{m}_1 - \tilde{m}_2|}{\tilde{s}_1^2 + \tilde{s}_2^2}$$

where $\tilde{s}_i^2$ is the **scatter** of class i after projection:

$$\tilde{s}_i^2 = \sum_{j=1}^{N_i} \left(y_j^{(i)} - \tilde{m}_i \right)^2$$

and $y_j^{(i)} = \mathbf{w}^T \mathbf{x}_j^{(i)}$ is the projected data point from class i .

Goal: Maximize $J(\mathbf{w})$, i.e.:

- **Numerator:** Projected means should be far apart
- **Denominator:** Scatter within each class should be as small as possible (samples cluster tightly around their projected mean)

59.4 Scatter Matrices

To express J as a function of $\mathbf{w}$, we define scatter matrices:

Within-class scatter matrix for class i :

$$S_i = \sum_{j=1}^{N_i} \left(\mathbf{x}_j^{(i)} - \mathbf{m}_i \right) \left(\mathbf{x}_j^{(i)} - \mathbf{m}_i \right)^T$$

Total within-class scatter matrix:

$$S_w = S_1 + S_2$$

Note. The scatter is just the sample variance multiplied by n . It measures the spread of data around the mean.

Between-class scatter matrix:

$$S_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$$

S_B measures the separation between the means of the two classes **before** projection.

59.5 Expressing $J(\mathbf{w})$ in Terms of Scatter Matrices

Using the identity $(\mathbf{w}^T \mathbf{v})^2 = \mathbf{w}^T \mathbf{v} \mathbf{v}^T \mathbf{w}$:

Denominator:

$$\begin{aligned}\tilde{s}_i^2 &= \sum_{j=1}^{N_i} \left(\mathbf{w}^T \mathbf{x}_j^{(i)} - \mathbf{w}^T \mathbf{m}_i \right)^2 = \mathbf{w}^T S_i \mathbf{w} \\ \tilde{s}_1^2 + \tilde{s}_2^2 &= \mathbf{w}^T S_1 \mathbf{w} + \mathbf{w}^T S_2 \mathbf{w} = \mathbf{w}^T S_w \mathbf{w}\end{aligned}$$

Numerator:

$$(\tilde{m}_1 - \tilde{m}_2)^2 = (\mathbf{w}^T \mathbf{m}_1 - \mathbf{w}^T \mathbf{m}_2)^2 = \mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w} = \mathbf{w}^T S_B \mathbf{w}$$

Therefore:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_w \mathbf{w}}$$

59.6 The Fisher Transform (Solution)

By taking the derivative $\frac{\partial J}{\partial \mathbf{w}} = 0$, we obtain the **Fisher transform**:

Fisher Transform

$$\mathbf{w} = S_w^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

This gives the optimal projection direction that maximizes the Fisher criterion $J(\mathbf{w})$.

59.7 FDA: Worked Example

Example 59.1. Class 1 has 5 samples: $\mathbf{x}_1 = \{(1, 2), (2, 3), (3, 3), (4, 5), (5, 5)\}$
Class 2 has 6 samples: $\mathbf{x}_2 = \{(1, 0), (2, 1), (3, 1), (3, 2), (5, 3), (6, 5)\}$

Step 1 — Compute class means:

$$\mathbf{m}_1 = [3, 3.6], \quad \mathbf{m}_2 = [3.3, 2]$$

Step 2 — Compute scatter matrices:

$$\begin{aligned}S_1 &= \begin{bmatrix} 10 & 8 \\ 8 & 7.2 \end{bmatrix}, & S_2 &= \begin{bmatrix} 17.3 & 16 \\ 16 & 16 \end{bmatrix} \\ S_w &= S_1 + S_2 = \begin{bmatrix} 27.3 & 24 \\ 24 & 23.2 \end{bmatrix}\end{aligned}$$

Step 3 — Compute inverse of S_w :

$$S_w^{-1} = \begin{bmatrix} 0.39 & -0.41 \\ -0.41 & 0.47 \end{bmatrix}$$

Step 4 — Compute $\mathbf{w}$:

$$\mathbf{w} = S_w^{-1}(\mathbf{m}_1 - \mathbf{m}_2) = [-0.79, 0.89]$$

Step 5 — Project data:

$$\mathbf{y}_1 = \mathbf{w}^T \mathbf{X}_1 = [-0.79 \ 0.89] \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 3 & 5 & 5 \end{bmatrix} = [0.99, 1.09, -0.70, 1.29, 0.50]$$

$$\mathbf{y}_2 = \mathbf{w}^T \mathbf{X}_2 = [-0.79 \ 0.89] \begin{bmatrix} 1 & 2 & 3 & 3 & 5 & 6 \\ 0 & 1 & 1 & 2 & 3 & 5 \end{bmatrix} = [-0.79, -0.69, -1.48, -0.59, -1.28, -0.29]$$

The two classes are now well separated in 1D.

59.8 FDA Drawbacks

FDA Drawbacks

- Reduces dimensionality only to $d - 1$ (for 2 classes, projects to a single line)
- For complex data, projection to even the best line may result in inseparable projected samples
- $J(\mathbf{w})$ is always 0 if $\mathbf{m}_1 = \mathbf{m}_2$ (e.g., concentric classes)
- When $J(\mathbf{w})$ is always large: classes have large overlap when projected to any line. Such cases are also problematic for PCA.

60 Summary: PCA vs. FDA

Benefits of Dimensionality Reduction

Dimensionality reduction with linear methods like PCA and FDA allows us to:

- Visualize the training set and then work on it
- Reduce the time and space complexity
- Remove multi-collinearity (can be helpful to handle overfitting)
- Remove irrelevant features from the data
- Handle the curse of dimensionality

Once dimensionality reduction is applied, we typically apply classification or other tasks using the new feature space. FDA is particularly suitable for classification tasks since it focuses on discrimination of classes.

Property	PCA	FDA (LDA)
Type	Unsupervised	Supervised
Objective	Maximize variance	Maximize class separability
Uses class labels	No	Yes
Max components	n (original dims)	$C - 1$ (C = number of classes)
Best for	General dim. reduction	Classification preprocessing

Part VII

Kernel-Based Learning and Support Vector Machines

61 Binary Classification Recap

Given training data $(\mathbf{x}_i, y_i)$ for $i = 1, \dots, N$ with $\mathbf{x}_i \in \mathbb{R}^n$ and $y_i \in \{+1, -1\}$, learn a classifier $f(\mathbf{x})$ such that it correctly predicts the class of unseen data.

62 Linear SVM

- Binary classification setting
- Assumes that data is linearly separable: there exists a hyperplane that divides the classes into positives and negatives

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad i \in \{1, \dots, m\}$$

where $\mathbf{w} \in \mathbb{R}^n$, $\mathbf{x}_i \in \mathbb{R}^n$, $b \in \mathbb{R}$, $y_i \in \{+1, -1\}$.

62.1 Large Margin Classifiers

Definition 62.1 (Margin). The **margin** of a classifier is the distance to the closest points of either class. Large margin classifiers attempt to maximize this distance.

SVMs maximize the margin $d = d_+ + d_-$, where d_+ and d_- are the distances from the separating hyperplane to the nearest positive and negative samples, respectively.

62.2 Support Vectors

Support Vectors

For any separating hyperplane, there exists a set of “closest points” called **support vectors**:

- For n dimensions, there will be at least $n + 1$ support vectors
- The samples that lie exactly on the two boundary hyperplanes H_1 and H_2 are the support vectors
- They are the **critical points** that determine the position and orientation of the separating hyperplane
- If you moved or removed any other points, the boundary wouldn't change, but moving a support vector would change it

Maximizing the margin is good since it implies that **only support vectors matter**; other training examples are ignorable.

62.3 Finding the Hyperplane

The margin is the distance to the support vectors on either side of the hyperplane. Computing the distance from the separating hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$ to one side of the margin (e.g., to $\mathbf{w}^T \mathbf{x} + b = 1$):

The distance from a point to the hyperplane is $\frac{1}{\|\mathbf{w}\|}$, so the full margin width is:

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

62.4 Maximizing the Margin

Select the hyperplane with the largest margin where all points are classified correctly and outside the margin. This is a **constrained optimization problem**.

Maximizing the margin $\frac{2}{\|\mathbf{w}\|}$ is equivalent to **minimizing** $\|\mathbf{w}\|$ (or equivalently $\frac{1}{2}\|\mathbf{w}\|^2$) subject to the separating constraints.

Hard Margin SVM Formulation

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2$$

subject to:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \dots, m$$

- The **minimization criterion** wants $\mathbf{w}$ to be as small as possible (large margin)
- The **constraint** ensures that the data is correctly separated

We minimize $\frac{1}{2}\|\mathbf{w}\|^2$ rather than $\|\mathbf{w}\|$ because the square is differentiable and easier to handle mathematically, while still achieving the same optimum.

63 Soft Margin SVM

63.1 Motivation

What if the dataset is **not linearly separable**? The hard margin quadratic optimization does not converge for non-linearly separable datasets. We need to allow some misclassifications.

63.2 Slack Variables

We modify the constraints by adding **slack variables** $\xi_i \geq 0$, which enable vectors to cross the margin:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$

The value of ξ_i indicates the position of the vector with respect to the hyperplane.

Soft Margin SVM Formulation

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

subject to:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i$$

where $C > 0$ is a **regularization parameter**.

63.3 The Role of C

- **Small C** (large margin): constraints can be easily ignored; more misclassifications tolerated; better generalization but possibly underfitting

- **Large C** (narrow margin): constraints are hard to ignore; fewer misclassifications; risk of overfitting
- $C = \infty$: enforces all constraints $\Rightarrow$ reduces to the **hard margin SVM**

C determines the **trade-off** between maximizing the margin and minimizing the classification error.

63.4 Understanding Slack Variables

Given the constraint $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$, equivalently $\xi_i \geq 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$:

Interpretation of Slack Variable Values

- $\xi_i = 0$: the data point is **correctly classified** and on or outside the margin
- $0 < \xi_i < 1$: **correctly classified** but inside the margin (between the hyperplane and the margin boundary)
- $\xi_i \geq 1$: **incorrectly classified** (misclassification error)

63.5 Hinge Loss

The soft margin SVM can also be expressed as an **unconstrained** optimization using the **hinge loss**:

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

The hinge loss $\max(0, 1 - y_i f(\mathbf{x}_i))$ penalizes points by how far they are from “correct” (i.e., from lying on the correct side of the margin).

63.6 Support Vectors in the Non-Linear (Soft Margin) Case

- Support vectors are all points for which a slack variable exists (i.e., $\xi_i > 0$, or points lying exactly on the margin)
- Points correctly classified and outside the margin ($\xi_i = 0$) are **not** support vectors
- Thus, support vectors lie **on the margin** (correctly classified) or **inside the margin / misclassified**

64 Non-Linear SVM

64.1 Motivation

Datasets that are linearly separable (with some noise) work well with linear SVMs. However, what if the dataset is too hard to separate with a hyperplane?

Key idea: Map data to a **higher-dimensional space** where the training set becomes linearly separable.

Non-Linear SVM — Core Idea

The original feature space can always be mapped to some higher-dimensional feature space where the training set is linearly separable.

A **linear classifier** in the higher-dimensional space corresponds to a **non-linear** boundary in the original space.

64.2 Example: Mapping to Higher Dimensions

Let $\mathbf{X} = (x, y, z)$ be a 3D feature space. We can map to a 9-dimensional space $\mathbf{Z}$:

$$\mathbf{Z} = (\phi_1(\mathbf{X}), \dots, \phi_9(\mathbf{X})) = (x, y, z, x^2, y^2, z^2, xy, xz, yz)$$

A linear classifier in $\mathbf{Z}$ corresponds to a **polynomial** classifier in the original space $\mathbf{X}$.

Problems with explicit mapping:

- The transformed space can have huge dimensionality (even infinite)
- Mappings are in general difficult to compute explicitly
- It is not clear how to find the proper mapping that will separate the data

64.3 The Kernel Trick

Instead of computing the transformation Φ explicitly, we use the **kernel trick**.

The Kernel Trick

The linear classifier relies on the inner product between vectors:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

If every datapoint is mapped into a high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

A **kernel function** is a function that is equivalent to an inner product in some feature space, allowing us to train the classifier **without ever explicitly computing** the mapping Φ .

Important: Since the classification algorithm is independent from the size of the target space, we **avoid the curse of dimensionality**.

64.4 Kernel Trick: Example

For 2-dimensional vectors $\mathbf{x} = [x_1, x_2]$, let $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$.

Expanding:

$$\begin{aligned} K(\mathbf{x}_i, \mathbf{x}_j) &= (1 + \mathbf{x}_i^T \mathbf{x}_j)^2 \\ &= 1 + x_{i1}^2 x_{j1}^2 + 2x_{i1}x_{j1}x_{i2}x_{j2} + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} \\ &= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) \end{aligned}$$

where:

$$\phi(\mathbf{x}) = [1, x_1^2, \sqrt{2}x_1x_2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2]$$

The kernel function implicitly maps data to a 6-dimensional space without computing $\phi(\mathbf{x})$ explicitly.

64.5 Standard Kernels

Kernel	Formula
Linear	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
Polynomial	$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^p$
RBF (Gaussian)	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\sigma^2}\right)$
Sigmoid	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(a \mathbf{x}_i^T \mathbf{x}_j + b)$

64.6 Kernel Selection

Kernel Selection Guidelines

Linear kernel:

- Used when the feature space is huge (e.g., text classification with word counts)
- Special case of the RBF kernel; no additional parameters

Polynomial kernel:

- Can have numerical difficulties (values approaching 0 or infinity)
- Good choice for well-known and well-conditioned tasks
- One additional parameter: degree p

RBF (Radial Basis Function) kernel:

- Generally indicated as the **best choice** in the literature
- One additional parameter: σ (or equivalently $\gamma = \frac{1}{2\sigma^2}$)

Sigmoid kernel:

- Two additional parameters (a and b); comes from neural networks
- Not much recommended in the literature

Note. Kernel selection and parameter tuning are critical. Lowering the degree for polynomial kernels or using larger σ for RBF kernels can help avoid overfitting. The number of support vectors is a measure of generalization performance.

65 Typical SVM Implementation

SVM Implementation Pipeline

1. **Apply scaling/normalization** on the data
2. **Consider the RBF kernel** as default
3. **Use cross-validation** to find the best parameters C and σ (or γ)
4. **Train the model** with the best C and σ using the whole training set
5. **Apply** the trained model on unseen data (test data)

Common problems and solutions:

- **Parameter search can be time-consuming** → conduct parameter search hierarchically
- **Search ranges are tricky to choose** → use exponentially growing values, e.g., $C = 2^{[-5, \dots, 15]}$ and $\gamma = 2^{[-15, \dots, 5]}$
- **RBF kernels can overfit** → consider high-degree polynomial kernels as alternative
- **Parameter search must be repeated for every feature set** → compare features on random subsets to contain computational cost

66 SVM Summary

SVM Key Concepts at a Glance

- **Hard margin SVM:** Finds the hyperplane with the maximum margin; requires linearly separable data
- **Soft margin SVM:** Introduces slack variables ξ_i and regularization parameter C to handle non-separable data
- **Support vectors:** The critical training points that lie on or within the margin; they alone determine the decision boundary
- **Kernel trick:** Allows SVMs to learn non-linear boundaries by implicitly mapping data to higher-dimensional spaces without computing the mapping explicitly
- **Key parameters:** C (regularization) and kernel parameters (σ , degree p , etc.)

Property	Hard Margin SVM	Soft Margin SVM
Linearly separable data required	Yes	No
Slack variables	No	Yes (ξ_i)
Regularization parameter	None	C
Allows misclassifications	No	Yes

SVM Facts:

- Originally proposed by Boser, Guyon and Vapnik in 1992
- Still among the best performers for several classification tasks (text, genomics, images)
- Can be applied to complex data types beyond feature vectors (graphs, sequences) by designing appropriate kernel functions
- Have been extended to regression, PCA, and other tasks

Part VIII

Clustering

67 Supervised vs. Unsupervised Learning

Up to now we considered **supervised learning** scenarios, where we are given:

- Samples $\mathbf{x}_1, \dots, \mathbf{x}_n$
- Class labels for all samples $\omega_1, \dots, \omega_n$

In **unsupervised learning**, and particularly in **clustering**, we have only:

- Samples $\mathbf{x}_1, \dots, \mathbf{x}_n$
- No class labels

67.1 Unsupervised Learning Approaches

- **Parametric approach:** Assume a parametric distribution of data and estimate its parameters. Much “harder” than supervised parametric approaches.
- **Non-parametric approach:** Group the data into clusters, each cluster hopefully corresponding to categories (classes) present in the data.

68 Clustering

Definition 68.1 (Clustering). Clustering is an area of unsupervised learning in which we find “clusters” or groups of similar points within the data **without having labels**.

Why Clustering?

- **Pattern recognition:** Discover inherent groupings, revealing relationships that might not be apparent through other means
- **No labels needed:** In many real-world scenarios, obtaining labeled data is time-consuming and expensive
- **Data preprocessing:** Segmenting data into meaningful groups makes subsequent tasks (classification, regression) easier

68.1 What Is Good Clustering?

Good Clustering Criteria

- **Internal** (within-cluster) distances should be **small** / similarities should be **high**
- **External** (inter-cluster) distances should be **large** / similarities should be **low**

We typically need:

- A **similarity measure** $s(\mathbf{x}_i, \mathbf{x}_k)$: large if $\mathbf{x}_i, \mathbf{x}_k$ are similar; or a **dissimilarity (distance) measure** $d(\mathbf{x}_i, \mathbf{x}_k)$: small if $\mathbf{x}_i, \mathbf{x}_k$ are similar
- A **criterion function** to evaluate a clustering
- An **algorithm** to compute the clustering (e.g., by optimizing the criterion function)

68.2 How Many Clusters?

- Some approaches **fix** the number of clusters (e.g., k -means requires k as input)
- Some find the best clustering according to a criterion function; therefore, the number of clusters may vary (e.g., DBSCAN)

69 Proximity / Distance Measures

The choice of distance function depends on the application (e.g., rotation invariance for object recognition, but not for character recognition where “6” and “9” must be distinguished).

69.1 Dissimilarity Measures

Euclidean distance (numerical attributes):

$$\text{Dist}(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_d (x_d - x'_d)^2}$$

- Translation invariant, symmetric, spherical, all dimensions equally treated
- Sensitive to large differences in single attributes

Manhattan (city block) distance:

$$\text{Dist}(\mathbf{x}, \mathbf{x}') = \sum_d |x_d - x'_d|$$

An approximation to Euclidean distance, cheaper to compute.

Hamming distance (categorical attributes):

$$\text{Dist}(\mathbf{x}, \mathbf{x}') = \sum_d \mathbf{1}_{x_d \neq x'_d}$$

Counts the number of attributes where $x_d \neq x'_d$.

69.2 Cosine Similarity

$$\text{CosSim}(\mathbf{x}, \mathbf{x}') = \frac{\mathbf{x}^T \mathbf{x}'}{\|\mathbf{x}\|_2 \|\mathbf{x}'\|_2}$$

- The smaller the angle between vectors, the larger the similarity
- **Scale invariant** measure (very popular, also in deep learning)
- Ranges from -1 (complete dissimilarity) to $+1$ (identical direction); 0 means orthogonal (no similarity)

70 Clustering Approaches

Taxonomy of Clustering Methods

1. **Hard vs. Soft Clustering:**
 - **Hard:** Each data point belongs to exactly one cluster (no overlap)
 - **Soft (Fuzzy):** Data points can belong to more than one cluster with varying degrees of membership
2. **Partitioning Clustering:** Data divided into non-overlapping subsets (e.g., k -means, k -medoids)
3. **Hierarchical Clustering:** Creates a tree of clusters (dendrogram); agglomerative (bottom-up) or divisive (top-down)
4. **Density-Based Clustering:** Clusters are dense regions separated by regions of lower density (e.g., DBSCAN)
5. **Centroid-Based Clustering:** Clusters represented by a central vector (centroid); points assigned to the nearest centroid (e.g., k -means)
6. **Distribution-Based Clustering:** Clusters modeled using statistical distributions (e.g., Gaussian Mixture Models)
7. **Graph-Based Clustering:** Data points as nodes in a graph; clusters as connected components (e.g., spectral clustering)
8. **Subspace Clustering:** Identifies clusters in subspaces considering only a subset of features per cluster; useful for high-dimensional data
9. **Self-Organizing Maps (SOM):** Neural network-based clustering that projects high-dimensional data onto a lower-dimensional grid

71 K-Means Clustering

- The most well-known and popular clustering algorithm
- We must know **a priori** how many clusters we need (k)
- Works with numerical attributes, but not with categorical ones
- **Fast:** $O(\#iterations \times k \times n \times d)$
- K -means is a **greedy algorithm** that converges to a **local minimum**
- Results can vary drastically based on centroid initialization

71.1 Problem Formulation

Given a data set $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ of N observations of a D -dimensional variable $\mathbf{x}$, the goal is to partition the data set into K clusters.

For each data point $\mathbf{x}_n$, a set of binary indicator variables $r_{nk} \in \{0, 1\}$, $k = 1, \dots, K$, describes which cluster $\mathbf{x}_n$ is assigned to (**hard clustering**).

Objective function:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

where $\boldsymbol{\mu}_k$ is the centroid of cluster k . The goal is to find r_{nk} and $\boldsymbol{\mu}_k$ that **minimize** J .

71.2 K-Means Algorithm

K-Means Algorithm

Initialization: Randomly select K centroids μ_k , $k = 1, \dots, K$.

Repeat:

1. **Assignment step** (minimize J w.r.t. r_{nk} , keeping μ_k fixed):

For each data point $\mathbf{x}_n$, assign it to the nearest centroid:

$$r_{nk} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_n - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases}$$

2. **Update step** (minimize J w.r.t. μ_k , keeping r_{nk} fixed):

Recompute each centroid as the mean of assigned points:

$$\mu_k = \frac{\sum_n r_{nk} \mathbf{x}_n}{\sum_n r_{nk}} = \frac{1}{|C_k|} \sum_{\mathbf{x}_n \in C_k} \mathbf{x}_n$$

Termination: Stop when there is no further change in assignments or after a maximum number of iterations.

71.3 Initialization Strategies

Results can vary drastically based on centroid selection. Common heuristics:

- Choose random points (not necessarily data examples)
- Choose data points least similar to any existing center (**furthest centers heuristic**)
- Try out **multiple starting points** (run k -means several times)
- Initialize cluster centers with results from another clustering method

71.4 Choosing the Number of Clusters

- **Domain knowledge:** e.g., digit clustering $\Rightarrow K = 10$
- **Data visualization** (for low-dimensional data)
- Run k -means with different k values and evaluate with metrics

Elbow Rule: Plot the **Within-Cluster Sum of Squares** (WCSS) as a function of k :

$$\text{WCSS} = \sum_{i=1}^k \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \mathbf{c}_i\|^2$$

The “elbow” point is the value of k where adding more clusters provides **diminishing returns** in reducing WCSS. It represents a balance between model simplicity and capturing meaningful patterns.

71.5 K -Means Drawbacks

K -Means Drawbacks

- Does not perform well when clusters are **not spherical** and/or have varying sizes
- Struggles with clusters of **different densities**
- **Outliers** can significantly influence centroid calculation, leading to suboptimal assignments
- Converges to a **local minimum** (not guaranteed to find global optimum)
- Number of clusters k must be specified **a priori**

72 Hierarchical Clustering

K -means is a **flat** clustering method. For some data, **hierarchical clustering** is more appropriate, e.g., when there is a natural taxonomy in the data.

72.1 Dendrograms

Hierarchical clustering is represented through a **dendrogram** (binary tree-like diagram) that shows the similarities of grouped clusters:

- Level k corresponds to a partitioning with $n - k + 1$ clusters
- If you need k clusters, take the clustering from level $n - k + 1$
- If samples are in the same cluster at level k , they remain in the same cluster at higher levels

72.2 Agglomerative vs. Divisive

- **Agglomerative (bottom-up)**: Start with each point as a singleton cluster and iteratively merge the two closest clusters
- **Divisive (top-down)**: Start with all data points in one cluster and iteratively split into smaller groups

72.3 Agglomerative Hierarchical Clustering

Agglomerative Algorithm

1. Initialize n clusters, each containing one data point
2. Compute the pairwise distance matrix D between all data points
3. **For** $k = 1$ to $n - 1$:
 - (a) Identify the two closest clusters C_i and C_j based on D
 - (b) Merge C_i and C_j into a single cluster C_{ij}
 - (c) Update D to reflect distances between C_{ij} and remaining clusters using the chosen **linkage criterion**
4. Continue until the desired number of clusters c is reached

72.4 Linkage Criteria

Four ways to calculate cluster distance between clusters D_i and D_j :

Single linkage (nearest neighbor)	$d_{\min}(D_i, D_j) = \min_{\mathbf{x} \in D_i, \mathbf{x}' \in D_j} \ \mathbf{x} - \mathbf{x}'\ $
Complete linkage (farthest neighbor)	$d_{\max}(D_i, D_j) = \max_{\mathbf{x} \in D_i, \mathbf{x}' \in D_j} \ \mathbf{x} - \mathbf{x}'\ $
Average linkage	$d_{\text{avg}}(D_i, D_j) = \frac{1}{n_i n_j} \sum_{\mathbf{x} \in D_i} \sum_{\mathbf{x}' \in D_j} \ \mathbf{x} - \mathbf{x}'\ $
Mean linkage (centroid)	$d_{\text{mean}}(D_i, D_j) = \ \mathbf{m}_i - \mathbf{m}_j\ $

Properties:

- **Single linkage:** Generates minimum spanning tree; encourages elongated clusters; very sensitive to noise/outliers
- **Complete linkage:** Results in compact clusters; does not work well with elongated clusters
- **Average / Mean linkage:** More robust; mean distance is cheaper to compute

All linkage methods behave similarly when clusters are hyperspherical and well separated.

72.5 Divisive Hierarchical Clustering

1. Split data into clusters using any flat-clustering method (e.g., k -means)
2. Choose the cluster with the **largest Sum of Squared Error** (SSE) to split further:

$$\text{SSE}_k = \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \mathbf{c}_i\|^2$$

3. Repeat until a single cluster is formed (or the desired number of clusters is reached)

72.6 Hierarchical Clustering: Limitations

- **High computational complexity:** Computing the proximity matrix is $O(N^2)$; total time complexity is $O(N^3)$
- There is **no objective function** being optimized
- Cannot be used for very large datasets due to complexity
- Sensitive to noise and outliers
- Difficulty handling large clusters

73 DBSCAN

Definition 73.1 (DBSCAN). **Density-Based Spatial Clustering of Applications with Noise** is a density-based clustering algorithm that finds clusters by grouping data points that are closely packed together, based on a specified distance and density threshold. It is particularly useful for identifying clusters of **arbitrary shapes** and for detecting **noise (outliers)**.

73.1 Parameters and Point Types

Two parameters:

- ε (**eps**): The radius defining the neighborhood around a data point

- **MinPts:** The minimum number of data points required to form a dense region (core point)

Three Types of Points

- **Core points:** Have at least MinPts points within radius ε
- **Border points:** Within the ε -neighborhood of a core point, but do not themselves have enough neighbors to be a core point
- **Noise points (outliers):** Neither core points nor border points

73.2 DBSCAN Algorithm

DBSCAN Algorithm

1. For each unvisited data point, determine its neighborhood based on ε
 2. If the number of neighbors $\geq$ MinPts, create a new cluster and add the point and its neighbors
 3. **Expand** the cluster by recursively checking the neighbors' neighbors (following the same MinPts criterion)
 4. Repeat for all unvisited data points
- Points not assigned to any cluster are labeled as **noise**.

73.3 DBSCAN: Advantages and Limitations

Advantages	Limitations
Can find clusters of arbitrary shapes	Sensitive to the choice of ε and MinPts
Can identify outliers/anomalies	Struggles with clusters of varying densities
No need to specify the number of clusters a priori	Performance may degrade with high-dimensional data
	Performance degrades as dataset size increases

74 Clustering Evaluation

74.1 Extrinsic Measures

Extrinsic measures evaluate clustering by how well it helps solve another problem (e.g., classification).

Purity: The proportion of the dominant class in each cluster.

- Good for **comparing** two algorithms, but not for understanding how well a single algorithm is doing
- **Caveat:** Increasing the number of clusters always increases purity (trivially, n clusters $\Rightarrow$ purity = 1)

74.2 Intrinsic Measures

Intrinsic measures evaluate clustering quality based on internal structure alone.

74.2.1 Davies-Bouldin Index (DBI)

Measures the compactness and separation between clusters:

$$\text{DBI} = \frac{1}{n} \sum_{i=1}^n \max_{j \neq i} \frac{\sigma_i + \sigma_j}{d(\mathbf{c}_i, \mathbf{c}_j)}$$

where σ_i is the average distance between points in cluster C_i and its centroid $\mathbf{c}_i$, and $d(\mathbf{c}_i, \mathbf{c}_j)$ is the distance between centroids.

- **Lower DBI** $\Rightarrow$ better clustering (more compact and well-separated)
- **Higher DBI** $\Rightarrow$ poorer clustering (more overlap or spread out)

74.2.2 Silhouette Index

Measures how well-separated clusters are and how similar each point is to its own cluster vs. other clusters.

For each data point i :

- $a(i)$: Average distance from point i to other points in the **same** cluster (intra-cluster cohesion)
- $b(i)$: Smallest average distance from point i to points in a **different** cluster, minimized over all other clusters (inter-cluster separation)

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

The overall silhouette score is the average over all data points:

$$\bar{s} = \frac{1}{N} \sum_{i=1}^N s(i)$$

- Ranges from -1 to $+1$
- **High value**: point is well-matched to its own cluster and poorly matched to neighbors
- **Low or negative value**: clustering may have too many or too few clusters

74.2.3 Dunn Index

Measures compactness (intra-cluster) and separation (inter-cluster):

$$\text{Dunn} = \frac{\min_{i \neq j} d(\mathbf{c}_i, \mathbf{c}_j)}{\max_k \max_{\mathbf{x}, \mathbf{y} \in C_k} d(\mathbf{x}, \mathbf{y})}$$

A **higher** Dunn index indicates better clustering (large inter-cluster distance relative to intra-cluster spread).

Clustering Evaluation Summary

Metric	Type	Better when
Purity	Extrinsic	Higher
Davies-Bouldin Index	Intrinsic	Lower
Silhouette Index	Intrinsic	Higher (close to +1)
Dunn Index	Intrinsic	Higher

Part IX

Parametric Unsupervised Learning

75 Parametric vs Non-Parametric Unsupervised Learning

Unsupervised learning can be divided into two main paradigms:

- **Non-parametric approaches** (e.g., clustering): no assumptions are made about the underlying data densities. Instead, we seek a partition of the data into clusters based on similarity measures.
- **Parametric approaches**: we *assume* the data was generated by a model with a known functional form but unknown parameters. The goal is to estimate these parameters from the observed data.

Definition 75.1 (Parametric Unsupervised Learning). In parametric unsupervised learning, we model the underlying class-conditional densities with a **mixture of parametric densities**. The objective is to find the model parameters that best explain the observed data, without knowing the class labels.

75.1 Advantages of Having a Parametric Model

Why Use a Parametric Model?

- We can adjust the parameters to **maximise the probability** that the model produced the observed data (maximum likelihood).
- We can **compute the likelihood** of new data under the model.
- We can **compare algorithms**: whichever model gives the highest likelihood for the observed data is preferred.

75.2 Comparison with Supervised Learning

In **parametric supervised learning**:

- Given m classes with samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ each labelled as class $1, 2, \dots, m$.
- $\mathcal{D}_i$ contains samples from class i .
- The probability distribution for class i is $p_i(\mathbf{x} \mid \boldsymbol{\theta}_i)$.
- Use Maximum Likelihood to estimate parameters $\boldsymbol{\theta}_i$.

In **parametric unsupervised learning**:

- We still know we have m classes.
- We have samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ each of **unknown class**.
- The probability distribution for class i is still $p_i(\mathbf{x} \mid \boldsymbol{\theta}_i)$.
- The task is to determine both the **class assignments** and the **parameters simultaneously**.

Note. **Example application — MRI brain segmentation.** Different brain tissues have different intensity distributions. There are 6 major tissue types, each modelled by a Gaussian distribution with unknown mean and standard deviation. The task is to simultaneously segment

(classify) each pixel *and* estimate the Gaussian parameters, without knowing the ground-truth labels.

76 Gaussian Mixture Models (GMM)

Definition 76.1 (Gaussian Mixture Model). A Gaussian Mixture Model with K components is a probabilistic model that represents the data distribution as a weighted sum of K Gaussian densities:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

where π_k are the **mixing coefficients**, $\boldsymbol{\mu}_k$ are the **means**, and $\boldsymbol{\Sigma}_k$ are the **covariance matrices** of each component.

76.1 Latent Variables

GMM introduces a **latent variable** $\mathbf{z}$ (not observed, but inferred) to indicate which Gaussian component a data point belongs to.

Latent Variable $\mathbf{z}$

- $\mathbf{z} = (z_1, \dots, z_K)$ is a **binary one-hot vector**: $z_k \in \{0, 1\}$ and $\sum_k z_k = 1$.
- Only one component of $\mathbf{z}$ is active (equal to 1) at a time.
- There are K possible states of $\mathbf{z}$, one for each component.
- Example: $\mathbf{z} = [0, 0, 1, 0, 0]$ indicates the data point belongs to the 3rd Gaussian component.

76.2 GMM Parameters

The aim is to find the maximum likelihood parameters:

- π_k : mixing coefficients (prior probability that a point belongs to component k).
- $\boldsymbol{\mu}_k$: mean of each Gaussian component.
- $\boldsymbol{\Sigma}_k$: covariance matrix of each Gaussian component.

76.3 Joint and Marginal Distributions

76.3.1 Prior Distribution $p(\mathbf{z})$

Each component z_k is assigned a prior probability:

$$p(z_k = 1) = \pi_k, \quad \text{where } 0 \leq \pi_k \leq 1 \quad \text{and} \quad \sum_{k=1}^K \pi_k = 1$$

Because $\mathbf{z}$ uses a 1-of- K (one-hot) encoding:

$$p(\mathbf{z}) = \prod_{k=1}^K \pi_k^{z_k}$$

Since $z_k \in \{0, 1\}$ and the components are mutually exclusive, only the term corresponding to the active component contributes.

76.3.2 Conditional Distribution $p(\mathbf{x} \mid \mathbf{z})$

For a particular component (value of $\mathbf{z}$):

$$p(\mathbf{x} \mid z_k = 1) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Thus $p(\mathbf{x} \mid \mathbf{z})$ can be written as:

$$p(\mathbf{x} \mid \mathbf{z}) = \prod_{k=1}^K \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}$$

Using the one-hot property of $\mathbf{z}$, only the term corresponding to the active component ($z_k = 1$) contributes to the product.

76.3.3 Joint Distribution $p(\mathbf{x}, \mathbf{z})$

The joint distribution factorises as:

$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z})$$

76.3.4 Marginal Distribution $p(\mathbf{x})$

By the law of total probability, the marginal is obtained by summing over all possible states of $\mathbf{z}$:

$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

This is the standard form of a Gaussian Mixture.

Graphical Model for GMM

The relationship between the latent variable $\mathbf{z}$ and the observed variable $\mathbf{x}$ can be represented as a directed graphical model:

$$\mathbf{z} \longrightarrow \mathbf{x}$$

- $\mathbf{z}$: latent variable representing the subclass (component membership).
- $\mathbf{x}$: observed variable (data point).

For N observations $\mathbf{x}_1, \dots, \mathbf{x}_N$, every observed data point $\mathbf{x}_n$ has a corresponding latent vector $\mathbf{z}_n$ indicating its sub-class.

76.4 Responsibilities

Definition 76.2 (Responsibility). The **responsibility** $\gamma(z_{nk})$ is the posterior probability that data point $\mathbf{x}_n$ belongs to component k :

$$\gamma(z_{nk}) \equiv p(z_k = 1 \mid \mathbf{x}_n) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

obtained via **Bayes' theorem**.

- π_k can be viewed as the **prior probability** of $z_k = 1$.
- $\gamma(z_{nk})$ can be interpreted as the responsibility that component k takes for “explaining” the observation $\mathbf{x}_n$.

76.5 Generating Synthetic Data from a GMM

There are two ways to generate data from a GMM:

1. **Ancestral sampling**: generate a value $\mathbf{z}^*$ from $p(\mathbf{z})$ (i.e., select a component), then generate $\mathbf{x}$ from $p(\mathbf{x} | \mathbf{z}^*)$.
2. **Marginal sampling**: generate directly from $p(\mathbf{x})$ by ignoring the values of $\mathbf{z}$.

Synthetic data generation is useful for evaluating model performance, diagnosing issues, and fine-tuning parameters.

77 Maximum Likelihood for GMM

77.1 Data Representation

Given a dataset $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ of N items each of dimension D :

- $\mathbf{X}$ is an $N \times D$ matrix, where the n -th row is $\mathbf{x}_n^\top$.
- $\mathbf{Z}$ is an $N \times K$ matrix of latent variables, where the n -th row is $\mathbf{z}_n^\top$.

The goal is to estimate $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$, and π_k by maximising the likelihood of observing the dataset $\mathbf{X}$.

77.2 Likelihood and Log-Likelihood

The joint likelihood for the dataset $\mathbf{X}$ (assuming i.i.d. samples) is:

$$p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \prod_{n=1}^N p(\mathbf{x}_n) = \prod_{n=1}^N \left[\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right]$$

The **log-likelihood** function is:

$$\ln p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

Why No Closed-Form Solution?

The log-likelihood has **no closed-form solution** because:

- The summation over K components appears *inside* the logarithm.
- The logarithm does not directly operate on the individual Gaussian terms.
- Taking derivatives with respect to $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$, or π_k leads to coupled, non-linear equations.

While gradient-based optimisation is possible, the standard approach is the iterative **EM algorithm**.

78 Technical Issues

Before applying the EM algorithm, two important technical problems must be considered.

78.1 Singularity Problem

Singularity Problem

Consider a GMM with isotropic covariance matrices $\Sigma_k = \sigma_k^2 \mathbf{I}$.

If a data point $\mathbf{x}_n$ falls exactly on the mean μ_j of component j , then $\exp\left(-\frac{(\mathbf{x}_n - \mu_j)^2}{2\sigma_j^2}\right) = 1$, and the likelihood contribution becomes:

$$\mathcal{N}(\mathbf{x}_n | \mu_j, \sigma_j^2 \mathbf{I}) = \frac{1}{(2\pi)^{D/2} |\Sigma_j|^{1/2}}$$

As $\sigma_j \rightarrow 0$, this term goes to **infinity**. The log-likelihood becomes unbounded, making maximisation **ill-posed**.

Solution: if a component collapses ($\sigma_k \rightarrow 0$), reset its mean or covariance to a small but non-zero value.

78.2 Identifiability Problem

Identifiability Problem

A density $p(\mathbf{x} | \theta)$ is **identifiable** if $\theta \neq \theta'$ implies there exists an $\mathbf{x}$ for which $p(\mathbf{x} | \theta) \neq p(\mathbf{x} | \theta')$.

A K -component mixture has $K!$ **equivalent solutions**, corresponding to the $K!$ ways of assigning K sets of parameters to K components.

- For $K = 3$: $K! = 6$ equivalent permutations (123, 132, 213, 231, 312, 321).
- For any point in parameter space, there are $K! - 1$ additional points giving the same distribution.

However, any of the equivalent solutions is equally valid.

Solution: initialise GMM parameters using **K-means clustering**, providing a better and more stable starting point for EM.

79 Expectation-Maximisation (EM) Algorithm for GMM

Definition 79.1 (EM Algorithm). The Expectation-Maximisation (EM) algorithm is an iterative method for finding maximum likelihood solutions for models with **latent variables**. It alternates between:

- **E-step:** compute posterior probabilities (responsibilities) using current parameters.
- **M-step:** re-estimate the parameters using these responsibilities.

79.1 Deriving the Parameter Updates

We want to find π_k, μ_k, Σ_k that maximise the log-likelihood. Taking derivatives one by one:

79.1.1 Update for μ_k

Taking the derivative of $\ln p(\mathbf{X} | \pi, \mu, \Sigma)$ w.r.t. μ_k and setting to zero, we obtain:

Mean Update

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n$$

The mean of the k -th component is the **weighted mean of all data points**, where each point $\mathbf{x}_n$ is weighted by the posterior probability (responsibility) $\gamma(z_{nk})$ that component k generated $\mathbf{x}_n$.

79.1.2 Update for $\boldsymbol{\Sigma}_k$

Taking the derivative w.r.t. $\boldsymbol{\Sigma}_k$ and setting to zero:

Covariance Update

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

Each data point is weighted by the corresponding posterior probability. The denominator N_k is the same effective number of points as in the mean update.

79.1.3 Update for π_k

Maximising w.r.t. π_k subject to the constraint $\sum_k \pi_k = 1$ (using a **Lagrange multiplier**):

Mixing Coefficient Update

$$\pi_k = \frac{N_k}{N}$$

The mixing coefficient for component k is the fraction of the effective number of points assigned to that component.

79.1.4 Effective Number of Points N_k

In all three updates above, N_k denotes:

$$N_k = \sum_{n=1}^N \gamma(z_{nk})$$

This represents the **effective number of data points** assigned to cluster k (a soft count based on responsibilities).

Note. All three parameter estimates depend on the responsibilities $\gamma(z_{nk})$, which in turn depend on the parameters. Therefore, these are **not closed-form solutions** — they must be solved **iteratively** via the EM algorithm.

79.2 The EM Algorithm — Step by Step

EM Algorithm for GMM

1. **Initialisation:** set initial values for π_k , $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$ and evaluate the initial log-likelihood.
2. **E-step:** evaluate responsibilities using the current parameter values:

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

3. **M-step:** re-estimate parameters using current responsibilities:

$$\begin{aligned} \boldsymbol{\mu}_k^{\text{new}} &= \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \\ \boldsymbol{\Sigma}_k^{\text{new}} &= \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^\top \\ \pi_k^{\text{new}} &= \frac{N_k}{N} \end{aligned}$$

where $N_k = \sum_{n=1}^N \gamma(z_{nk})$.

4. **Convergence check:** evaluate the log-likelihood

$$\ln p(\mathbf{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

Check convergence of either the parameters or the log-likelihood. If not converged, go to Step 2.

79.3 Visualising EM Iterations

The EM algorithm proceeds iteratively:

1. Start with data points and an initial mixture model (random or K-means initialised).
2. **Initial E-step:** determine responsibilities (soft assignments of points to components).
3. **First M-step:** re-evaluate parameters — the Gaussian components shift and reshape.
4. After 2, 5, 20 cycles, the model converges to a stable solution.

80 Properties of EM for GMM

Key Properties of EM

- EM takes **many more iterations** than K-means, and each cycle requires significantly more computation.
- It is common to run **K-means first** to find a suitable initialisation for the means.
- Covariance matrices can be initialised to the covariances of clusters found by K-means.
- EM is **not guaranteed** to find the global maximum of the log-likelihood — it can converge to local maxima.
- The log-likelihood is guaranteed to be **non-decreasing** at each iteration.

81 GMM Summary

GMM Summary

- GMM provides a **probabilistic view** of unsupervised learning (and clustering).
- Each cluster corresponds to a different Gaussian component.
- The model uses **latent variables** to represent component membership.
- It is a **general approach**: Gaussians can be replaced with other distributions (continuous or discrete).
- Mixture models are very powerful — they are **universal approximators**.
- Parameter optimisation is performed using the **EM algorithm**.

Aspect	K-Means	GMM + EM
Assignment	Hard (each point to one cluster)	Soft (probabilities)
Cluster shape	Spherical (Voronoi cells)	Arbitrary (full covariance)
Output	Cluster labels	Posterior probabilities
Parameters	Centroids only	μ_k, Σ_k, π_k
Convergence	Fast	Slower (more computation/iter)
Likelihood	Not modelled	Explicit probabilistic model
Initialisation	Random or K-means++	K-means recommended

Part X

Hidden Markov Models

82 Sequential Data

Many real-world problems involve **sequential data** — observations ordered in time where the order carries important information.

- **Speech recognition:** acoustic features at successive time frames.
- **Natural language processing:** sequences of characters, words, or sentences.
- **Weather forecasting:** data of successive days.
- **Financial series:** daily values of currency exchange rates.
- **Biomedical data:** EEG, ECG, and other time-series signals.
- **Gesture recognition:** body pose sequences over time.

83 Markov Models (Markov Chains)

Definition 83.1 (Markov Chain). A **Markov chain** is a stochastic model describing a sequence of possible events in which the probability of each event depends **only on the state attained in the previous event**. It is a process in time where the state at time t is influenced only by the state at time $t - 1$.

The key motivation is that it is impractical to consider a general dependence of future observations on *all* previous observations, since complexity grows with the number of observations. Markov models restrict dependence to only the most recent observations.

83.1 Independence Model (Baseline)

The simplest model assumes all observations are **independent**:

- Graph without links between states.
- Example: predicting rain for tomorrow based only on the relative frequency of rainy days, ignoring whether it rained today.

83.2 First-Order Markov Model

First-Order Markov Property

The distribution of the next state depends **only on the current state**:

$$P(q_{t+1} = S_j \mid q_t = S_i, q_{t-1} = S_k, \dots, q_1 = S_l) = P(q_{t+1} = S_j \mid q_t = S_i)$$

If the model is used to predict the next observation, the distribution of the prediction depends only on the preceding observation and is independent of all earlier observations.

83.3 Markov Model Ingredients

A Markov model is characterised by:

1. **State set:** $\mathcal{S} = \{S_1, S_2, \dots, S_N\}$ — a set of N states.
2. **State sequence:** $\mathcal{Q} = \{q_1, q_2, \dots, q_T\}$ — at each discrete time step t , the process is in exactly one state $q_t \in \mathcal{S}$.
3. **Transition probability matrix:** $\mathbf{A} = \{a_{ij}\}$ where

$$a_{ij} = P(q_t = S_j \mid q_{t-1} = S_i), \quad \sum_{j=1}^N a_{ij} = 1 \quad \forall i$$

4. **Initial state distribution:** $\boldsymbol{\Pi} = \{\pi_i\}$ where

$$\pi_i = P(q_1 = S_i), \quad \sum_{i=1}^N \pi_i = 1$$

Example 83.1. Consider a weather model with states {sunny, rainy, snowy}:

$$\mathbf{A} = \begin{pmatrix} 0.8 & 0.15 & 0.05 \\ 0.38 & 0.6 & 0.02 \\ 0.75 & 0.05 & 0.2 \end{pmatrix}, \quad \boldsymbol{\Pi} = (0.7, 0.25, 0.05)$$

The probability of the sequence *Sunny* $\rightarrow$ *Rainy* $\rightarrow$ *Rainy* $\rightarrow$ *Rainy* $\rightarrow$ *Snowy* $\rightarrow$ *Snowy*:

$$\begin{aligned} P &= P(\text{Su}) \cdot P(\text{R} \mid \text{Su}) \cdot P(\text{R} \mid \text{R}) \cdot P(\text{R} \mid \text{R}) \cdot P(\text{Sn} \mid \text{R}) \cdot P(\text{Sn} \mid \text{Sn}) \\ &= 0.7 \times 0.15 \times 0.6 \times 0.6 \times 0.02 \times 0.2 = 0.0001512 \end{aligned}$$

83.4 Higher-Order Markov Models

- **2nd order:** each observation depends on the previous *two* observations.
- **M -th order:** the conditional distribution depends on the previous M variables.
- **Number of parameters:**
 - First order: $K(K-1)$ parameters (where K is the number of states).
 - M -th order: $K^{M-1}(K-1)$ parameters — grows exponentially with M .

84 Hidden Markov Models (HMM)

Markov models are computationally tractable but limited: the states are directly observed. To build a more general framework, we introduce **latent variables**.

Definition 84.1 (Hidden Markov Model). When the latent variables are discrete, the model is called a **Hidden Markov Model (HMM)**. Each hidden state is associated with a probability function describing the probability that a certain output (observation symbol) is emitted by that state. The states are *not* directly observed — only the emissions are.

84.1 Formal Definition

An HMM is denoted by the triple $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$:

HMM Components

1. **Hidden states:** $\mathcal{S} = \{S_1, S_2, \dots, S_N\}$.
2. **Transition matrix:** $\mathbf{A} = \{a_{ij}\}$ where $a_{ij} = P(q_t = S_j \mid q_{t-1} = S_i)$.
3. **Initial distribution:** $\boldsymbol{\pi} = \{\pi_i\}$ where $\pi_i = P(q_1 = S_i)$.
4. **Observation symbols:** $\mathcal{V} = \{v_1, v_2, \dots, v_M\}$.
5. **Emission probability matrix:** $\mathbf{B} = \{b_{jk}\}$ where $b_{jk} = P(o_t = v_k \mid q_t = S_j)$, i.e., the probability of emitting symbol v_k when in state S_j .

Example 84.1. Weather–ice cream HMM. A climatologist in 2799 wants to reconstruct the weather in Verona during the summer of 2008. The only record is a diary listing how many ice creams someone ate each day.

- **Hidden states:** {Hot, Cold} (the unknown weather).
- **Observations:** {1, 2, 3} (number of ice creams eaten — the known data).
- **Task:** given the ice cream observation sequence, determine the most likely weather sequence.

84.2 Conditional Independence in HMMs

The graphical model structure of an HMM implies important **conditional independence** properties:

$$\begin{array}{ccccccc}
 s_1 & \rightarrow & s_2 & \rightarrow & s_3 & \rightarrow & s_4 \rightarrow \dots \\
 & & \downarrow & & \downarrow & & \downarrow & & \downarrow \\
 & & O_1 & & O_2 & & O_3 & & O_4
 \end{array}$$

- O_t is conditionally independent of everything else given s_t .
- s_{t+1} is conditionally independent of everything else given s_t .

84.3 Types of HMM Structure

- **Fully connected (ergodic):** every state can transition to every other state.
- **Left-to-right:** restrictions on $\mathbf{A}$ such that $a_{jk} = 0$ if $k < j$. Once a state is vacated, it cannot be re-entered. Common in speech recognition.

85 The Three Key Tasks of HMMs

Three Fundamental HMM Problems

1. **Task 1 — Evaluation (Likelihood):** given λ and observation sequence $O = (O_1, \dots, O_T)$, compute $P(O \mid \lambda)$.
 - Solved by the **Forward algorithm**.
2. **Task 2 — Decoding:** given O and λ , find the most likely hidden state sequence $S_1, \dots, S_T$.
 - Solved by the **Viterbi algorithm**.
3. **Task 3 — Training (Learning):** given observations $\{O\}$, find the model $\lambda = (\boldsymbol{\pi}, \mathbf{A}, \mathbf{B})$ that maximises $P(O \mid \lambda)$.
 - Solved by the **Baum-Welch algorithm** (forward-backward), a special case of EM.

86 Task 1: Computing the Observation Likelihood

86.1 Problem Statement

Given an HMM λ and an observation sequence $O = (O_1, O_2, \dots, O_T)$, determine the likelihood $P(O \mid \lambda)$.

86.2 Naïve Approach

For a *known* hidden state sequence Q , the observation likelihood is:

$$P(O \mid Q) = \prod_{i=1}^T P(o_i \mid q_i)$$

The joint probability of observations and a specific state sequence is:

$$P(O, Q) = P(O \mid Q) \cdot P(Q) = \prod_{i=1}^T P(o_i \mid q_i) \cdot \prod_{i=1}^T P(q_i \mid q_{i-1})$$

The total likelihood requires summing over *all* possible state sequences:

$$P(O) = \sum_Q P(O, Q) = \sum_Q P(O \mid Q) P(Q)$$

Complexity Problem

For an HMM with N hidden states and an observation sequence of length T , there are N^T possible hidden state sequences. This makes the naïve approach **computationally intractable** for any realistic problem.

86.3 The Forward Algorithm

The Forward algorithm computes $P(O \mid \lambda)$ efficiently using **dynamic programming**.

Definition 86.1 (Forward Variable). The forward variable $\alpha_t(j)$ represents the probability of being in state j after seeing the first t observations:

$$\alpha_t(j) = P(o_1, o_2, \dots, o_t, q_t = j \mid \lambda)$$

Forward Algorithm

1. **Initialisation** ($t = 1$):

$$\alpha_1(j) = \pi_j \cdot b_j(o_1), \quad 1 \leq j \leq N$$

2. **Recursion** ($t = 2, \dots, T$):

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) \cdot a_{ij} \right] \cdot b_j(o_t), \quad 1 \leq j \leq N$$

3. **Termination**:

$$P(O \mid \lambda) = \sum_{i=1}^N \alpha_T(i) \cdot a_{i,F}$$

where $a_{i,F}$ is the transition probability to the final (non-emitting) state. If no final state is used, simply $P(O \mid \lambda) = \sum_{i=1}^N \alpha_T(i)$.

How each cell is updated: $\alpha_t(j)$ combines three quantities:

- $\alpha_{t-1}(i)$: the previous forward path probability from state i .
- a_{ij} : the transition probability from state i to state j .
- $b_j(o_t)$: the emission probability of observation o_t given state j .

Complexity Reduction

The Forward algorithm reduces the complexity from $O(N^T)$ (naïve) to $O(T \cdot N^2)$ by storing and reusing intermediate computations instead of recomputing the same subproblems repeatedly for different state sequences.

87 Task 2: Decoding (Viterbi Algorithm)

87.1 Problem Statement

Given an observation sequence O and an HMM λ , find the **most likely hidden state sequence** $Q^* = (q_1^*, q_2^*, \dots, q_T^*)$ that generated O :

$$Q^* = \arg \max_Q P(Q \mid O, \lambda)$$

The naïve approach (evaluating $P(O \mid Q)$ for every possible Q and picking the maximum) has $O(N^T)$ complexity. The **Viterbi algorithm** solves this efficiently.

87.2 The Viterbi Algorithm

Definition 87.1 (Viterbi Variable). The Viterbi variable $v_t(j)$ is the probability of the most likely state sequence ending in state j at time t :

$$v_t(j) = \max_{q_0, q_1, \dots, q_{t-1}} P(q_0, q_1, \dots, q_{t-1}, o_1, \dots, o_t, q_t = j \mid \lambda)$$

Viterbi Algorithm

1. **Initialisation** ($t = 1$):

$$v_1(j) = \pi_j \cdot b_j(o_1), \quad \text{bt}_1(j) = 0$$

2. **Recursion** ($t = 2, \dots, T$):

$$v_t(j) = \max_{i=1}^N [v_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

$$\text{bt}_t(j) = \arg \max_{i=1}^N [v_{t-1}(i) \cdot a_{ij}]$$

where $\text{bt}_t(j)$ is the **backpointer** storing which previous state led to the maximum.

3. **Termination**:

$$P^* = \max_{i=1}^N v_T(i), \quad q_T^* = \arg \max_{i=1}^N v_T(i)$$

4. **Backtrace** ($t = T - 1, \dots, 1$):

$$q_t^* = \text{bt}_{t+1}(q_{t+1}^*)$$

Note. The Viterbi algorithm is **identical to the Forward algorithm** except:

- It takes the **max** over previous path probabilities, whereas the Forward algorithm takes the **sum**.
- It maintains **backpointers** to reconstruct the best state sequence.

While $v_T(j)$ gives the maximum probability for ending in state j , it alone does not store information about the preceding states. The backtracing process recovers the full optimal path.

88 Task 3: Training (Baum-Welch Algorithm)

88.1 Problem Statement

Given an observation sequence O and the set of possible hidden states, **learn** the HMM parameters $\mathbf{A}$ and $\mathbf{B}$.

- **Input:** an unlabelled observation sequence O and a vocabulary of potential hidden states $\mathcal{Q}$.
- The standard algorithm is the **Baum-Welch** (forward-backward) algorithm, a special case of the **EM algorithm**.

88.2 Fully Visible Case (Baseline)

If both the hidden states and observations are known, parameters can be estimated directly by **maximum likelihood**:

Example 88.1. Given the sequence:

Ice creams:	3	3	2	1	1	2	1	2, 3
Weather:	Hot	Hot	Cold	Cold	Cold	Cold	Cold	Hot, Hot

Transition probabilities (ignoring final states):

- $P(\text{Hot} \mid \text{Hot}) = 2/3$, $P(\text{Cold} \mid \text{Hot}) = 1/3$
- $P(\text{Cold} \mid \text{Cold}) = 2/3$, $P(\text{Hot} \mid \text{Cold}) = 1/3$

Emission probabilities:

- $P(1 \mid \text{Hot}) = 0$, $P(2 \mid \text{Hot}) = 0.25$, $P(3 \mid \text{Hot}) = 0.75$
- $P(1 \mid \text{Cold}) = 0.6$, $P(2 \mid \text{Cold}) = 0.4$, $P(3 \mid \text{Cold}) = 0$

However, in the real HMM setting the hidden states are **unknown**, so we cannot use simple counting. We need the Baum-Welch algorithm.

88.3 Backward Probability

Definition 88.1 (Backward Variable). The backward variable $\beta_t(i)$ is the probability of observing the *remaining* observations $o_{t+1}, \dots, o_T$ given that we are in state i at time t :

$$\beta_t(i) = P(o_{t+1}, o_{t+2}, \dots, o_T \mid q_t = i, \lambda)$$

It is computed analogously to the forward algorithm, but moving **backwards in time**.

The backward variable is computed by summing all successive values $\beta_{t+1}(j)$, weighted by their transition probabilities a_{ij} and observation probabilities $b_j(o_{t+1})$.

88.4 Key Quantities for Re-Estimation

88.4.1 Transition Count: $\xi_t(i, j)$

Definition 88.2 ($\xi_t(i, j)$). The probability of being in state i at time t and state j at time $t + 1$, given the entire observation sequence:

$$\xi_t(i, j) = P(q_t = i, q_{t+1} = j \mid O, \lambda)$$

To compute $\xi_t(i, j)$, we first compute the joint probability (the “not-quite- ξ_t ”):

$$P(q_t = i, q_{t+1} = j, O \mid \lambda) = \alpha_t(i) \cdot a_{ij} \cdot b_j(o_{t+1}) \cdot \beta_{t+1}(j)$$

This combines four quantities:

- $\alpha_t(i)$: forward probability up to state i at time t .
- a_{ij} : transition probability from state i to state j .
- $b_j(o_{t+1})$: emission probability of the next observation given state j .
- $\beta_{t+1}(j)$: backward probability from state j at time $t + 1$.

Then normalise:

$$\xi_t(i, j) = \frac{\alpha_t(i) \cdot a_{ij} \cdot b_j(o_{t+1}) \cdot \beta_{t+1}(j)}{P(O \mid \lambda)}$$

where $P(O \mid \lambda) = \sum_{j=1}^N \alpha_t(j) \cdot \beta_t(j)$.

88.4.2 State Occupancy: $\gamma_t(j)$

Definition 88.3 ($\gamma_t(j)$). The probability of being in state j at time t , given the entire observation sequence:

$$\gamma_t(j) = P(q_t = j \mid O, \lambda) = \frac{\alpha_t(j) \cdot \beta_t(j)}{P(O \mid \lambda)}$$

Note. $\gamma_t(j)$ is a degenerate case of ξ_t : $\gamma_t(j) = \sum_{k=1}^N \xi_t(j, k)$.

88.5 Parameter Re-Estimation Formulas

Using ξ_t and γ_t , the parameters are re-estimated as:

Baum-Welch Re-Estimation**Transition probabilities:**

$$\hat{a}_{ij} = \frac{\text{expected number of transitions from } i \text{ to } j}{\text{expected number of transitions from } i} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Emission probabilities:

$$\hat{b}_j(v_k) = \frac{\text{expected number of times in state } j \text{ observing } v_k}{\text{expected number of times in state } j} = \frac{\sum_{t=1}^T \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)}$$

88.6 The Baum-Welch Algorithm (EM for HMMs)**Baum-Welch Algorithm**

1. **Initialise:** set initial estimates for $\mathbf{A}$, $\mathbf{B}$, and $\boldsymbol{\pi}$.
2. **E-step:** compute the expected state occupancy $\gamma_t(j)$ and the expected state transition count $\xi_t(i, j)$ using the current $\mathbf{A}$ and $\mathbf{B}$ (via forward and backward probabilities).
3. **M-step:** use γ and ξ to re-estimate $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$.
4. **Repeat** steps 2–3 until convergence.

Note. Although the Baum-Welch algorithm can in principle perform completely unsupervised learning, in practice the **initial conditions are very important**. For this reason, the algorithm is often given extra information. For example, in speech recognition, the HMM structure is often set by hand, and only the emission ($\mathbf{B}$) and non-zero transition ($\mathbf{A}$) probabilities are trained from data.

89 HMM Summary**HMM Summary**

- HMMs relate a sequence of **observations** to a sequence of **hidden states** that explain the observations.
- **Decoding** (finding the hidden state sequence) is solved by the **Viterbi algorithm**.
- The HMM parameters are $\mathbf{A}$ (transition matrix) and $\mathbf{B}$ (emission matrix).
- Both can be trained with the **Baum-Welch** (forward-backward) algorithm, which is a special case of EM.

Task	Problem	Algorithm	Complexity
Evaluation	$P(O \mid \lambda)$	Forward	$O(T \cdot N^2)$
Decoding	Best state sequence Q^*	Viterbi	$O(T \cdot N^2)$
Training	Learn $\mathbf{A}, \mathbf{B}$	Baum-Welch (EM)	$O(T \cdot N^2)$ per iteration

Algorithm	Key Operation	Output
Forward	$\sum$ (sum over paths)	Observation likelihood
Viterbi	max (best path) + backpointers	Most likely state sequence
Backward	$\sum$ (sum, reverse direction)	Used in Baum-Welch
Baum-Welch	E-step (γ, ξ) + M-step $(\hat{A}, \hat{B})$	Trained model parameters

Part XI

Machine Learning vs. Deep Learning — Recap

90 ML vs DL: Overview

Machine Learning (ML) and Deep Learning (DL) are related but distinct paradigms. Deep Learning is a **subset** of Machine Learning that uses multi-layered neural networks to learn hierarchical representations of data.

Aspect	Machine Learning	Deep Learning
Training dataset size	Small (100s–1000s)	Large (10 000s–millions)
Feature engineering	Manual (domain knowledge)	Automatic (learned by network)
# of classifiers available	Many	Few (architecture families)
Training time	Short	Long

91 ML vs DL: Detailed Comparison

91.1 Data Dependency

- **ML**: performs well with **smaller datasets** (hundreds to thousands of samples).
- **DL**: requires **larger datasets** (tens of thousands to millions) due to the high complexity and vast number of parameters.

91.2 Feature Engineering

- **ML**: relies on **manual feature extraction and selection**. Works best when meaningful features can be crafted using domain knowledge.
- **DL**: **automatically** performs feature extraction via hidden layers in neural networks, with no need for manual feature engineering. Better suited for complex data such as images, videos, or audio.

91.3 Computational Resources

- **ML**: less computationally intensive; can run efficiently on **CPUs**.
- **DL**: highly resource-intensive; typically requires **GPUs** (or TPUs) for training.

91.4 Model Interpretability

- **ML**: often **interpretable** — e.g., decision trees, linear regression, logistic regression.
- **DL**: often a **black box**. Requires additional techniques such as **LIME** and **SHAP** to perform interpretability analysis.

91.5 Training Time

- **ML**: generally **faster** to train.

- **DL**: generally **slower** to train, especially with large models and datasets.

91.6 Problem Complexity

- **ML**: best for problems that do **not require complex representations**.
- **DL**: excels at complex data structures where **hierarchical or abstract feature representation** is necessary.

91.7 Flexibility

- **ML**: more flexible across a wider range of tasks. With minor adjustments, the same algorithm can be applied to regression and classification.
- **DL**: specialised architectures tailored to the task at hand — e.g., **CNNs** for image data, **RNNs** or **Transformers** for sequential data.

91.8 Scalability

- **ML**: may struggle with scalability; when data size grows significantly, training time can increase disproportionately.
- **DL**: performance **improves significantly with more data**; scales better to very large datasets.

ML vs DL — Full Comparison

Criterion	Machine Learning	Deep Learning
Data dependency	Small datasets sufficient	Large datasets required
Feature engineering	Manual	Automatic
Computation	CPU sufficient	GPU/TPU required
Interpretability	Often interpretable	Black box (needs LIME/SHAP)
Training time	Fast	Slow
Problem complexity	Simpler representations	Complex, hierarchical features
Flexibility	General-purpose algorithms	Task-specific architectures
Scalability	Limited	Scales well with data

92 Course Recap

A structured overview of all topics covered in the course.

92.1 Foundations

- Definition of machine learning; types of ML (supervised, unsupervised).
- Applications, error functions, regularisation.
- Overfitting and underfitting.

92.2 Data Cleaning and Preprocessing

- Categorical data to numeric data, handling missing values.
- Data cleaning, outlier removal.
- Feature normalisation.
- Dimensionality reduction (e.g., PCA).
- Feature selection (regression-based, forward feature selection).

92.3 Model Evaluation

- Train / validation / test split; cross-validation.
- Metrics for regression, classification, and clustering.
- Grid search for parameter selection.

92.4 Supervised Learning — Linear Models

- Linear regression, L1/L2 regularised regression.
- Logistic regression, linear SVM, perceptron.

92.5 Supervised Learning — Instance-Based

- K-Nearest Neighbours (K-NN).
- Distance metrics and their impact on performance.

92.6 Supervised Learning — Decision Trees and Ensembles

- CART algorithm, pruning, entropy, information gain.
- Random Forest, bagging, boosting, AdaBoost.

92.7 Support Vector Machines

- Large margin classifiers, SVM for binary classification.
- Kernel trick for non-linear classification.
- Regularisation and the role of the C parameter.

92.8 Unsupervised Learning — Clustering

- K-means, hierarchical clustering, DBSCAN.
- Evaluation metrics: silhouette score, Davies-Bouldin index, etc.

92.9 Unsupervised Learning — Dimensionality Reduction

- PCA, Fisher Discriminant (LDA), t-SNE.

92.10 Probabilistic Models

- Naïve Bayes.
- Gaussian Mixture Models (GMM) and the EM algorithm.
- Hidden Markov Models (HMM).